

Tecnologie del Linguaggio Naturale

Parte III

Appunti e materiali didattici

Docente: Prof. Luigi Di Caro

Anno Accademico 2025/2026

Disclaimer

Il presente materiale didattico costituisce un supporto allo studio e non sostituisce in alcun modo la frequenza delle lezioni né i materiali ufficiali del corso. I contenuti derivano da appunti personali, rielaborazioni e integrazioni basate su fonti didattiche, nonché da strumenti di supporto alla scrittura, inclusi Large Language Models. Il contenuto concettuale è stato verificato e riflette, per quanto possibile, le idee, i concetti e le definizioni presentate dal docente; la forma linguistica è il risultato di una revisione ibrida, manuale e automatizzata. Pur ponendo la massima attenzione all'accuratezza, non si garantisce l'assenza di errori, omissioni o semplificazioni. Si invita pertanto lo studente a utilizzare questo materiale in modo critico e a confrontarlo con le fonti ufficiali del corso. È vietata la riproduzione, modifica o distribuzione, totale o parziale, del presente materiale senza esplicita autorizzazione. L'uso è consentito esclusivamente per fini personali di studio.

Indice

Disclaimer	2
1 Semantica computazionale: tra rappresentazione, uso e generazione del significato	13
1.1 Introduzione	13
1.2 Semantica computazionale: tre prospettive	14
1.2.1 Semantica lessicale: il significato come inventario	14
1.2.2 Semantica formale: il significato come verità	15
1.2.3 Semantica distribuzionale: il significato come uso	17
1.3 Dal significato al Question Answering	19
1.4 Dai sistemi modulari ai modelli generativi	22
1.5 Ambiguità: WSD e WSI	24
1.5.1 Word Sense Disambiguation (WSD)	24
1.5.2 Word Sense Induction (WSI)	26
1.5.3 Valutazione: pseudo-word	27
1.6 Risorse lessico-semantiche	28
1.7 Il ruolo delle definizioni	30
1.8 Conclusione	31
2 Definizioni, semasiologia e ricerca onomasiologica	35
2.1 Introduzione	35
2.2 Le definizioni come oggetti linguistici e computazionali	36
2.3 Tipi di definizioni	37
2.3.1 Definizioni genus-differentia	38
2.3.2 Definizioni per esempio	38
2.3.3 Definizioni per riferimento o rinvio	39
2.3.4 Definizioni per parafrasi, sinonimia o descrizione operativa	39
2.4 Aspetti metodologici	39

2.5	Risorse lessico-semantiche e definizioni	40
2.6	Il problema della qualità definizionale	42
2.7	Valutazione delle definizioni: approcci intrinseci ed estrinseci	43
2.7.1	Valutazione intrinseca	43
2.7.2	Valutazione estrinseca	44
2.7.3	Confronto tra valutazione intrinseca ed estrinseca	45
2.7.4	Limiti delle valutazioni quantitative	46
2.7.5	Best practice: triangolazione delle valutazioni	46
2.7.6	Ruolo nel contesto delle definizioni e dell’NLP	47
2.8	Semasiologia e onomasiologia	47
2.8.1	Approccio semasiologico	48
2.8.2	Approccio onomasiologico	48
2.8.3	Aspetti correlati alla ricerca onomasiologica	49
2.9	Definizioni, retrieval e NLP	49
2.10	Esercizio di laboratorio: complessità definizionale	50
2.11	Esercizio di laboratorio: valutazione delle definizioni (esperimento <i>content-to-form</i>)	54
2.12	Esempi applicativi e laboratorio	56
2.13	Perché tutto questo è importante per l’NLP contemporaneo	57
2.14	Conclusione	58
3	Privacy, rilascio dei dati e disinformazione online	62
3.1	Introduzione	62
3.2	Il rilascio dei dati come problema semantico	63
3.3	La stima del rischio e il ruolo dell’utente	63
3.4	Accesso all’informazione e misinformation	64
3.5	Explainability e fiducia nei sistemi	65
3.6	LLM, RAG e conoscenza esterna	65
3.7	Il legame tra privacy e misinformation	65
3.8	Sperimentazione: annotazione assistita e ruolo del RAG	66
3.8.1	Struttura generale dell’attività	66
3.8.2	Annotazione PRE e POST: una prospettiva comparativa	67
3.8.3	Definizioni operative	68
3.8.4	Valutazione della confidenza e della spiegazione	68

3.8.5	Interpretazione dei risultati	69
3.9	Discussione: annotazione, explainability e user empowerment	69
3.9.1	Esempio di schema di annotazione PRE-RAG-POST	70
3.10	Conclusione	72
3.11	Bibliografia e approfondimenti	73
3.11.1	Il progetto KURAMi: framework concettuale e contributi	73
3.11.2	Riferimenti bibliografici essenziali	74
3.11.3	Risorse online	76
4	Polisemia: complessità, struttura e variazioni multilingue	77
4.1	Introduzione	77
4.2	Dal problema della disambiguazione al problema della complessità	78
4.2.1	La prospettiva tradizionale	78
4.2.2	La polisemia come complessità intrinseca	79
4.3	Dimensioni della polisemia	79
4.3.1	Numero di sensi	79
4.3.2	Distribuzione dei sensi	80
4.3.3	Separabilità semantica	81
4.4	Polisemia e embeddings contestuali	82
4.4.1	Dagli embeddings statici agli embeddings contestuali	82
4.4.2	Cluster di senso	82
4.4.3	Distanza inter-senso	83
4.4.4	Overlap contestuale	83
4.5	Polisemia e variazione multilingue	84
4.5.1	Non esiste una polisemia universale	84
4.5.2	Asimmetrie cross-linguistiche	84
4.5.3	Implicazioni per il NLP multilingue	85
4.6	Una metodologia per misurare la complessità della polisemia	85
4.6.1	Distribuzione dei sensi	85
4.6.2	Separabilità nello spazio vettoriale	86
4.6.3	Visualizzazione in spazi ridotti	87

4.7	Polisemia e qualità dei corpora	87
4.7.1	Distribuzione sbilanciata della complessità	87
4.7.2	Valutazione più realistica	88
4.8	Effetto della complessità polisemica sulla disambiguazione	88
4.9	Polisemia come struttura semantica	89
4.10	Polisemia e Large Language Models	89
4.11	Conclusione	90
	Domande di Riflessione	92
	Idee Progettuali	93
	Bibliografia Essenziale	96
5	Il Problema della Complessità Lessicale	98
5.1	Introduzione	98
5.1.1	Perché il Basic Level è Importante?	98
5.1.2	Il Dinamismo del Vocabolario	99
5.2	L'Origine: Roger Brown e la Domanda Fondamentale . . .	100
5.2.1	"How Shall a Thing be Called?" (1958)	100
5.2.2	Criteri di Brown per il Basic Level	100
5.2.3	Il Contributo di Rosch: Basic Level Categories . . .	101
5.3	Basic Level vs. Middle Level: Collocazione Gerarchica . .	102
5.3.1	La Gerarchia Tassonomica	102
5.3.2	Sequenza di Acquisizione: Dal Base al Superordi- nato e Subordinato	103
5.3.3	Asimmetria nella Generalizzazione	103
5.4	Definizioni Moderne del Basic Level	104
5.4.1	Evoluzione del Concetto: Da Ogden a Oggi	104
5.4.2	Basic Vocabulary per la Sopravvivenza Sociale . . .	105
5.5	Proprietà e Fenomenologia del Basic Level	106
5.5.1	Il Legame Termine-Concetto: Non Sempre 1:1 . . .	106
5.5.2	Soggettività del Basic Level	107
5.5.3	La Definizione di "Advanced" per Esclusione . . .	109
5.6	Applicazioni del Basic Level in NLP	110
5.6.1	Text Simplification	110
5.6.2	Second Language Learning	111
5.6.3	Word Sense Disambiguation (WSD)	112
5.6.4	Machine Translation (MT)	112

5.6.5	Summarization	113
5.6.6	Health Communication e Language Disorders . . .	114
5.7	Metriche e Risorse Computazionali	114
5.7.1	Come Misurare la Basicness	114
5.7.2	Risorse e Dataset	117
5.8	Esperimenti e Annotazione	117
5.8.1	Task di Annotazione: Classificazione Basic/Advanced	117
5.8.2	Modelli Supervised per Basicness Prediction . . .	119
5.8.3	Possibile progetto di laboratorio: Ranking di Synset in WordNet	121
5.9	Sfide Aperte e Direzioni Future	122
5.9.1	Multilingualità	122
5.9.2	Dinamicità Temporale	122
5.9.3	Context-Dependent Basicness	123
5.9.4	Granularità: Oltre la Dicotomia	123
5.9.5	Integrazione in LLM	124
5.10	Conclusioni	124
5.10.1	Ricapitolazione	124
5.10.2	Il Paradosso del Basic Level	125
5.10.3	Implicazioni per NLP	126
	Domande di Riflessione	127
	Idee Progettuali	128
	Bibliografia Essenziale	132
6	Pre-LLM NLP: Dal Text Mining in poi	134
6.1	Introduzione	134
6.1.1	Il Text Mining: Quando i Dati Divennero Protagonisti	135
6.2	Una Storia in Quattro Atti: L'Evoluzione del NLP	136
6.2.1	Atto I: L'Era Simbolica (1950s-1980s) — Il sogno della macchina pensante	136
6.2.2	Atto II: L'Era Statistica (1990s-2010s) — Quando i numeri presero il sopravvento	137
6.2.3	Atto III: L'Era Neurale Pre-Transformer (2010-2017) — Il significato come Geometria	139

6.2.4	Atto IV: L'Era Transformer e LLM (2017-presente) — L'unificazione	140
6.2.5	Il Quinto Atto Ancora da Scrivere	141
6.2.6	Dove Si Colloca il Text Mining?	142
6.3	Dalle Frequenze al Significato: L'Intuizione Distribuzionale	143
6.3.1	“You Shall Know a Word by the Company It Keeps”	144
6.3.2	Dal Vector Space Model agli Embeddings: Un Viaggio Concettuale	144
6.3.3	Cosa Abbiamo Guadagnato, Cosa Abbiamo Perso .	146
6.4	I Task del Text Mining: Cosa Cercavamo di Risolvere . . .	147
6.4.1	Clustering: Scoprire Strutture Nascoste	147
6.4.2	Classificazione: Apprendere da Esempi Etichettati	149
6.4.3	Summarization: L'Essenza del Testo	150
6.4.4	Information Retrieval: Dal Boolean Search al Se- mantic Search	151
6.4.5	Topic Modeling: Scoprire Temi Nascosti	152
6.5	Conclusione: Cosa Abbiamo Imparato, Dove Stiamo An- dando	157
6.5.1	Il Paradosso del Progresso	157
6.5.2	Cosa Gli LLM Hanno Risolto (E Cosa No)	158
6.5.3	La Stratificazione, Non La Sostituzione	159
6.5.4	Quattro Lezioni dal Passato per il Futuro	160
6.5.5	Guardando Avanti: Il Dialogo Tra Generazioni . . .	163
6.6	Laboratorio LAB-4: Topic Modeling con BERTopic	164
6.6.1	Obiettivo	164
6.6.2	Dataset: ArXiv NLP Papers	164
6.6.3	Pipeline Completa	165
6.6.4	Visualizzazione e Validazione	169
6.6.5	Requisiti e Deliverable	170
6.6.6	Suggerimenti Pratici	171
	Domande di Riflessione	172
	Bibliografia Essenziale	173

7 Retrieval-Augmented Generation: tra memoria esterna, retrieval e LLMs	175
7.1 Introduzione	175
7.1.1 Il problema dell'aggiornamento	176
7.2 Dal sapere implicito al sapere accessibile	176
7.3 Information Retrieval: il problema del trovare	177
7.3.1 Approcci classici: TF-IDF e BM25	178
7.4 Semantica distribuzionale e retrieval semantico	179
7.4.1 Modelli per dense retrieval	180
7.5 Architettura RAG: uno schema concettuale	181
7.5.1 Esempio completo	181
7.5.2 Varianti architetturali	183
7.6 Chunking: il problema della granularità della conoscenza	183
7.6.1 Strategie di chunking	184
7.6.2 Metadata enrichment	186
7.7 Vector Databases e scalabilità	186
7.7.1 Approximate Nearest Neighbors (ANN)	187
7.8 Hybrid Search: combinazione livelli lessicale e semantico	187
7.9 RAG come integrazione di paradigmi	188
7.10 Evaluation: misurare la qualità di un sistema RAG	189
7.10.1 Metriche per il retrieval	189
7.10.2 Metriche per la generazione	189
7.10.3 End-to-end evaluation	189
7.11 Limiti e questioni aperte	190
7.11.1 Limiti tecnici	190
7.11.2 Problemi aperti	191
7.12 Direzioni future	191
7.13 Conclusione	191
7.14 Lab-5: Costruire un sistema RAG su articoli scientifici NLP	193
7.14.1 Setup e dipendenze	193
7.14.2 Step 1: Caricamento e preprocessing dei dati	193
7.14.3 Step 2: Generazione degli embeddings	195
7.14.4 Step 3: Creazione del vector store con FAISS	196
7.14.5 Step 4: Implementazione del retriever	196
7.14.6 Step 5: Integrazione con LLM locale	197
7.14.7 Step 6: Sistema RAG completo	199

7.14.8	Estensioni e miglioramenti	201
7.14.9	Conclusioni del laboratorio	202
8	Large Language Models, Prompting e Human-AI Interaction	203
8.1	Introduzione	203
8.1.1	Dai modelli base agli instruction-tuned models	204
8.1.2	Risorse per comprendere i Transformer	205
8.2	Prompting: L'arte di comunicare con gli LLMs	205
8.2.1	Perché il prompting è cruciale	205
8.2.2	Principi fondamentali del prompting	206
8.2.3	Strategie avanzate di prompting	207
8.2.4	Cosa evitare nel prompting	208
8.3	Task comuni con LLMs	209
8.3.1	Summarization	209
8.3.2	Inference e Classificazione	210
8.3.3	Trasformazione	211
8.3.4	Espansione	211
8.4	Limiti e Problematiche degli LLMs	212
8.4.1	Allucinazioni	212
8.4.2	Conoscenza statica	213
8.4.3	Bias e fairness	213
8.4.4	Ragionamento superficiale	214
8.4.5	Context window limitato	215
8.4.6	Costi computazionali	215
8.5	Human-AI Interaction: La nuova frontiera	216
8.5.1	Dal task-oriented al goal-oriented	216
8.5.2	Nuovi pattern di interazione	217
8.5.3	Sfide nella Human-AI Interaction	218
8.5.4	Design principles per interfacce LLM	221
8.5.5	Casi d'uso emergenti	222
8.6	Laboratorio (non obbligatorio): Esperimenti pratici con strategie di prompting	224
8.6.1	Obiettivi	224
8.6.2	Setup: Modelli open-source	224
8.6.3	Setup tecnico	225
8.6.4	Esperimento 1: Matematica semplice	225

8.6.5	Esperimento 2: Classificazione e Sentiment	227
8.6.6	Esperimento 3: Allucinazioni	228
8.6.7	Esperimento 4: Effetto della temperatura	229
8.6.8	Esperimento 5: Robustezza a variazioni prompt	230
8.6.9	Esperimento 6: Limiti di ragionamento	230
8.6.10	Metriche di valutazione	231
8.6.11	Deliverable del laboratorio	231
8.6.12	Domande di riflessione	232
8.7	Conclusioni	232
8.7.1	Takeaway chiave	232
8.7.2	Direzioni future	233
8.7.3	Messaggio finale	233
9	Valutazione per la terza parte del corso di TLN (2025/2026)	234
9.1	La mia idea di valutazione	234
9.2	Struttura dell'esame	234
9.3	Parte I: Teoria e pensiero critico	235
9.3.1	Cosa viene valutato (e perché)	235
9.4	Parte II: Laboratorio e Implementazione	236
9.4.1	Cosa viene valutato (e perché)	236
9.5	Criteri di valutazione: griglia sintetica	238
9.5.1	Parte I: Teoria (circa 50% del voto)	238
9.5.2	Parte II: Laboratorio (circa 50% del voto)	238
10	Conclusioni: Semantica, Conoscenza e Futuro del NLP	239
10.1	Il percorso compiuto	239
10.2	Temi trasversali	241
10.2.1	La tensione tra simbolico e subsimbolico	242
10.2.2	Conoscenza implicita vs esplicita	242
10.2.3	Task-specific vs general-purpose	242
10.2.4	Dati vs modelli	243
10.2.5	Evaluation: il problema della misura	243
10.2.6	Multilingualità e universalità	243
10.3	Sfide aperte	244
10.3.1	Comprensione profonda vs pattern matching	244
10.3.2	Groundedness e riferimento al mondo	244

10.3.3	Ragionamento e composizionalità	245
10.3.4	Long-term memory e learning continuo	245
10.3.5	Efficienza e sostenibilità	245
10.3.6	Robustezza e adversarial attacks	246
10.3.7	Alignment e valori	246
10.4	Direzioni future	246
10.4.1	Agenticità e tool use	247
10.4.2	Multimodalità profonda	247
10.4.3	Personalizzazione e contestualizzazione	247
10.4.4	Neuro-symbolic integration	247
10.4.5	Modelli small e più efficienti	248
10.4.6	Evaluation e benchmarking next-generation	248
10.5	Riflessioni metodologiche	248
10.5.1	Pensiero critico sui sistemi	248
10.5.2	Iterazione e sperimentazione	249
10.5.3	Interdisciplinarietà	249
10.5.4	Consapevolezza storica	249
10.5.5	Responsabilità etica	250
10.6	Il futuro è conversazione	250

1 Semantica computazionale: tra rappresentazione, uso e generazione del significato

1.1 Introduzione

Prima di addentrarsi nei singoli approcci, è utile fare un passo indietro.

Quando parliamo di semantica nel contesto del linguaggio naturale, stiamo parlando di qualcosa che sembra intuitivo — il significato delle parole — ma che diventa estremamente complesso non appena proviamo a formalizzarlo, rappresentarlo o automatizzarlo.

Il problema è antico quanto la filosofia del linguaggio stessa. Già Aristotele, nel *De Interpretatione*, distingueva tra significante (la parola), significato (il concetto mentale) e referente (l'oggetto nel mondo). Questa triangolazione semiotica, ripresa da Ogden e Richards (1923) nel loro celebre "triangolo semiotico", rimane centrale ancora oggi quando costruiamo sistemi computazionali che processano linguaggio.

Nel corso degli anni, la semantica computazionale ha sviluppato diversi modi per affrontare questo problema. Non si tratta semplicemente di tecniche diverse: si tratta di *modi diversi di intendere cosa sia il significato*.

In questa lezione non introduciamo nuovi strumenti. Piuttosto, cerchiamo di mettere ordine: capire cosa abbiamo già visto, come si collega, e dove iniziano i limiti.

1.2 Semantica computazionale: tre prospettive

Una prima semplificazione utile è quella di distinguere tre grandi prospettive: semantica lessicale, semantica formale e semantica statistico-distribuzionale.

Non sono alternative, ma complementari. Ciascuna risponde a domande diverse sul significato e presenta vantaggi e limitazioni specifiche. Come osservato da Sahlgren (2008), non esiste un'unica "teoria del significato", ma piuttosto un insieme di approcci che catturano aspetti diversi di un fenomeno multidimensionale.

1.2.1 Semantica lessicale: il significato come inventario

La semantica lessicale è probabilmente l'approccio più intuitivo. Parte dall'idea che le parole abbiano significati che possono essere descritti, organizzati e messi in relazione tramite una *rete di relazioni paradigmatiche*.

Definizione: La semantica lessicale studia il significato delle parole come entità discrete all'interno di un sistema organizzato di relazioni semantiche (sinonimia, iponimia, meronimia, antonimia, ecc.).

Ad esempio, sappiamo che:

- *cane* e *gatto* sono entrambi *animali* (relazione di iponimia)
- *cane* e *canide* sono in relazione tassonomica
- *zampa* è parte di *cane* (relazione meronimica)
- *bank* può significare sia banca che riva del fiume (polisemia)
- *grande* e *piccolo* sono antonimi

Questo approccio porta naturalmente alla costruzione di risorse come:

- **WordNet** (Miller, 1995): organizza i concetti in synset (insiemi di sinonimi) collegati da relazioni semantiche. Contiene oltre 117.000 synset per l'inglese.
- **BabelNet** (Navigli & Ponzetto, 2012): estende WordNet integrando Wikipedia in oltre 500 lingue.
- **VerbNet** (Kipper et al., 2008): si concentra sulla semantica verbale e sui ruoli tematici.

Esempio concreto: In WordNet, il synset per *dog* include anche *domestic dog*, *Canis familiaris*. Questo synset è collegato tramite iponimia a synset più specifici come *puppy* (cucciolo), *working dog* (cane da lavoro), *toy dog* (cane da compagnia), ciascuno con ulteriori specificazioni (es. *German shepherd*, *poodle*, ecc.).

Il vantaggio è evidente: il significato è leggibile, esplicito, interpretabile. Un sistema può utilizzare queste relazioni per ragionamenti simbolici (es. "se X è un cane, allora X è un mammifero").

Ma emerge subito un problema fondamentale: *chi decide cosa è un significato?*

Questo è noto come il **problema della granularità** (Kilgarriff, 1997): come decidere se due usi di una parola costituiscono due sensi diversi o due sfumature dello stesso senso? WordNet, ad esempio, distingue 10 sensi diversi per il verbo "run", ma questa distinzione è arbitraria e dipende dalle scelte dei lessicografi.

1.2.2 Semantica formale: il significato come verità

La semantica formale nasce con un obiettivo diverso: non descrivere il significato, ma formalizzarlo in modo matematicamente rigoroso.

Definizione: La semantica formale (o semantica logica) definisce il significato di espressioni linguistiche in termini di condizioni di verità, utilizzando la logica matematica come metalinguaggio.

Il linguaggio viene tradotto in formule logiche, e il significato di una frase è definito in termini di condizioni di verità rispetto a un modello del mondo.

Ad esempio:

La frase "Tutti i gatti sono mammiferi" viene formalizzata come:

$$\forall x(Gatto(x) \rightarrow Mammifero(x))$$

E interpretata come: "In ogni mondo possibile (o modello), per ogni individuo x , se x ha la proprietà di essere un gatto, allora x ha la proprietà di essere un mammifero".

Esempio più articolato: Consideriamo la frase "Ogni studente ha letto almeno un libro":

$$\forall x(Studiante(x) \rightarrow \exists y(Libro(y) \wedge Letto(x, y)))$$

Questa rappresentazione permette inferenze rigorose:

- Se Maria è una studentessa, allora esiste almeno un libro che Maria ha letto
- Se non esiste alcun libro che Paolo ha letto, allora Paolo non è uno studente

La semantica formale include diversi approcci:

- **Semantica model-theoretic** (Montague, 1970): interpreta le espressioni linguistiche come funzioni da modelli a valori di verità
- **Discourse Representation Theory** (Kamp, 1981): gestisce fenomeni come anafora e quantificazione in contesto
- **Lambda calculus** (Church, 1940): fornisce un formalismo per compositional semantics

Questa rappresentazione permette inferenze rigorose, ma introduce una separazione netta: non interessa cosa significhi "gatto" nel mondo

reale (il referente concreto), ma come si comporta all'interno del sistema logico.

È un approccio estremamente potente per ragionamento automatico, sistemi di query su database, e verifica formale, ma poco adatto a catturare la ricchezza, la vaghezza e l'ambiguità del linguaggio naturale quotidiano.

Limite fondamentale: Come rappresentare "Questo film è noioso"? O "Maria è piuttosto alta"? La vaghezza, la soggettività e i concetti gradualmente sfuggono alla logica classica, richiedendo estensioni come la fuzzy logic (Zadeh, 1965).

1.2.3 Semantica distribuzionale: il significato come uso

Il terzo approccio rompe completamente con i precedenti.

Non si parte da definizioni o regole, ma dai dati osservabili dell'uso linguistico.

Definizione: La semantica distribuzionale postula che il significato di una parola è determinato dalla distribuzione dei suoi contesti d'uso in un corpus linguistico.

L'idea, formulata da Firth (1957), è semplice ma rivoluzionaria:

You shall know a word by the company it keeps

Il significato emerge dall'uso, non da definizioni a priori.

Harris (1954) aveva già osservato che "parole che occorrono in contesti simili tendono ad avere significati simili" — questa è l'**ipotesi distribuzionale**.

Analizzando grandi quantità di testo, è possibile costruire rappresentazioni vettoriali delle parole (embeddings), in cui la vicinanza tra vettori riflette similarità semantica.

Esempio concreto: Consideriamo le frasi in un corpus:

- "Il *cane* abbaia nel giardino"
- "Il *gatto* miagola nel giardino"
- "Il *cane* gioca con la palla"
- "Il *gatto* dorme sul divano"

Costruendo una matrice di co-occorrenza, osserviamo che:

- *cane* e *gatto* condividono molti contesti: "nel giardino", verbi di azione, articoli, possessivi
- entrambi differiscono da "giardino" o "palla"

Tecniche come Word2Vec (Mikolov et al., 2013), GloVe (Pennington et al., 2014) e i transformer-based embeddings (Devlin et al., 2019) implementano questa idea in modi diversi:

- **Word2Vec** (CBOW e Skip-gram): predice parole dai contesti o contesti dalle parole
- **GloVe**: fattorizza matrici di co-occorrenza globale
- **BERT/ELMo**: embeddings contestuali che cambiano in base al contesto (risolvendo il problema della polisemia)

Esempio algebrico famoso:

$$\vec{king} - \vec{man} + \vec{woman} \approx \vec{queen}$$

Questo mostra che le relazioni semantiche sono catturate geometricamente nello spazio vettoriale.

La similarità viene tipicamente misurata tramite la **similarità coseno**, che considera l'angolo tra vettori:

$$\text{sim}(\vec{a}, \vec{b}) = \frac{\vec{a} \cdot \vec{b}}{|\vec{a}| \cdot |\vec{b}|} = \cos(\theta)$$

Valori vicini a 1 indicano alta similarità; valori vicini a 0 indicano ortogonalità (non correlazione); valori vicini a -1 indicano opposizione.

Questo approccio è alla base di gran parte dell’NLP moderno, ma introduce un problema opposto rispetto alla semantica lessicale: il significato non è più esplicito, ma distribuito in centinaia di dimensioni e difficile da interpretare.

Problema dell’interpretabilità: Cosa significa la dimensione 247 in un embedding di 300 dimensioni? Non c’è una risposta chiara. Tentativi di interpretazione (Mikolov et al., 2013; Fyshe et al., 2015) mostrano che alcune dimensioni catturano features semantiche (animato/inanimato, concreto/astratto), ma la maggior parte resta opaca.

1.3 Dal significato al Question Answering

Per capire perché queste differenze sono importanti, è utile guardare uno dei problemi storici del NLP: il Question Answering (QA).

L’idea originaria dell’AI, già nei sistemi degli anni ’60 come BASEBALL (Green et al., 1961) e LUNAR (Woods, 1973), era permettere una comunicazione naturale tra uomo e macchina: leggere un testo e rispondere a domande su di esso.

Ma le domande non sono tutte uguali.

Consideriamo:

- *Quando è nata Marie Curie?* → domanda fattuale (risposta: 7 novembre 1867)
- *Cos’è un algoritmo?* → domanda definitoria

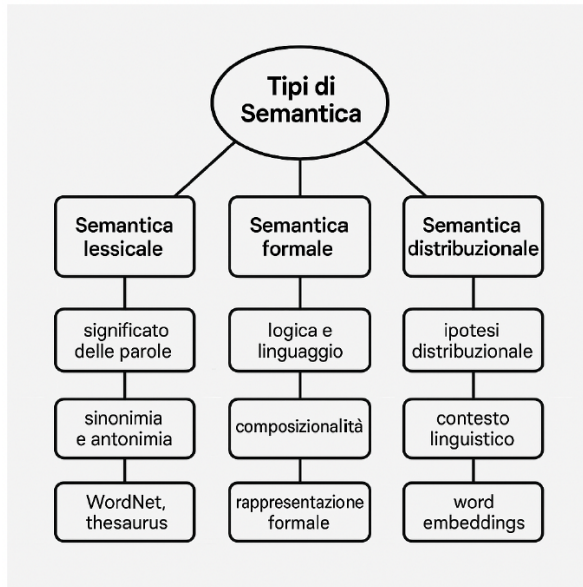


Figura 1.1: Tipi di semantica: complementarità degli approcci lessicale, formale e distribuzionale.

- *Perché il cielo è blu?* → domanda causale (richiede spiegazione fisica)
- *Cosa succederebbe se smettessimo di usare combustibili fossili?* → domanda ipotetica/controfattuale

Queste domande richiedono capacità cognitive diverse:

- **Recupero di fatti** (information retrieval)
- **Definizioni** (semantica lessicale)
- **Spiegazioni causali** (ragionamento inferenziale)
- **Ragionamento ipotetico** (simulazione mentale, modelli del mon-

do)

Per questo motivo, è stato necessario costruire **tassonomie delle domande** (Figura seguente), seguendo classificazioni come quella di Lehnert (1978) o più recentemente di Li & Roth (2002), in modo da associare a ciascuna tipologia una strategia di risposta.

Tassonomia di Li & Roth (2002):

- **ABBR** (abbreviation): Cosa significa "NASA"?
- **ENTY** (entity): Cos'è un transistor?
- **DESC** (description): Come funziona un motore?
- **HUM** (human): Chi ha scritto "1984"?
- **LOC** (location): Dove si trova il Taj Mahal?
- **NUM** (numeric): Quanto è alto l'Everest?

Il punto chiave è che ogni domanda attiva diversi livelli di informazione semantica: lessicale (quale entità?), sintattica (quale relazione?), pragmatica (quale informazione è rilevante dato il contesto?).

Esempio di analisi multi-livello:

Domanda: "Chi ha dipinto la Gioconda?"

- **Livello lessicale:** "Gioconda" = opera d'arte
- **Livello sintattico:** soggetto = X, predicato = dipingere, oggetto = Gioconda
- **Livello semantico:** relazione CREATOR-OF(X, Gioconda)
- **Livello pragmatico:** tipo di risposta attesa = HUMAN (nome proprio)

what		64	54			
	basic what	40	34	MONEY/NUMBER/ DEFINITION/TITLE/ SNF/UNDEFINED	What was the monetary value of the Nobel Peace Prize in 1989?	monetary value
	what-who	7	7	PERSON/ ORGANIZATION	What costume designer decided that Michael Jackson should only wear one glove?	costume designer
	what-when	3	2	DATE	In what year did Ireland elect its first woman president?	year
	what-where	14	12	LOCATION	What is the capital of Uruguay?	capital
who		47	37	PERSON/ ORGANIZATION	Who is the author of the book "The Iron Lady: A Biography of Margaret Thatcher"?	author
how		31	21			
	basic how	1	0	MANNER	How did Socrates die?	Socrates
	how-many	18	13	NUMBER	How many people died when the Titanic sank in 1911?	people
	how-long	2	2	TIME/DISTANCE	How long does it take to travel from Tokyo to Niagara?	-
	how-much	3	2	MONEY/PRICE	How much did Mercury spend on advertising in 1997?	Mercury
	how-much- <modifier>	1	0	UNDEFINED	How much stronger is the new carbon material invented by the Tokyo Institute of Technology compared with the material made from cellulose?	new carbon material
	how-far	1	1	REFERENCE	How far is Yaroslavl from Moscow?	Yaroslavl
	how-tall	3	3	NUMBER	How tall is Bill Gates?	Bill Gates
	how-rich	1	0	UNDEFINED	How rich is Bill Gates?	Bill Gates
	how-large	1	0	NUMBER	How large is the Arctic refuge to preserve unique wildlife and wilderness value on Alaska's north coast?	Arctic refuge
where		22	16	LOCATION	Where is Taj Mahal?	Taj Mahal
when		19	13	DATE	When did the Jurassic Period end?	Jurassic Period
which		10	7			
	which-who	1	1	PERSON	Which former Klu Klux Klan member won an elected office in the U.S.?	former Klu Klux Klan member
	which-where	4	3	LOCATION	Which city has the oldest relationship as sister-city with Los Angeles?	city
	which-when	1	1	DATE	In which year was New Zealand excluded from the ANZUS alliance?	year
	which-what	4	3	SNF/ ORGANIZATION	Which Japanese car maker had its biggest percentage of sale in the domestic market?	Japanese car maker
name		4	4			
	name-who	2	2	PERSON/ ORGANIZATION	Name the designer of the show that spawned millions of plastic imitations, known as "dolls"?	designer
	name-where	1	1	LOCATION	Name a country that is developing a magnetic levitation railway system?	country
	name-what	1	1	TITLE/SNP	Name a film that has won the Golden Bear in the Berlin Film Festival?	film
why		2	0	REASON	Why did David Korsh ask for a	David Korsh

Figura 1.2: Tipi di domande e livelli di complessità semantica richiesti.

1.4 Dai sistemi modulari ai modelli generativi

Storicamente, il QA è stato affrontato tramite **pipeline modulari**:

1. **Classificazione della domanda** (question classification)
2. **Estrazione di entità** (NER - Named Entity Recognition)
3. **Estrazione di relazioni** (relation extraction)
4. **Disambiguazione** (WSD - Word Sense Disambiguation)
5. **Retrieval** (document/passage retrieval)
6. **Answer extraction/generation**

Sistemi come Watson di IBM (Ferrucci et al., 2010), che vinse a Jeopardy! nel 2011, utilizzavano oltre 100 componenti diversi in pipeline,

combinando approcci statistici, simbolici e knowledge-based.

Oggi, con i Large Language Models (GPT-3/4, Claude, LLaMA, ecc.), questo paradigma è cambiato radicalmente.

I modelli non si limitano a recuperare informazioni: **le generano**.

Possono:

- **Sintetizzare** informazioni da fonti diverse
- **Spiegare** concetti complessi in modo adattivo
- **Riformulare** risposte per diversi livelli di competenza
- **Tradurre** non solo tra lingue, ma tra modalità espressive
- **Ragionare** (apparentemente) su catene di inferenze

Esempio di generazione vs retrieval:

Domanda: "Spiega la relatività a un bambino di 8 anni"

Sistema retrieval-based: cercherebbe documenti con "relatività" + "spiegazione semplice" e restituirebbe un passaggio estratto.

LLM generativo: produce una spiegazione originale, adattata al livello richiesto, usando analogie creative: "Immagina di essere su un treno velocissimo..."

Ma questo introduce nuove criticità:

- **Allucinazioni** (hallucinations): generazione di fatti plausibili ma falsi (Maynez et al., 2020; Ji et al., 2023)
- **Difficoltà di controllo:** non è chiaro su quali informazioni si basa la risposta
- **Assenza di grounding:** nessuna garanzia di verità o tracciabilità delle fonti
- **Biases:** amplificazione di bias presenti nei dati di training (Bender

et al., 2021)

Approcci recenti come RAG (Retrieval-Augmented Generation, Lewis et al., 2020) tentano di combinare i due paradigmi: prima recuperare, poi generare basandosi sui documenti recuperati.

1.5 Ambiguità: WSD e WSI

La gestione dell'ambiguità è uno dei problemi centrali della semantica computazionale.

Definizione di ambiguità lessicale: Una parola è ambigua quando può riferirsi a concetti diversi in contesti diversi.

Distinzioni fondamentali:

- **Polisemia:** sensi correlati (es. "libro" = oggetto fisico vs contenuto testuale)
- **Omonimia:** sensi non correlati (es. "banco" = pesce vs mobile)

1.5.1 Word Sense Disambiguation (WSD)

La Word Sense Disambiguation (WSD) assume che esista un inventario predefinito di sensi (tipicamente da WordNet) e che il compito sia scegliere quello corretto dato il contesto.

Definizione formale: Dato un inventario di sensi $S = \{s_1, s_2, \dots, s_n\}$ per una parola w , e un contesto C , WSD consiste nel trovare:

$$s^* = \arg \max_{s \in S} P(s|w, C)$$

Esempio:

Frase: "The bank can guarantee deposits will eventually cover future tuition costs"

La parola "bank" ha multiple interpretazioni in WordNet:

- bank#1: istituto finanziario
- bank#2: riva di fiume
- bank#3: riserva o scorta
- bank#4: banca del sangue

Contesto rilevante: "guarantee", "deposits", "cover", "costs" → Senso corretto: bank#1

Approcci principali:

- **Knowledge-based** (Lesk, 1986): confronto overlap tra contesto e definizioni
- **Supervised** (Zhong & Ng, 2010): classificazione con features contestuali
- **Neural** (Raganato et al., 2017): embeddings contestuali

Ma questo approccio presenta limiti intrinseci:

- **Eccessiva specificità dei sensi**: WordNet distingue sensi che gli umani non percepiscono come diversi (Kilgarrieff, 1997)
- **Copertura incompleta**: nuovi sensi emergono continuamente (es. "viral" nel senso di contenuto virale online)
- **Soggettività**: l'accordo inter-annotatore è spesso basso (70-80%), indicando che la distinzione è arbitraria
- **Domain specificity**: sensi specialistici mancanti

1.5.2 Word Sense Induction (WSI)

La Word Sense Induction (WSI) propone un'alternativa: i sensi non sono dati a priori, ma emergono dai dati tramite clustering non supervisionato.

Definizione: WSI è il compito di scoprire automaticamente i sensi di una parola raggruppando i suoi contesti d'uso in cluster semanticamente coerenti.

Analizzando i contesti di una parola, è possibile raggrupparli in cluster coerenti usando tecniche di clustering (k-means, hierarchical clustering, spectral clustering).

Esempio dettagliato per *play*:

Contesti raccolti da un corpus:

- Cluster 1 (musica): "play guitar", "play piano", "play violin", "play a song"
- Cluster 2 (sport): "play football", "play basketball", "play tennis"
- Cluster 3 (gaming): "play videogames", "play chess", "play cards"
- Cluster 4 (teatro): "play a role", "play Hamlet", "play in a theater"

Ogni gruppo rappresenta un senso indotto automaticamente, senza riferimento a un dizionario esterno.

Algoritmi principali:

- **Context clustering** (Schütze, 1998): clustering di vettori contestuali
- **Graph-based** (Agirre et al., 2006): grafi di co-occorrenza e community detection
- **Topic modeling** (Brody & Lapata, 2009): LDA per sensi come topic latenti
- **Neural clustering** (Amrami & Goldberg, 2019): auto-encoder e

clustering di embeddings

Vantaggi:

- Adattabilità a domini specifici
- Nessuna dipendenza da risorse esterne
- Capacità di scoprire sensi emergenti

Svantaggi:

- Numero di sensi non determinato a priori
- Difficile valutazione (non esiste gold standard)
- Sensi spesso non interpretabili direttamente

1.5.3 Valutazione: pseudo-word

Per valutare questi sistemi, si usa il metodo della **pseudo-word** (Gale et al., 1992; Schütze, 1992).

Idea: Si fondono artificialmente parole non ambigue in una parola artificiale, creando ambiguità controllata.

Procedura:

1. Selezionare due parole non correlate: es. *apple* e *keyboard*
2. Creare pseudo-word: *apple-keyboard*
3. Sostituire ogni occorrenza originale con la pseudo-word
4. Verificare se il sistema riesce a riassegnare correttamente ogni istanza al senso originale

Esempio:

- "I ate an *apple-keyboard*" → dovrebbe essere disambiguato come

apple

- "I typed on the *apple-keyboard*" → dovrebbe essere disambiguato come *keyboard*

Questo approccio è semplice ma efficace, anche se presenta limiti:

- Non riflette perfettamente l'ambiguità reale (sensi correlati vs non correlati)
- I sensi artificiali sono più distanti di quelli naturali
- Inflaziona artificialmente le performance

Valutazioni più realistiche utilizzano dataset annotati come SemEval tasks (Agirre et al., 2007, 2010, 2013).

1.6 Risorse lessico-semantiche

Le risorse disponibili riflettono compromessi diversi lungo diverse dimensioni.

- **WordNet** (Miller, 1995), **BabelNet** (Navigli & Ponzetto, 2012):
 - Alta espressività: relazioni semantiche ricche
 - Bassa scalabilità: costruzione manuale, lenta evoluzione
 - Alta interpretabilità
- **FrameNet** (Baker et al., 1998):
 - Organizza il lessico in "frame semantici" (situazioni)
 - Es. frame COMMERCIAL_TRANSACTION: buyer, seller, goods, money
 - Utile per semantic role labeling

- **PropBank** (Palmer et al., 2005):
 - Annotazione di ruoli semantici verbali
 - Focus su alternanze di diathesis (voce attiva/passiva)
- **ConceptNet** (Speer et al., 2017):
 - Conoscenza di senso comune: "UsedFor", "CapableOf", "AtLocation"
 - Es. (coffee, UsedFor, staying awake), (bird, CapableOf, flying)
 - Crowd-sourced, multi-lingua
- **Property Norms** (McRae et al., 2005):
 - Proprietà percettive e funzionali di concetti
 - Es. "banana": is yellow, is curved, is eaten, grows on trees
 - Basate su dati psicologici sperimentali
- **Embeddings** (Word2Vec, GloVe, BERT, GPT):
 - Alta scalabilità: trainabili su terabytes di testo
 - Bassa interpretabilità: dimensioni latenti non trasparenti
 - Catturano similarità statistica, non necessariamente relazioni logiche

Tendenza recente: Integrazione di risorse simboliche con embeddings (retrofitting: Faruqui et al., 2015; knowledge graph embeddings: Bordes et al., 2013) per ottenere il meglio di entrambi gli approcci.

Risorsa	Potere espressivo	Scalabilità	Sorgente	Ambiguità / Soggettività
Dizionari elettronici	Medio-alto	Medio-bassa	Manuale	Bassa
Property norms	Alto	Bassa	Esperimenti/Crowd	Alta
Frames	Alto	Media	Annotatori esperti	Media
Common-sense knowledge	Molto-alto	Alta	Crowd-sourcing/ML	Molto alta
Visual Attributes	Medio	Medio-alta	Immagini annotate	Medio-alta
Word/sense embeddings	Molto-alto	Molto alta	Testi su larga scala	Alta
Corpus manager	Variabile	Alta	Corpora reali	Media

Figura 1.3: Risorse lessico-semantiche: trade-off tra potere espressivo, scalabilità e interpretabilità.

1.7 Il ruolo delle definizioni

Tutte queste risorse condividono un principio fondamentale: **descrivere un concetto tramite altri concetti**.

Questo principio, noto come **circularità definitoria**, è inevitabile ma problematico.

Consideriamo le definizioni dizionariali:

- *Cane*: mammifero domestico della famiglia dei canidi
- *Mammifero*: animale vertebrato che allatta i propri piccoli
- *Animale*: essere vivente dotato di sensibilità e movimento
- *Essere vivente*: organismo capace di nascere, crescere, riprodursi e morire

La catena definitoria forma un **grafo concettuale** potenzialmente ciclico. La domanda è: dove si ferma la regressione? Esistono "primitive semantiche" (Wierzbicka, 1996)?

Questo solleva domande fondamentali:

- **Cosa significa definire?**

- Definizione per genere e differenza specifica (Aristotele)
- Definizione ostensiva (per esempi)
- Definizione operativa (per procedure)
- **Quali proprietà sono rilevanti?**
 - Proprietà necessarie vs sufficienti (problema della teoria classica dei concetti)
 - Proprietà prototipiche (Rosch, 1973): un pettirosso è più "uccello" di un pinguino
 - Proprietà distintive: cosa differenzia questo concetto da simili?
- **Come distinguere concetti simili?**
 - "Cup" vs "mug" vs "glass": confini sfumati, dipendenti da contesto culturale
 - Minimal pairs: coppie che differiscono per una sola feature critica

Prospettiva computazionale moderna:

I Large Language Models imparano definizioni implicitamente dai pattern d'uso, senza rappresentazioni esplicite. Questo solleva la domanda: un modello che usa correttamente "cane" in migliaia di contesti "comprende" cosa è un cane?

Questa è una variante del **Chinese Room argument** (Searle, 1980): performance linguistica vs comprensione semantica?

1.8 Conclusione

Possiamo ora vedere chiaramente il quadro complessivo:

- **Semantica lessicale:** interpretabile ma rigida
 - Pro: trasparenza, ragionamento simbolico
 - Contro: copertura limitata, arbitrarietà dei confini
- **Semantica distribuzionale:** flessibile ma opaca
 - Pro: scalabilità, apprendimento automatico
 - Contro: black-box, difficoltà di debugging, bias nascosti
- **Modelli generativi:** potenti ma difficili da controllare
 - Pro: versatilità, adattabilità, performance impressionanti
 - Contro: allucinazioni, assenza di grounding, costi computazionali

Il problema del significato non è risolto. È semplicemente cambiato.

Siamo passati da:

- "Come *rappresentare* il significato?" (anni '80-'90)
- a "Come *apprendere* il significato dai dati?" (anni 2000-2010)
- a "Come *generare* comportamento semanticamente appropriato?" (anni 2020+)

Non stiamo più cercando di rappresentare il significato. Stiamo cercando di costruirlo.

E con questa costruzione emergono nuove responsabilità epistemologiche e etiche: cosa significa delegare la costruzione del significato a sistemi opachi? Come garantire che il significato generato sia allineato con valori umani, verità fattuale, e utilità sociale?

Lecture e approfondimenti

Classici:

- Firth, J. R. (1957). A synopsis of linguistic theory 1930-1955.
- Miller, G. A. (1995). WordNet: A lexical database for English.
- Montague, R. (1970). Universal grammar.

Distributional semantics:

- Mikolov, T., et al. (2013). Efficient estimation of word representations in vector space.
- Pennington, J., et al. (2014). GloVe: Global vectors for word representation.
- Devlin, J., et al. (2019). BERT: Pre-training of deep bidirectional transformers.

Question Answering:

- Ferrucci, D., et al. (2010). Building Watson: An overview of the DeepQA project.
- Lewis, P., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks.

Ambiguità e WSD:

- Navigli, R. (2009). Word sense disambiguation: A survey.
- Agirre, E., & Edmonds, P. (2007). Word Sense Disambiguation: Algorithms and Applications.

Questioni filosofiche:

- Searle, J. (1980). Minds, brains, and programs.
- Bender, E., & Koller, A. (2020). Climbing towards NRU: On

1 *Semantica computazionale: tra rappresentazione, uso e generazione del significato*

meaning, form, and understanding in the age of data.

2 Definizioni, semasiologia e ricerca onomasiologica

2.1 Introduzione

Nella lezione precedente abbiamo osservato che la semantica computazionale non coincide con un unico modo di rappresentare il significato, ma con una pluralità di approcci: lessicale, formale, distribuzionale, generativo. Tutti questi approcci, pur nelle loro differenze, condividono una domanda di fondo: *come si describe un concetto?*

Questa domanda sembra semplice solo in apparenza. In realtà, ogni volta che definiamo una parola, stiamo compiendo un'operazione teorica e pratica estremamente delicata: selezioniamo alcune proprietà, ne escludiamo altre, decidiamo cosa è centrale e cosa è periferico, scegliamo un livello di generalità, e costruiamo un piccolo frammento di conoscenza linguisticamente accessibile.

In questo senso, la definizione non è solo uno strumento lessicografico. È una forma di modellizzazione del significato.

Nel contesto della linguistica, della lessicografia e del Natural Language Processing, la nozione di **definizione** riveste quindi un ruolo fondamentale. Una definizione è, in termini generali, un testo progettato per guidare il lettore — o, più in generale, il *consumer* — verso un possibile significato associato a un termine all'interno di un determinato contesto.

Tuttavia, la relazione tra termini e significati non è univoca. Un singolo termine può essere associato a una molteplicità di significati, e uno stesso significato può essere espresso attraverso definizioni differenti,

più o meno accurate, più o meno orientate a uno scopo comunicativo specifico.

Ne deriva che il problema della definizione è anche un problema di:

- selezione delle proprietà rilevanti;
- granularità concettuale;
- controllo dell'ambiguità;
- adeguatezza rispetto al pubblico di riferimento;
- rappresentabilità computazionale.

Questa lezione è dedicata a esplorare proprio questi aspetti. Vedremo come le definizioni possano essere formulate, confrontate, analizzate e valutate; introdurremo la distinzione tra approccio **semasiologico** e **onomasiologico**; e mostreremo come questi temi abbiano ricadute concrete sia nella progettazione di risorse lessico-semantiche sia nello sviluppo di sistemi NLP.

2.2 Le definizioni come oggetti linguistici e computazionali

Quando si pensa a una definizione, si tende a immaginare la voce di un dizionario. Ma da un punto di vista teorico, definire significa molto di più che spiegare una parola: significa stabilire un ponte tra una forma linguistica e una struttura concettuale.

Definizione generale: Una definizione è un enunciato il cui scopo è rendere accessibile un contenuto semantico, specificando proprietà, relazioni o usi di un termine o di un concetto.

Le definizioni svolgono quindi una funzione insieme:

- **descrittiva**, perché forniscono informazioni sul significato;

- **distintiva**, perché separano un concetto da altri concetti simili;
- **comunicativa**, perché devono essere comprensibili a un destinatario;
- **operativa**, perché possono essere utilizzate in compiti di retrieval, classificazione, disambiguazione e generazione.

Da questo punto di vista, la definizione non è mai neutrale. Cambia in funzione dello scopo, del dominio, del livello di specializzazione e persino del tipo di utente previsto.

Esempio semplice: Consideriamo il termine *microscopio*. Possiamo definirlo in modi diversi:

- **Definizione genus-differentia:** strumento ottico usato per osservare oggetti troppo piccoli per essere visti a occhio nudo.
- **Definizione per esempio d'uso:** apparecchio usato in laboratorio per ingrandire cellule, tessuti o microrganismi.
- **Definizione operativa:** dispositivo che consente l'osservazione ad alta risoluzione di strutture microscopiche tramite lenti o altri sistemi di ingrandimento.

Le tre definizioni si riferiscono allo stesso concetto, ma mettono in rilievo aspetti diversi.

2.3 Tipi di definizioni

Esistono diversi **tipi di definizioni**, classificabili in base alla strategia esplicativa adottata. Nessuna tipologia è universalmente migliore delle altre: ogni scelta dipende dal compito e dall'obiettivo.

Tra le principali tipologie troviamo:

2.3.1 Definizioni genus-differentia

Sono il modello classico di derivazione aristotelica. Una definizione identifica anzitutto una categoria generale (*genus*) e poi specifica le proprietà distintive (*differentia*) che separano il concetto dai suoi vicini semantici.

Esempio:

- *Cane*: mammifero domestico della famiglia dei canidi.
- *Triangolo*: poligono con tre lati.

Questo tipo di definizione è molto utile perché:

- esplicita la collocazione tassonomica;
- facilita la classificazione;
- si presta bene alla formalizzazione computazionale.

Tuttavia, non sempre è sufficiente per concetti astratti, prototipici o culturalmente variabili.

2.3.2 Definizioni per esempio

Invece di esplicitare la struttura concettuale, mostrano casi tipici o contesti d'uso.

Esempio:

- *Euristica*: una strategia mentale come “provare la soluzione più semplice per prima” o “seguire una regola approssimativa”.

Queste definizioni sono spesso più accessibili per un pubblico non specialista, ma meno rigorose dal punto di vista classificatorio.

2.3.3 Definizioni per riferimento o rinvio

Descrivono un concetto richiamandone altri già noti.

Esempio:

- *Felino*: animale appartenente alla stessa famiglia del gatto, del leone e della tigre.

Sono utili in sistemi lessicali fortemente interconnessi, ma rischiano di amplificare la circolarità definitoria.

2.3.4 Definizioni per parafrasi, sinonimia o descrizione operativa

Queste strategie puntano a riformulare il significato in modo più accessibile o funzionale.

Esempi:

- *Pericolo*: situazione in cui esiste la possibilità di un danno.
- *Algoritmo*: insieme ordinato di passi per risolvere un problema.

Sono particolarmente frequenti nei contesti educativi e nella comunicazione tecnico-divulgativa.

2.4 Aspetti metodologici

Tutte queste risorse e tutti questi modi di definire si basano, direttamente o indirettamente, sul concetto di *definizione*, cioè sulla descrizione di una parola tramite l'uso di altre parole. Questo meccanismo, sebbene intuitivo, porta con sé alcune riflessioni fondamentali.

- *Come si descrive un concetto?*

- *Quali caratteristiche devono essere menzionate?*
- *Quali proprietà lo distinguono da altri concetti?*
- *Come si scrive una definizione? Chi decide se è “giusta”?*
- *Come si valuta la qualità e la completezza di una definizione?*

Queste domande non sono solo teoriche. Sono centrali per chi costruisce dizionari, thesauri, knowledge graph, risorse terminologiche, dataset per il NLP, o sistemi generativi che devono produrre spiegazioni coerenti.

Altre dimensioni rilevanti sono:

- **Espressività:** quanto una risorsa riesce a rappresentare conoscenze ricche, astratte e relazionali.
- **Scalabilità:** quanto facilmente può essere ampliata automaticamente o semi-automaticamente.
- **Sorgente:** se le informazioni provengono da annotatori esperti, crowd-sourcing, dati testuali, dati visivi o esperimenti cognitivi.
- **Ambiguità e soggettività:** alcune risorse tentano di eliminare l’ambiguità, altre la accolgono come parte costitutiva del linguaggio naturale.

Questi aspetti sono centrali sia per comprendere l’uso di una risorsa in un task NLP, sia per valutarne l’affidabilità.

2.5 Risorse lessico-semantiche e definizioni

Molte delle risorse che incontriamo in semantica computazionale possono essere lette come modi diversi di organizzare definizioni, relazioni o descrizioni di concetti.

- **Dizionari elettronici:** WordNet, BabelNet, e altre risorse analoghe organizzano il lessico tramite synset, glosse e relazioni semantiche.

Risorsa	Potere espressivo	Scalabilità	Sorgente	Ambiguità / Soggettività
Dizionari elettronici	Medio-alto	Medio-bassa	Manuale	Bassa
Property norms	Alto	Bassa	Esperimenti/Crowd	Alta
Frames	Alto	Media	Annotatori esperti	Media
Common-sense knowledge	Molto-alto	Alta	Crowd-sourcing/ML	Molto alta
Visual Attributes	Medio	Medio-alta	Immagini annotate	Medio-alta
Word/sense embeddings	Molto-alto	Molto alta	Testi su larga scala	Alta
Corpus manager	Variabile	Alta	Corpora reali	Media

Figura 2.1: Risorse e dimensioni descrittive su vari aspetti quali il potere espressivo.

- **Risorse linguistico-cognitive:** le *Property Norms* raccolgono proprietà associate ai concetti sulla base di esperimenti cognitivi. Sono utili, ad esempio, per studiare quali caratteristiche vengono spontaneamente considerate salienti.
- **Common-sense knowledge:** risorse come ConceptNet rappresentano conoscenza di senso comune sotto forma di relazioni del tipo *UsedFor*, *CapableOf*, *AtLocation*. Spesso contengono informazione non completamente disambiguata, ma molto utile per compiti applicativi.
- **Visual Attributes:** raccolgono proprietà fisionomiche o percettive degli oggetti, come colore, forma, texture, parti visibili.
- **Corpus managers:** piattaforme come SketchEngine permettono di esplorare grandi corpora e osservare usi reali dei termini, collocazioni e pattern definitori.

Queste risorse differiscono per struttura, granularità, copertura, grado di formalizzazione e interpretabilità. Tuttavia, tutte contribuiscono a una domanda centrale: *quali informazioni ci servono per riconoscere, distinguere e recuperare un concetto?*

2.6 Il problema della qualità definizionale

Una definizione non è utile solo perché è grammaticalmente ben formata. Deve essere anche informativa, discriminante e adeguata a uno scopo.

Criteri possibili di qualità di una definizione:

- **Chiarezza:** è comprensibile senza ulteriori spiegazioni?
- **Accuratezza:** descrive correttamente il concetto?
- **Specificità:** distingue il concetto da altri simili?
- **Completezza:** include le proprietà essenziali?
- **Adeguatezza:** è appropriata per il pubblico o per il task?
- **Disambiguazione:** evita letture alternative fuorvianti?

Esempio: Consideriamo il concetto *pericolo*.

- Definizione A: “qualcosa di brutto che può succedere”
- Definizione B: “situazione o condizione in cui esiste la possibilità di un danno, di una perdita o di una minaccia”

La definizione A è intuitiva ma vaga; la definizione B è più strutturata, più informativa e più adatta a un contesto lessicografico o computazionale.

Un punto fondamentale è che la qualità definizionale dipende anche dal tipo di concetto:

- i concetti **concreti** tendono a essere più facilmente ancorabili a proprietà percettive e funzionali;
- i concetti **astratti** richiedono spesso relazioni, situazioni, effetti, valutazioni o inferenze;
- i concetti **specifici** sono spesso più vincolati;

- i concetti **generici** tendono ad avere confini più sfumati.

2.7 Valutazione delle definizioni: approcci intrinseci ed estrinseci

Nel contesto della semantica computazionale e del Natural Language Processing, la valutazione della qualità di una definizione non può essere considerata un problema puramente intuitivo o qualitativo. Al contrario, richiede metodologie sistematiche che permettano di misurare in modo riproducibile l'efficacia di una rappresentazione semantica.

Due approcci principali vengono comunemente adottati: la **valutazione intrinseca** e la **valutazione estrinseca**. Questi approcci riflettono due prospettive complementari sul significato e sulla sua utilità computazionale.

2.7.1 Valutazione intrinseca

La **valutazione intrinseca** misura la qualità di una definizione o di una rappresentazione semantica considerando esclusivamente le sue proprietà interne, senza fare riferimento a un task applicativo specifico.

In questo caso, l'obiettivo è verificare quanto una definizione sia coerente, informativa e semanticamente allineata al concetto che intende rappresentare.

Tra le metriche più comuni troviamo:

- **Similarità semantica**: ad esempio tramite cosine similarity tra embedding della definizione e embedding del termine definito;
- **Overlap lessicale**: misurato con indici come Jaccard similarity o overlap coefficient;

- **Ricchezza lessicale:** varietà e distribuzione del vocabolario utilizzato;
- **Coerenza interna:** consistenza tra le proprietà espresse nella definizione.

Questo approccio presenta diversi vantaggi:

- è generalmente **veloce e scalabile**;
- non richiede la costruzione o l'esecuzione di task complessi;
- permette una valutazione immediata durante lo sviluppo di risorse.

Tuttavia, presenta anche limiti significativi:

- non misura direttamente l'**utilità pratica** della definizione;
- dipende fortemente dalla **metrica** e dal **modello** utilizzato;
- può assegnare punteggi elevati anche a definizioni poco informative dal punto di vista umano.

2.7.2 Valutazione estrinseca

La **valutazione estrinseca** misura la qualità di una definizione in base al contributo che essa fornisce alla performance di un sistema in un **task applicativo reale**.

In questo caso, la definizione non viene valutata isolatamente, ma come componente di un sistema più ampio.

Esempi di task utilizzati per la valutazione estrinseca includono:

- **Word Sense Disambiguation** (es. algoritmo di Lesk);
- **Question Answering**;

- **Information Retrieval;**
- **Machine Translation;**
- **Text classification.**

I principali vantaggi di questo approccio sono:

- misura l'**utilità concreta** della definizione;
- consente una **validazione empirica** su dati reali;
- riflette il comportamento del sistema in condizioni operative.

D'altra parte, presenta anche alcune criticità:

- è **computazionalmente costoso** e più lento;
- i risultati sono spesso **task-specific**;
- la performance dipende anche da altri componenti del sistema (error propagation).

2.7.3 Confronto tra valutazione intrinseca ed estrinseca

Le due modalità di valutazione possono essere confrontate lungo diverse dimensioni:

Aspetto	Intrinseca	Estrinseca
Velocità	Alta	Bassa
Costo computazionale	Basso	Alto
Necessità di task	No	Sì
Utilità reale	Limitata	Elevata
Dipendenza dal modello	Alta	Media
Generalizzabilità	Bassa	Media

Questo confronto evidenzia come i due approcci rispondano a esigenze diverse e non possano essere considerati alternativi in senso stretto.

2.7.4 Limiti delle valutazioni quantitative

Entrambi gli approcci presentano limiti che devono essere considerati attentamente.

Limiti della valutazione intrinseca

- forte **dipendenza dal modello** utilizzato (es. BERT vs GPT);
- incapacità di valutare la **qualità lessicografica** in senso umano;
- sensibilità al **contesto** nella rappresentazione semantica.

Limiti della valutazione estrinseca

- dipendenza dal **task specifico**;
- influenza del **dataset** e dei suoi bias;
- presenza di **error propagation** dovuta ad altri moduli del sistema.

2.7.5 Best practice: triangolazione delle valutazioni

Alla luce di questi limiti, l'approccio più efficace consiste nell'adottare una strategia di **triangolazione**, che combina diverse modalità di valutazione.

In particolare:

- la **valutazione intrinseca** è utile per uno screening rapido e per lo sviluppo iterativo;

- la **valutazione estrinseca** è fondamentale per la validazione finale;
- l'analisi **qualitativa** permette di interpretare i risultati e comprendere eventuali discrepanze.

Quando i risultati delle diverse valutazioni convergono, è possibile avere maggiore fiducia nella qualità della definizione. Al contrario, divergenze tra le metriche possono indicare la presenza di problemi strutturali o di rappresentazione che meritano un'analisi più approfondita.

2.7.6 Ruolo nel contesto delle definizioni e dell'NLP

Nel quadro più ampio della lezione, la distinzione tra valutazione intrinseca ed estrinseca si collega direttamente al problema della qualità definizionale.

Una definizione non è soltanto una descrizione linguistica, ma una rappresentazione operativa del significato. La sua efficacia dipende dalla capacità di:

- rappresentare in modo adeguato un concetto;
- distinguere tra significati simili;
- supportare processi computazionali come retrieval, classificazione e generazione.

Di conseguenza, la valutazione delle definizioni non è un aspetto accessorio, ma un elemento centrale nella progettazione di risorse semantiche e sistemi NLP.

2.8 Semasiologia e onomasiologia

Nel campo della lessicografia e della rappresentazione del significato, è possibile distinguere due approcci fondamentali alla relazione tra forma

e contenuto: **semasiologico** e **onomasiologico**.

2.8.1 Approccio semasiologico

L'approccio **semasiologico** parte da un termine linguistico e mira a determinarne il significato o i significati possibili. È l'approccio tipico dei dizionari tradizionali: data una forma, si cercano i concetti associati.

Esempio:

- data la parola *penna*, il dizionario può fornire definizioni relative a “strumento per scrivere”, “piuma di uccello”, ecc.

L'operazione va dunque:

forma → contenuto

Questo approccio è centrale in compiti come:

- definizione lessicale;
- disambiguazione di sensi;
- consultazione di dizionari;
- interpretazione di termini ambigui.

2.8.2 Approccio onomasiologico

L'approccio **onomasiologico** procede nel verso opposto. Si parte da un concetto, una definizione o una descrizione, e si cerca di risalire alla forma linguistica che può esprimerlo.

Esempio:

- data la definizione “strumento usato per scrivere con inchiostro”, si cerca di risalire al termine *penna*.

L'operazione va dunque:

contenuto → forma

Si tratta di un problema cruciale non solo in lessicografia, ma anche in NLP, information retrieval, text generation e supporto alla scrittura.

2.8.3 Aspetti correlati alla ricerca onomasiologica

La ricerca onomasiologica si collega a diversi fenomeni e strumenti linguistici rilevanti:

- **Dizionari analogici:** dizionari organizzati per concetti o campi semantici, anziché per forma alfabetica.
- **Tip-of-the-tongue phenomenon:** il fenomeno della “parola sulla punta della lingua”, in cui il parlante dispone del contenuto concettuale ma non riesce a recuperare la forma lessicale appropriata.
- **Meccanismo genus-differentia:** una definizione strutturata tramite categoria generale e tratti distintivi può facilitare la risalita al concetto corretto.

Comprendere la differenza tra semasiologia e onomasiologia è essenziale perché molti sistemi computazionali moderni combinano continuamente i due movimenti:

- capiscono parole e frasi per risalire a concetti;
- generano parole e frasi a partire da rappresentazioni concettuali o istruzioni.

2.9 Definizioni, retrieval e NLP

Il rapporto tra definizioni e NLP è oggi più rilevante che mai.

Nei sistemi classici, una definizione ben costruita poteva essere usata per:

- classificare concetti;
- popolare risorse lessicali;
- facilitare la disambiguazione;
- guidare la ricerca in basi di conoscenza.

Nei sistemi moderni, soprattutto con LLM e approcci retrieval-based, le definizioni hanno assunto anche altre funzioni:

- diventano query semantiche per il recupero di concetti;
- vengono utilizzate come prompt o descrizioni di target concettuali;
- possono essere confrontate automaticamente tramite similarità lessicale o semantica;
- fungono da ponte tra linguaggio naturale e strutture formali come synset, frame o entità in un knowledge graph.

In altre parole, una definizione può essere vista come una rappresentazione linguistica compressa di un contenuto semantico. La sua qualità influenza direttamente la capacità di un sistema di recuperare, classificare o generare in modo appropriato.

2.10 Esercizio di laboratorio: complessità definizionale

[LAB-1]

Misurazione dell'overlap lessicale tra una serie di definizioni per concetti generici/specifici e concreti/astratti.

Questo esercizio ha l'obiettivo di esplorare in modo pratico la complessità insita nella formulazione di definizioni. Attraverso un'attività collaborativa, i partecipanti saranno guidati nella produzione e nell'analisi di definizioni per concetti concreti e astratti, evidenziando le differenze nella rappresentazione linguistica e nella densità semantica.

Fase 1: Creazione condivisa di definizioni

L'attività si articola nella creazione, collettivamente, di definizioni per quattro concetti distinti:

- **Due concetti concreti:**
 - **Uno generico**, ad esempio: *pantalone*
 - **Uno specifico**, ad esempio: *microscopio*
- **Due concetti astratti:**
 - **Uno generico**, ad esempio: *pericolo*
 - **Uno specifico**, ad esempio: *euristica*

Ogni definizione dovrà essere pensata per essere comprensibile a un lettore generico, e potrà seguire strategie diverse: genus-differentia, esempi, parafrasi, descrizioni operative, o combinazioni di queste.

Fase 2: Analisi collettiva

Dopo la redazione delle definizioni, si procederà a una breve analisi condivisa. I partecipanti saranno invitati a riflettere su:

- le scelte linguistiche adottate;
- il grado di specificità;

- le difficoltà incontrate nel definire concetti astratti rispetto a quelli concreti;
- l'uso, oppure l'assenza, di strategie definitorie canoniche.

Fase 3: Task di analisi lessicale e semantica

Come estensione dell'esercizio, verrà assegnato un task di confronto tra le definizioni create. I partecipanti dovranno calcolare:

- il grado di **sovrapposizione lessicale** tra definizioni dello stesso concetto;
- il grado di **sovrapposizione semantica**, attraverso tecniche a scelta (sinonimia, risorse come WordNet, embedding, ecc.).

Questo compito ha lo scopo di stimolare una riflessione critica sulla variabilità definizionale, nonché sull'efficacia delle definizioni nel trasmettere significati in modo condiviso.

Fase 4: Calcolo della similarità lessicale e semantica

In questa fase, l'attenzione si sposta sull'analisi quantitativa delle definizioni prodotte. L'obiettivo è misurare quanto le definizioni condividano elementi linguistici e concettuali, considerando due dimensioni principali: **similarità lessicale (simlex)** e **similarità semantica (simsem)**.

Procedimento:

1. Calcolare la **similarità lessicale (simlex)** tra tutte le definizioni:
 - una possibile formula consiste nel conteggio dei termini in comune tra due definizioni, dopo *stopword removal*, lemmatizzazione e normalizzazione;

- in alternativa si possono usare Jaccard similarity o overlap coefficient.
2. Calcolare la **similarità semantica (simsem)**:
 - una possibile strategia consiste nel rappresentare ciascuna definizione con un vettore e calcolare la *cosine similarity*;
 - il modello semantico può essere scelto liberamente: Word2Vec, GloVe, fastText, BERT, Sentence-BERT, ecc.
 3. Aggregare i valori di **simlex** e **simsem** in base a due dimensioni:
 - **Concretezza**: concetti concreti vs. astratti
 - **Specificità**: concetti generici vs. specifici
 4. Calcolare i valori medi, oppure altre statistiche descrittive, tra le definizioni prese due a due all'interno di ciascun gruppo.

Fase 5: Analisi e riflessione sui risultati

Sulla base dei risultati ottenuti, provate a rispondere alle seguenti domande:

- La **sovrapposizione lessicale** è risultata più alta o più bassa rispetto alle vostre aspettative?
- La **sovrapposizione semantica** riflette meglio la vostra intuizione di somiglianza tra definizioni?
- Avete notato differenze significative tra concetti concreti e astratti, o tra quelli generici e specifici?
- Alcune definizioni vi sono sembrate più prototipiche o più condizionate di altre? Perché?

Questa attività introduce in modo concreto il problema della variabilità definizionale e mostra come strumenti computazionali relativamente

semplici possano aiutare a rendere osservabile il confronto tra testi apparentemente molto simili.

2.11 Esercizio di laboratorio: valutazione delle definizioni (esperimento *content-to-form*)

[LAB-2]

I comuni dizionari a cui siamo abituati partono dalle parole, cioè dalla forma, per arrivare al contenuto. Esistono però alcuni tipi di dizionario, spesso chiamati dizionari analogici, che funzionano “al contrario”: non si ricerca per parola, ma per definizione o per contenuto concettuale. Questo tipo di ricerca viene chiamato **ricerca onomasiologica**.

Dopo aver prodotto e analizzato definizioni per diversi concetti, ci proponiamo ora di sperimentare una procedura di valutazione basata su un approccio **content-to-form**. In altre parole, partiremo dal contenuto concettuale espresso in ciascuna definizione per cercare di risalire alla forma linguistica codificata nei dizionari semantici strutturati.

Obiettivo

Verificare se e quanto le definizioni prodotte siano sufficientemente informative e ben strutturate da permettere l'identificazione del **synset corretto** in WordNet, o in una risorsa equivalente.

Materiale

Le definizioni create nella fase precedente dell'esercizio sulla complessità definizionale.

Procedimento

1. Per ciascun concetto affrontato, raccogliere tutte le definizioni disponibili.
2. Analizzare ogni definizione singolarmente o in modalità aggregata, cercando in WordNet il **synset** che corrisponde meglio al significato espresso.
3. Durante la ricerca:
 - fate attenzione a possibili **ambiguità lessicali**;
 - considerate l'uso di iperonimi e relazioni tassonomiche come guida;
 - utilizzate il principio del **genus** per orientarvi nel grafo semantico.
4. Registrare, per ogni definizione, il synset individuato e motivare la scelta.

Domande guida per la riflessione

- Quanto è stato facile risalire al synset corretto usando solo la definizione?
- Quali definizioni si sono rivelate più efficaci per l'identificazione del concetto?
- Alcune definizioni hanno generato ambiguità o errori di interpretazione? Perché?
- Le definizioni basate su *genus-differentia* si sono dimostrate più utili di altre?

Questo esercizio permette di osservare in modo pratico la relazione tra contenuto informativo delle definizioni e struttura semantica forma-

le, evidenziando il legame tra linguaggio naturale e rappresentazioni lessicali computazionali.

2.12 Esempi applicativi e laboratorio

Come esercizio di consolidamento, singolarmente o a gruppi, si potrà estendere il lavoro dei laboratori precedenti con una delle seguenti tracce:

1. **Integrare risorse eterogenee** (ad esempio WordNet + ConceptNet, oppure FrameNet + embeddings) per valutare la complementarità tra definizioni esplicite, relazioni simboliche e similarità distribuzionale.
2. **Estendere o arricchire una risorsa esistente**, ad esempio migliorando la copertura semantica delle Property Norms o aggiungendo metadati sulle strategie definitorie impiegate.

Come criteri di valutazione, verranno presi in esame i seguenti aspetti:

- corretta comprensione delle risorse e della loro struttura;
- completezza dell'analisi e capacità di confronto tra fonti;
- capacità critica e interpretativa dei risultati;
- qualità delle visualizzazioni esplicative (tabelle, grafi, heatmap);
- codice ben documentato e funzionante;
- presenza di una riflessione finale sull'utilità e sui limiti dell'integrazione delle risorse.

2.13 Perché tutto questo è importante per l’NLP contemporaneo

A prima vista, il problema delle definizioni potrebbe sembrare un tema marginale o tradizionale, lontano dall’NLP contemporaneo dominato da embeddings, transformer e modelli generativi. In realtà, è vero il contrario.

Ogni sistema che:

- deve spiegare un concetto,
- distinguere tra significati simili,
- recuperare un termine a partire da una descrizione,
- generare una risposta definitoria,
- valutare la correttezza semantica di un testo,

si confronta direttamente con il problema della definizione.

Anche i Large Language Models, pur non possedendo definizioni esplicite nel senso tradizionale, mostrano comportamenti che dipendono fortemente da regolarità definizionali apprese dai dati. Quando un modello riesce a completare correttamente una frase come:

Uno strumento usato per osservare oggetti molto piccoli si chiama

sta sfruttando associazioni distribuzionali, pattern definitori e conoscenza lessicale interiorizzata durante l’addestramento.

Questo non significa che il modello “possessa” il significato come una risorsa simbolica, ma mostra che il confine tra definizione esplicita e conoscenza implicita è oggi uno dei punti più interessanti della riflessione semantica.

2.14 Conclusione

Con questa lezione abbiamo spostato l'attenzione da una panoramica generale sulla semantica computazionale a un problema più mirato, ma altrettanto fondamentale: **come si costruisce e si valuta una definizione**.

Abbiamo visto che:

- una definizione non è solo una spiegazione, ma una forma di rappresentazione del significato;
- i diversi tipi di definizione mettono in luce strategie descrittive differenti;
- la qualità definizionale dipende da chiarezza, specificità, adeguatezza e capacità di disambiguazione;
- l'approccio **semasiologico** parte dalla forma per arrivare al contenuto;
- l'approccio **onomasiologico** parte dal contenuto per arrivare alla forma;
- questi problemi hanno applicazioni dirette in lessicografia, retrieval semantico, classificazione e NLP.

Il punto più importante, però, è forse un altro: definire significa sempre scegliere una prospettiva sul concetto. Non esiste quasi mai una definizione perfettamente neutrale, definitiva o completa. Esistono invece definizioni più o meno utili, più o meno efficaci, più o meno adatte a un certo scopo cognitivo o computazionale.

Per questo motivo, studiare le definizioni non significa occuparsi di un residuo teorico della lessicografia tradizionale. Significa affrontare una domanda centrale della semantica: *in che modo il significato può essere reso accessibile, confrontabile e operativamente utilizzabile?*

Domande di riflessione, spunti di discussione e idee di progetto

Esempi di domande possibili

- Quali sono le principali differenze tra approccio semasiologico e approccio onomasiologico?
- Perché una definizione basata su *genus-differentia* può essere utile in un task computazionale?
- Quali differenze emergono nel definire concetti concreti rispetto a concetti astratti?
- In che modo la similarità lessicale e la similarità semantica possono portare a risultati diversi?
- Quali criteri permettono di valutare la qualità di una definizione?
- In che modo una definizione può facilitare o ostacolare l'identificazione di un synset in WordNet?

Spunti di riflessione

Ulteriori spunti:

- Esiste una definizione “giusta” di un concetto, o esistono solo definizioni più o meno utili a seconda dello scopo?
- Le definizioni dei concetti astratti sono intrinsecamente più difficili da costruire e da valutare?
- Quanto pesa la scelta delle parole nella qualità percepita di una definizione?

- Un sistema automatico può valutare la bontà di una definizione senza comprenderne davvero il contenuto?
- La ricerca onomasiologica può essere vista come un precursore di molti sistemi moderni di retrieval semantico?

Idee per lavori di ricerca e proposte di progetti

- Implementazione di un sistema di ranking di definizioni basato su misure di similarità lessicale e semantica.
- Studio comparativo tra definizioni prodotte da esseri umani e definizioni generate da LLM.
- Costruzione di un piccolo sistema onomasiologico che, data una descrizione, suggerisca possibili termini candidati.
- Analisi delle differenze definizionali tra risorse lessicali, corpora e conoscenza di senso comune.
- Studio dell'effetto della struttura *genus-differentia* sull'identificazione corretta di synset o concetti.

Lecture e approfondimenti

Lessicografia e definizioni:

- Wierzbicka, A. (1996). *Semantics: Primes and Universals*.
- Atkins, B. T. S., & Rundell, M. (2008). *The Oxford Guide to Practical Lexicography*.

Semantica lessicale e risorse:

- Miller, G. A. (1995). WordNet: A lexical database for English.

- Navigli, R., & Ponzetto, S. P. (2012). BabelNet: The automatic construction, evaluation and application of a wide-coverage multilingual semantic network.

Risorse cognitive e common-sense knowledge:

- McRae, K., et al. (2005). Semantic feature production norms for a large set of living and nonliving things.
- Speer, R., Chin, J., & Havasi, C. (2017). ConceptNet 5.5: An open multilingual graph of general knowledge.

Ricerca semantica e NLP:

- Turney, P. D., & Pantel, P. (2010). From frequency to meaning: Vector space models of semantics.
- Devlin, J., et al. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding.

3 Privacy, rilascio dei dati e disinformazione online

3.1 Introduzione

Nel contesto contemporaneo del Natural Language Processing, il linguaggio non è più soltanto un oggetto di analisi, ma un punto di intersezione tra sistemi computazionali, pratiche sociali e processi decisionali. Ogni interazione mediata dal linguaggio — dalla scrittura di un messaggio alla formulazione di una query, dalla pubblicazione di un contenuto alla lettura di una notizia — contribuisce alla costruzione di un ambiente informativo complesso, nel quale gli utenti producono e consumano informazioni in modo continuo. All'interno di questo ecosistema emergono due problemi che, pur essendo spesso trattati separatamente, risultano profondamente interconnessi: il **rilascio di dati personali e sensibili** e la **circolazione di contenuti non verificati o fuorvianti**. Da un lato, gli utenti condividono informazioni che possono esporli a rischi di privacy, talvolta senza esserne pienamente consapevoli. Dall'altro lato, si trovano immersi in flussi informativi in cui la qualità dei contenuti è estremamente variabile e in cui la distinzione tra informazione affidabile e misinformation non è sempre immediata. Il punto centrale non è soltanto proteggere ciò che viene comunicato o filtrare ciò che viene ricevuto. Il problema più generale riguarda la possibilità di costruire sistemi che aiutino l'utente a comprendere entrambi i processi. In questo senso, la nozione di **user empowerment** assume un ruolo chiave: l'obiettivo non è sostituire il giudizio umano con decisioni automatiche, ma fornire strumenti che rendano tale giudizio più informato, più consapevole e più controllabile.

3.2 Il rilascio dei dati come problema semantico

Il rilascio dei dati viene spesso interpretato come una questione puramente tecnica, legata alla presenza o all'assenza di determinati attributi identificativi. Tuttavia, questa visione risulta oggi riduttiva. Nei contesti digitali contemporanei, il rischio di un contenuto non dipende esclusivamente da ciò che è esplicitamente dichiarato, ma anche da ciò che può essere inferito. Un testo apparentemente innocuo può contenere elementi che, se combinati con altre informazioni disponibili online, permettono di ricostruire aspetti rilevanti dell'identità o del comportamento di un individuo. In questo senso, il problema della privacy è intrinsecamente semantico: riguarda la relazione tra espressioni linguistiche, contesto e interpretazione. La conseguenza è che non è sufficiente individuare entità sensibili. È necessario comprendere come tali entità si inseriscano in una rete di relazioni e quali inferenze possano essere attivate. Il rischio privacy emerge quindi da un'interazione tra contenuto, contesto e conoscenza esterna. Questa prospettiva porta a superare i modelli binari di accesso, nei quali un dato è semplicemente visibile o nascosto. In molti casi, è più appropriato adottare strategie di rilascio controllato, in cui il contenuto viene trasformato, generalizzato o parzialmente oscurato in modo da ridurre il rischio mantenendo una certa utilità informativa.

3.3 La stima del rischio e il ruolo dell'utente

Una volta riconosciuto che il rischio privacy non è una proprietà statica, ma dipende da molteplici fattori, diventa necessario sviluppare modelli capaci di stimarlo in modo dinamico. Questo implica considerare non solo la presenza di dati sensibili, ma anche il contesto di pubblicazione, le possibilità di propagazione e la combinabilità con altre fonti. In questo quadro, l'utente assume un ruolo centrale. Non si tratta più soltanto di applicare regole predefinite, ma di supportare decisioni situate. Un

sistema efficace dovrebbe quindi fornire indicazioni comprensibili sul rischio associato a un contenuto, esplicitare quali elementi contribuiscono a tale rischio e suggerire possibili strategie di mitigazione. In questo capitolo faremo riferimento a un insieme di nozioni e strumenti sviluppati nell'ambito di un progetto nazionale PRIN, KURAMi¹, che propone un approccio integrato ai problemi del rilascio dei dati personali e della disinformazione online. In particolare, utilizzeremo questo quadro per approfondire i concetti di *privacy release* e *misinformation*, per costruire e analizzare dati in modo collaborativo e, infine, per riflettere sul tema dello *user empowerment* in relazione a soluzioni tecnologiche progettate per mitigare tali fenomeni.

3.4 Accesso all'informazione e misinformation

Se il rilascio dei dati riguarda ciò che l'utente espone, l'accesso all'informazione riguarda ciò a cui l'utente è esposto. In questo ambito, il problema principale non è soltanto recuperare contenuti pertinenti, ma garantire che tali contenuti siano affidabili. Tradizionalmente, i sistemi di Information Retrieval sono stati progettati per massimizzare la rilevanza rispetto a una query. Tuttavia, la rilevanza non coincide necessariamente con la veridicità. Un documento può essere altamente pertinente e allo stesso tempo contenere informazioni inaccurate o fuorvianti. La misinformation introduce quindi una nuova dimensione nella valutazione dei risultati. Non si tratta più soltanto di stabilire se un documento risponde alla query, ma anche se fornisce una rappresentazione adeguata dei fatti. Questo richiede modelli che integrino diverse dimensioni di valutazione, tra cui la coerenza con fonti affidabili, la presenza di evidenze e la qualità della fonte.

¹<https://kurami.disco.unimib.it>

3.5 Explainability e fiducia nei sistemi

Un elemento comune ai problemi di privacy e misinformation è la necessità di rendere le decisioni dei sistemi interpretabili. Quando un sistema segnala un rischio o valuta un contenuto come poco affidabile, l'utente deve poter comprendere le ragioni di tale valutazione. L'*explainability* svolge quindi una funzione cruciale. Non si limita a rendere il sistema più trasparente, ma contribuisce a costruire un rapporto di fiducia basato sulla comprensione. Senza spiegazioni, le decisioni automatiche rischiano di apparire arbitrarie o di essere accettate in modo acritico.

3.6 LLM, RAG e conoscenza esterna

L'introduzione dei Large Language Models ha ampliato significativamente le possibilità di analisi e generazione del linguaggio. Tuttavia, ha anche sollevato nuove questioni legate all'affidabilità delle informazioni prodotte. Uno dei problemi principali è rappresentato dalle *hallucinations*, ovvero la generazione di contenuti plausibili ma non supportati da evidenze. Questo fenomeno è particolarmente critico nei contesti in cui i modelli vengono utilizzati per verificare informazioni o supportare decisioni. Per affrontare questo problema, si è diffuso l'uso di tecniche di **Retrieval-Augmented Generation**, che combinano modelli generativi con sistemi di recupero dell'informazione. In questo modo, la generazione viene ancorata a fonti esterne, migliorando la verificabilità dei contenuti. In un capitolo successivo, guarderemo più da vicino la tecnologia RAG e le sue varianti.

3.7 Il legame tra privacy e misinformation

Un aspetto infine interessante riguarda l'ipotesi di un legame tra rilascio dei dati personali e vulnerabilità alla misinformation. Questa relazione

non è immediata, ma suggerisce che determinati comportamenti di disclosure possano essere associati a una maggiore esposizione a contenuti non verificati. Questo tema solleva questioni teoriche importanti. Se esiste un legame tra ciò che un utente condivide e ciò a cui è esposto, allora privacy e qualità dell'informazione non possono essere trattate come problemi indipendenti. Diventano invece due aspetti di una stessa dinamica informativa.

3.8 Sperimentazione: annotazione assistita e ruolo del RAG

Una componente centrale di questo capitolo riguarda la sperimentazione diretta, finalizzata a mettere in relazione i concetti teorici discussi con un'attività pratica di analisi del testo. In particolare, verrà proposta un'attività di annotazione che ha lo scopo di osservare come il giudizio umano su fenomeni complessi come la privacy violation e la misinformation possa essere influenzato dall'interazione con sistemi automatici basati su Retrieval-Augmented Generation. L'obiettivo non è soltanto classificare contenuti, ma comprendere il processo attraverso cui una valutazione viene costruita, modificata o rafforzata. L'attività è progettata per rendere esplicita la differenza tra una valutazione autonoma e una valutazione supportata da un sistema, permettendo di riflettere sul ruolo dell'explainability e sull'effetto che una spiegazione automatica può avere sul giudizio umano.

3.8.1 Struttura generale dell'attività

L'esercitazione si articola in tre fasi distinte, che devono essere svolte rispettando un ordine preciso. Nella prima fase, ciascun partecipante annota una serie di contenuti basandosi esclusivamente sul testo fornito. In questa fase, non è consentito consultare l'output del sistema RAG e, nel caso della privacy, non è consentito utilizzare fonti esterne. Per

quanto riguarda la misinformation, è possibile effettuare una consultazione minima di fonti informative, limitata alla verifica di fatti, evitando l'uso di modelli generativi. Nella seconda fase, viene reso visibile l'output prodotto dal sistema RAG, che include una classificazione e una spiegazione. Questo output non deve essere modificato, ma solo letto e interpretato. Nella terza fase, i partecipanti sono invitati a rivedere la propria annotazione alla luce dell'informazione fornita dal sistema. In questa fase, possono mantenere la valutazione iniziale, modificarla oppure confermarla con un diverso livello di confidenza. Inoltre, devono valutare la qualità della spiegazione fornita dal sistema, considerando aspetti come chiarezza, pertinenza e completezza.

3.8.2 Annotazione PRE e POST: una prospettiva comparativa

La distinzione tra annotazione PRE e POST costituisce il cuore metodologico dell'esperimento. L'annotazione PRE rappresenta una forma di giudizio non assistito, basato esclusivamente sulla comprensione individuale del testo e sulle conoscenze pregresse. L'annotazione POST, invece, introduce una dimensione riflessiva, in cui il partecipante può confrontare la propria interpretazione con quella proposta dal sistema. Questa struttura consente di osservare diversi fenomeni. In alcuni casi, il sistema può portare a una revisione della classificazione, suggerendo elementi che non erano stati inizialmente considerati. In altri casi, può rafforzare una valutazione già formulata, aumentando il livello di confidenza. In altri ancora, può risultare poco convincente, evidenziando limiti o ambiguità nella spiegazione. In questo senso, l'attività non ha lo scopo di stabilire quale sia la risposta 'corretta', ma di analizzare il processo di costruzione del giudizio e il modo in cui esso viene influenzato da un supporto computazionale.

3.8.3 Definizioni operative

Per garantire una certa uniformità nell'annotazione, vengono adottate definizioni operative precise per i due fenomeni analizzati. Nel caso della privacy, si considera violazione la presenza di informazioni sensibili che potrebbero identificare individui, esporre credenziali o dati finanziari, rivelare comunicazioni riservate o rendere inferibili aspetti personali come abitudini, relazioni o condizioni di salute. L'annotazione è di tipo binario e richiede di stabilire se il contenuto presenti o meno una violazione. Nel caso della misinformation, si adotta una scala ordinale che riflette il grado di distorsione dell'informazione. Questa scala consente di distinguere tra contenuti accurati, contenuti parzialmente fuorvianti, contenuti misti e contenuti prevalentemente o completamente falsi. L'uso di una scala graduata è particolarmente importante, perché riconosce che la misinformation non è un fenomeno binario, ma si distribuisce lungo un continuum.

3.8.4 Valutazione della confidenza e della spiegazione

Oltre alla classificazione, l'attività richiede di esprimere un livello di confidenza associato alla propria annotazione, sia prima sia dopo l'interazione con il sistema. Questo elemento è fondamentale perché permette di distinguere tra cambiamenti nella decisione e cambiamenti nella sicurezza percepita. Parallelamente, viene richiesto di valutare la qualità della spiegazione fornita dal sistema. Questa valutazione si basa su tre dimensioni principali: la chiarezza, cioè quanto la spiegazione sia comprensibile; la pertinenza, cioè quanto sia rilevante rispetto al contenuto; e la completezza, cioè quanto riesca a coprire gli aspetti importanti del caso. Infine, viene richiesto di indicare esplicitamente se e come il sistema ha influenzato la valutazione. Questo passaggio è particolarmente importante perché consente di distinguere tra diverse forme di impatto: cambiamenti nella classificazione, variazioni nella confidenza oppure effetti più sottili che emergono nei commenti qualitativi.

3.8.5 Interpretazione dei risultati

Una volta completata l'annotazione, i dati raccolti permettono di avviare una riflessione su diversi aspetti. È possibile osservare in quali casi il sistema ha avuto un impatto maggiore, se questo impatto riguarda più frequentemente la classificazione o la confidenza, e se esistono differenze tra il dominio della privacy e quello della misinformazione. Inoltre, l'attività consente di valutare indirettamente la qualità delle spiegazioni generate dal sistema. Se una spiegazione è chiara, pertinente e completa, è più probabile che influenzi il giudizio dell'utente. Al contrario, spiegazioni vaghe o poco informative tendono a essere ignorate o a generare incertezza. In questo senso, la sperimentazione non rappresenta soltanto un esercizio di annotazione, ma un dispositivo per osservare l'interazione tra esseri umani e sistemi di supporto basati su linguaggio naturale. Essa rende visibile il modo in cui il giudizio viene costruito, negoziato e talvolta modificato attraverso il dialogo con un sistema automatico.

3.9 Discussione: annotazione, explainability e user empowerment

L'attività di annotazione proposta in questo capitolo permette di collegare in modo concreto tre dimensioni fondamentali: la valutazione del contenuto, la spiegabilità del sistema e il ruolo attivo dell'utente. Attraverso il confronto tra annotazione PRE e POST, emerge chiaramente che il giudizio non è un processo statico, ma una costruzione dinamica che può essere influenzata da nuove informazioni, da spiegazioni e da strumenti di supporto. Questo risultato è particolarmente rilevante nel quadro dello user empowerment. Un sistema non contribuisce all'empowerment semplicemente fornendo una risposta, ma nella misura in cui rende l'utente capace di comprendere, valutare e, se necessario, contestare tale risposta. L'explainability, in questo senso, non è un accessorio, ma una condizione per una interazione più equilibrata tra umano e macchina. La sperimentazione mostra anche che l'influenza del sistema

non è uniforme. In alcuni casi, il sistema può correggere errori evidenti; in altri, può introdurre dubbi o suggerire interpretazioni alternative; in altri ancora, può risultare poco convincente. Questa variabilità è un elemento importante da considerare, perché indica che il rapporto tra sistemi automatici e utenti non è deterministico, ma dipende dalla qualità delle spiegazioni, dalla competenza dell'utente e dal contesto del compito. In definitiva, l'attività proposta rappresenta un esempio concreto di come i concetti introdotti nel capitolo — privacy, misinformation, explainability e knowledge grounding — possano essere messi alla prova in un contesto controllato, permettendo di osservare non solo cosa fa un sistema, ma come le sue decisioni vengono interpretate e utilizzate dagli utenti.

3.9.1 Esempio di schema di annotazione PRE-RAG-POST

Per chiarire la struttura dell'esercitazione, presentiamo un esempio semplificato di annotazione articolata nelle tre fasi. La tabella mostra come una stessa istanza venga analizzata prima senza supporto, poi con il contributo del sistema RAG e infine rivalutata dall'utente.

Interpretazione dello schema L'esempio evidenzia due dinamiche tipiche osservabili durante l'esercitazione. Nel primo caso (P5), la violazione è esplicita e facilmente identificabile già nella fase preliminare; il sistema RAG rafforza la valutazione, aumentando la confidenza dell'annotatore. Nel secondo caso (P8), invece, la violazione è meno evidente perché basata su inferenze contestuali. In questo scenario, il sistema contribuisce a rendere espliciti elementi che non erano stati inizialmente considerati, portando a una revisione della classificazione. Questa differenza mette in luce uno degli aspetti più rilevanti dell'attività: il supporto del sistema non si limita a confermare decisioni già prese, ma può anche rendere visibili forme di rischio meno immediate, legate a informazioni implicite, routine e relazioni personali.

ID	Testo (estratto)	PRE	Conf.	RAG	POST	Conf.	Sintesi spiegazione RAG
P5	Joe, we would like to make an urgent request... Michelle Lokay's extension 57932... hotline number 18000...	Sì	4	Violation	Sì	5	Presenza di nomi propri, numeri inter-ni e contatti telefonici...
P8	Glad you finally made it!... Saturday I have a haircut...	No	3	Violation	Sì	4	Informazioni implicite su relazioni familiari e routine...

Tabella 3.1: Esempio di annotazione PRE-RAG-POST

Estensione dello schema Nel dataset completo utilizzato durante l'esercitazione, ogni istanza include ulteriori campi, tra cui:

- motivazione dell'annotazione nella fase PRE;
- valutazione dettagliata della spiegazione RAG (chiarezza, pertinenza, completezza);
- indicazione esplicita dell'influenza del sistema sulla decisione;
- commenti qualitativi dell'annotatore.

Questi elementi permettono un'analisi più approfondita del processo decisionale e del ruolo delle spiegazioni nel modificare o rafforzare il giudizio umano.

3.10 Conclusione

In questo capitolo abbiamo analizzato il rapporto tra rilascio dei dati, accesso all'informazione e affidabilità dei sistemi. Abbiamo visto come la privacy non possa essere ridotta a una semplice questione di controllo dell'accesso, ma debba essere interpretata come un problema di inferenza e contesto. Allo stesso tempo, abbiamo osservato come la misinformation richieda modelli di valutazione più complessi, capaci di integrare rilevanza e veridicità. Abbiamo inoltre discusso il ruolo dell'explainability e delle tecniche di knowledge grounding, evidenziando come esse siano essenziali per costruire sistemi affidabili. Infine, abbiamo introdotto l'idea di un legame tra privacy e misinformation, che rappresenta una delle direzioni di ricerca più promettenti. Nel loro insieme, questi elementi convergono verso una visione dell'NLP come disciplina non solo tecnica, ma anche epistemica e sociale. Il linguaggio non è soltanto un dato da elaborare, ma un mezzo attraverso cui si costruiscono conoscenza, identità e decisioni. Progettare sistemi che operano sul linguaggio significa quindi intervenire direttamente su questi processi, con tutte le responsabilità che ne derivano."

3.11 Bibliografia e approfondimenti

I temi trattati in questo capitolo si collocano all'intersezione di diverse aree di ricerca, dalla privacy computazionale alla fact-checking automatico, dall'explainable AI ai modelli di linguaggio su larga scala. In questa sezione fornisco indicazioni bibliografiche organizzate per area tematica, con l'obiettivo di offrire punti di partenza per ulteriori approfondimenti.

3.11.1 Il progetto KURAMi: framework concettuale e contributi

Come anticipato nell'introduzione, i concetti e gli strumenti discussi in questo capitolo si inseriscono nel quadro del progetto nazionale PRIN **KURAMi** (*Knowledge-based, explainable User empowerment in Releasing private data and Assessing Misinformation in online environments*), finanziato dal Ministero dell'Università e della Ricerca e coordinato dal sottoscritto (Università degli Studi di Torino) con la collaborazione dall'Università di Milano-Bicocca e dell'Università Statale di Milano.

Il progetto KURAMi si propone di sviluppare un approccio integrato ai problemi della privacy e della misinformation, basato su due pilastri fondamentali:

1. **Rappresentazione della conoscenza:** costruzione di modelli che catturino non soltanto informazioni fattuali, ma anche relazioni argomentative, fonti e gradi di certezza.
2. **User empowerment:** progettazione di sistemi che non sostituiscano il giudizio umano, ma lo supportino attraverso spiegazioni, visualizzazioni e strumenti di controllo.

3.11.2 Riferimenti bibliografici essenziali

Di seguito riporto una selezione ristretta di riferimenti fondamentali, organizzati per area tematica:

Privacy e anonimizzazione

- Sweeney, L. (2002). k-anonymity: A model for protecting privacy. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, 10(05), 557–570.
- Dwork, C. (2006). Differential privacy. *Proceedings of ICALP*, 1–12.
- Lison, P., Pilán, I., Sanchez, D., Batet, M., & Øvrelid, L. (2021). Anonymisation models for text data: State of the art, challenges and future directions. *Proceedings of ACL*, 4188–4203.
- Pilán, I., Lison, P., Øvrelid, L., Papadopoulou, A., Sánchez, D., & Batet, M. (2022). The Text Anonymization Benchmark (TAB): A dedicated corpus and evaluation framework for text anonymization. *Computational Linguistics*, 48(4), 1053–1101.

Misinformation e fact-checking

- Thorne, J., Vlachos, A., Christodoulopoulos, C., & Mittal, A. (2018). FEVER: a large-scale dataset for fact extraction and VERification. *Proceedings of NAACL-HLT*, 809–819.
- Zhou, X., & Zafarani, R. (2020). A survey of fake news: Fundamental theories, detection methods, and opportunities. *ACM Computing Surveys*, 53(5), 1–40.
- Kotonya, N., & Toni, F. (2020). Explainable automated fact-checking for public health claims. *Proceedings of EMNLP*, 7740–7754.

Explainability

- Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). "Why should I trust you?" Explaining the predictions of any classifier. *Proceedings of KDD*, 1135–1144.
- Doshi-Velez, F., & Kim, B. (2017). Towards a rigorous science of interpretable machine learning. *arXiv preprint arXiv:1702.08608*.
- Jacovi, A., & Goldberg, Y. (2020). Towards faithfully interpretable NLP systems: How should we define and evaluate faithfulness? *Proceedings of ACL*, 4198–4205.
- Atanasova, P., Simonsen, J. G., Lioma, C., & Augenstein, I. (2020). Generating fact checking explanations. *Proceedings of ACL*, 7352–7364.

Retrieval-Augmented Generation

- Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Proceedings of NeurIPS*, 9459–9474.
- Guu, K., Lee, K., Tung, Z., Pasupat, P., & Chang, M. (2020). REALM: Retrieval-augmented language model pre-training. *Proceedings of ICML*, 3929–3938.
- Gao, L., Dai, Z., Pasupat, P., et al. (2023). RARR: Researching and revising what language models say, using language models. *Proceedings of ACL*, 16477–16508.

Human-AI interaction e user empowerment

- Shneiderman, B. (2020). Human-centered artificial intelligence: Reliable, safe & trustworthy. *International Journal of Human–Computer Interaction*, 36(6), 495–504.
- Bansal, G., Wu, T., Zhou, J., et al. (2021). Does the whole exceed its parts? The effect of AI explanations on complementary team

performance. *Proceedings of CHI*, 1–16.

3.11.3 Risorse online

Segnalo le seguenti risorse online essenziali:

- **KURAMi Project:** <https://kurami.disco.unimib.it> — Sito ufficiale del progetto, con pubblicazioni, dataset annotati, documentazione tecnica e demo interattive.
- **ACL Anthology:** <https://aclanthology.org> — Archivio delle pubblicazioni nelle principali conferenze di NLP.
- **Papers With Code:** <https://paperswithcode.com> — Piattaforma che collega articoli scientifici a implementazioni e dataset.

4 Polisemia: complessità, struttura e variazioni multilingue

4.1 Introduzione

Nelle lezioni precedenti abbiamo affrontato il problema dell'ambiguità lessicale soprattutto in termini di **disambiguazione**: dato un contesto, quale senso è quello corretto? Questa è la prospettiva classica della Word Sense Disambiguation (WSD), ed è stata centrale nella semantica computazionale degli ultimi decenni.

In questa lezione cambiamo punto di vista.

Non ci chiediamo più soltanto *come scegliere il senso corretto*, ma piuttosto:

Quanto è complesso il sistema dei sensi di una parola? Come si distribuiscono questi sensi nell'uso reale? Quanto sono separabili tra loro? E come varia tutto questo tra lingue diverse?

La polisemia, infatti, non è un fenomeno binario. Non basta dire che una parola è polisemica oppure no. Alcune parole hanno pochi sensi e un significato dominante, altre presentano molti sensi distribuiti in modo relativamente equilibrato; alcune hanno sensi fortemente distinti, altre mostrano confini semantici sfumati.

Possiamo quindi considerare la polisemia come una **proprietà graduale, strutturata e misurabile**. Questo è particolarmente importante nel NLP contemporaneo, perché la complessità polisemica influisce direttamente sulla difficoltà di task come:

- Word Sense Disambiguation;

- Machine Translation;
- Information Retrieval;
- Question Answering;
- uso di embeddings contestuali e Large Language Models.

4.2 Dal problema della disambiguazione al problema della complessità

4.2.1 La prospettiva tradizionale

Nel paradigma classico della WSD, il problema può essere espresso in forma schematica come:

$$(w, C) \longrightarrow s_i$$

dove w è una parola, C è il contesto e s_i è il senso corretto selezionato da un inventario di sensi.

Questa formulazione è utile, ma lascia in ombra una serie di domande importanti:

- perché alcune parole sono molto più difficili da disambiguare di altre?
- il numero di sensi è sufficiente a spiegare la difficoltà?
- conta di più il numero dei sensi o la loro distribuzione?
- i sensi sono sempre chiaramente separabili?

4.2.2 La polisemia come complessità intrinseca

La ricerca più recente suggerisce che la difficoltà della disambiguazione non dipende soltanto dai modelli o dai dataset, ma anche da una proprietà più profonda:

La complessità della polisemia è una proprietà intrinseca della parola e del suo comportamento nell'uso.

Questo significa che alcune parole sono più complesse non semplicemente perché hanno “molti sensi”, ma perché i loro sensi:

- sono distribuiti in modo più equilibrato;
- sono meno facilmente separabili nei contesti reali;
- presentano relazioni semantiche interne articolate;
- variano in modo diverso da lingua a lingua.

4.3 Dimensioni della polisemia

La polisemia non è una singola quantità. Possiamo descriverla almeno lungo tre dimensioni principali.

4.3.1 Numero di sensi

La misura più intuitiva consiste nel contare quanti sensi sono associati a una parola.

$$N(w) = |\{s_1, s_2, \dots, s_n\}|$$

Per esempio:

- *oxygen* o *giraffe* hanno un profilo quasi monosemico;

- *bank* o *paper* hanno pochi sensi ma non banali;
- *run*, *charge*, *set* mostrano una polisemia molto elevata.

Tuttavia, questa misura è insufficiente. Una parola può avere molti sensi nel dizionario, ma usarne abitualmente solo pochi. Inoltre, le risorse lessicali dipendono dalle scelte di granularità: WordNet, per esempio, distingue sensi che non sempre gli umani percepiscono come nettamente separati.

4.3.2 Distribuzione dei sensi

Due parole con lo stesso numero di sensi possono avere una complessità molto diversa.

Consideriamo due casi ipotetici:

- Parola A: due sensi con distribuzione 95% / 5%
- Parola B: due sensi con distribuzione 50% / 50%

La parola B è chiaramente più complessa, perché il sistema deve scegliere tra due interpretazioni entrambe plausibili.

Per misurare questa distribuzione si usa spesso l'**entropia**:

$$H(w) = - \sum_i p(s_i|w) \log p(s_i|w)$$

dove $p(s_i|w)$ rappresenta la probabilità del senso s_i dato il lemma w .

Interpretazione:

- entropia bassa → un senso dominante;
- entropia alta → sensi più bilanciati;
- maggiore entropia → maggiore complessità polisemica.

Esempio intuitivo

- *school*: nella maggior parte dei casi significa “istituzione educativa”, mentre il senso “gruppo di pesci” è molto raro → entropia bassa;
- *bank*: in inglese i sensi “istituto finanziario” e “riva del fiume” possono essere entrambi rilevanti → entropia più alta;
- *run*: presenta numerosi sensi, con una distribuzione più articolata → alta complessità.

4.3.3 Separabilità semantica

La terza dimensione riguarda quanto i sensi siano effettivamente distinguibili.

Non tutti i sensi sono uguali:

- alcuni sono semanticamente lontani;
- altri sono vicini e sfumati;
- altri ancora sono collegati da relazioni regolari, come metafora o metonimia.

Per esempio:

- *bass* “pesce” vs *bass* “strumento musicale” sono sensi molto distanti;
- *paper* “materiale”, “documento”, “articolo” forma una catena di estensioni collegate;
- *book* può indicare sia l’oggetto fisico sia il contenuto informativo.

La polisemia non riguarda solo quanti sensi esistono, ma anche quanto sono vicini o lontani tra loro nello spazio semantico.

4.4 Polisemia e embeddings contestuali

I modelli distribuzionali moderni permettono di osservare la polisemia in modo nuovo.

4.4.1 Dagli embeddings statici agli embeddings contestuali

Nei modelli classici come Word2Vec o GloVe, ogni parola ha un solo vettore. Questo significa che tutti i sensi vengono “compressi” in una singola rappresentazione.

Con i modelli contestuali, come ELMo, BERT o i transformer più recenti, ogni occorrenza di una parola genera invece un embedding diverso a seconda del contesto.

$$V(w) = \{v_1, v_2, \dots, v_n\}$$

dove ogni v_i è la rappresentazione contestuale di un'occorrenza della parola w .

Questo rende possibile osservare la polisemia come **distribuzione geometrica** di punti nello spazio vettoriale.

4.4.2 Cluster di senso

Se i contesti che esprimono sensi diversi sono sufficientemente distinti, i vettori tendono a formare cluster separati.

- sensi ben distinti → cluster separati;
- sensi correlati → cluster vicini;
- sensi sfumati → forte sovrapposizione.

Per esempio:

- *bass* tende a produrre cluster lontani;
- *bank* mostra cluster distinguibili ma non sempre del tutto separati;
- *paper* tende a dare una struttura più continua e sovrapposta.

4.4.3 Distanza inter-senso

Una misura utile consiste nel calcolare la distanza tra i centroidi dei cluster corrispondenti ai diversi sensi:

$$d(s_i, s_j) = 1 - \cos(\vec{s}_i, \vec{s}_j)$$

In modo intuitivo:

- distanza alta $\rightarrow$ sensi ben separati;
- distanza bassa $\rightarrow$ sensi simili o parzialmente sovrapposti.

4.4.4 Overlap contestuale

Un'altra misura importante riguarda la sovrapposizione locale dei contesti.

Anche se due sensi sono separabili in media, possono comunque presentare molti casi borderline. Per questo motivo, la polisemia va osservata non solo in termini di distanza globale, ma anche di **confusione locale** tra occorrenze reali.

4.5 Polisemia e variazione multilingue

Uno degli aspetti più interessanti emersi dagli studi recenti è che la polisemia varia sensibilmente tra lingue diverse.

4.5.1 Non esiste una polisemia universale

Una stessa area semantica può essere lessicalizzata in modi diversi.

L'esempio classico è il seguente:

- inglese: *bank*
- italiano: *banca* / *riva*
- spagnolo: *banco* / *orilla*

In inglese, un'unica forma copre significati che in italiano e spagnolo vengono separati lessicalmente.

4.5.2 Asimmetrie cross-linguistiche

Questa osservazione porta a una conclusione importante:

La polisemia non è solo una proprietà della parola, ma anche del modo in cui una lingua organizza il proprio lessico.

Lingue diverse:

- possono fondere più significati in una sola parola;
- oppure distribuire significati diversi su parole distinte;
- possono avere profili polisemici simili solo parzialmente;
- possono presentare convergenze o divergenze influenzate da vicinanza genealogica, tipologica e culturale.

4.5.3 Implicazioni per il NLP multilingue

Queste asimmetrie hanno effetti diretti su:

- machine translation;
- cross-lingual transfer;
- allineamento di embeddings multilingui;
- rappresentazione dei sensi in risorse come BabelNet o WordNet multilingue.

Un modello addestrato su una lingua può trovarsi in difficoltà quando incontra, in un'altra lingua, un'organizzazione polisemica diversa da quella appresa.

4.6 Una metodologia per misurare la complessità della polisemia

Un punto centrale della nuova prospettiva è la possibilità di costruire una metodologia integrata per descrivere la polisemia lungo più dimensioni.

Possiamo distinguere tre livelli di analisi.

4.6.1 Distribuzione dei sensi

Il primo livello considera la distribuzione empirica dei sensi in corpora annotati.

Le metriche principali sono:

- numero di sensi attestati;
- entropia normalizzata;

- quota del senso più frequente (*top-1 share*).

Queste misure permettono di distinguere:

- parole quasi monosemiche;
- parole con un senso dominante;
- parole con distribuzione più equilibrata;
- parole ad alta complessità polisemica.

4.6.2 Separabilità nello spazio vettoriale

Il secondo livello osserva come i sensi si organizzano nello spazio degli embeddings contestuali.

Le metriche rilevanti includono:

- distanza media tra sensi;
- overlap contestuale;
- coesione dei cluster;
- dispersione intra-senso.

Questo consente di distinguere parole che hanno:

- sensi molti ma facilmente separabili;
- sensi pochi ma molto confondibili;
- strutture polisemiche compatte;
- strutture diffuse e complesse.

4.6.3 Visualizzazione in spazi ridotti

Il terzo livello usa tecniche come UMAP o t-SNE per proiettare gli embeddings in due dimensioni.

Queste visualizzazioni non sostituiscono l'analisi quantitativa, ma la rendono intuitivamente interpretabile:

- cluster compatti → struttura semantica stabile;
- cluster separati → buona distinguibilità dei sensi;
- cluster sovrapposti → polisemia più sfumata;
- strutture diverse tra lingue → differenze nell'organizzazione lessicale.

4.7 Polisemia e qualità dei corpora

La metodologia appena descritta non serve solo a studiare le parole, ma anche a valutare la qualità dei corpora annotati.

4.7.1 Distribuzione sbilanciata della complessità

Molti corpora utilizzati per WSD e analisi semantica mostrano un problema ricorrente:

- contengono molte parole a bassa entropia;
- sottorappresentano i casi realmente complessi;
- privilegiano parole frequenti e relativamente stabili.

Questo ha una conseguenza importante:

Le performance dei sistemi possono essere sovrastimate se il dataset contiene soprattutto parole facili.

4.7.2 Valutazione più realistica

Per questo motivo è utile costruire benchmark più bilanciati, che includano:

- parole a bassa, media e alta entropia;
- parole con diversa separabilità geometrica;
- casi sia monolingui sia multilingui;
- parole con senso dominante e parole con sensi distribuiti in modo più uniforme.

4.8 Effetto della complessità polisemica sulla disambiguazione

Uno degli aspetti più rilevanti è che la complessità misurata con queste metriche non è solo descrittiva, ma anche predittiva.

Se prendiamo un sistema classico di disambiguazione, come Lesk, e confrontiamo parole a bassa e alta complessità, osserviamo che:

- le parole a bassa complessità ottengono accuratezza sensibilmente più alta;
- le parole ad alta complessità sono molto più difficili da trattare;
- la difficoltà non dipende solo dal numero di sensi, ma dalla combinazione di entropia, separabilità e struttura contestuale.

Questo conferma che la polisemia non è soltanto una questione teorica, ma ha un impatto operativo diretto sulle performance dei sistemi NLP.

4.9 Polisemia come struttura semantica

Un errore frequente è pensare alla polisemia come a un problema o a un rumore da eliminare.

In realtà, la polisemia è anche una risorsa cognitiva e linguistica fondamentale.

La polisemia non è un difetto del lessico: è una forma di organizzazione del significato.

Le estensioni polisemiche spesso seguono principi regolari:

- metafora;
- metonimia;
- generalizzazione;
- specializzazione;
- convenzioni culturali e discorsive.

Per esempio:

- *head*: testa, capo, cima;
- *paper*: materiale, documento, articolo;
- *book*: oggetto fisico e contenuto.

Questi sensi non sono arbitrari: riflettono modi sistematici in cui i concetti si estendono e si riorganizzano.

4.10 Polisemia e Large Language Models

I modelli generativi attuali non rappresentano esplicitamente i sensi come nodi discreti, ma apprendono regolarità d'uso distribuite su enormi

quantità di dati.

Questo permette loro di:

- usare spesso correttamente parole polisemiche;
- adattare il significato al contesto;
- produrre parafrasi o spiegazioni coerenti.

Tuttavia, restano problemi aperti:

- difficoltà con sensi rari;
- difficoltà con sensi molto simili;
- instabilità nelle disambiguazioni sottili;
- scarsa trasparenza su come il senso venga effettivamente rappresentato internamente.

In questo senso, la quantificazione della complessità polisemica può diventare anche uno strumento per valutare i limiti dei modelli neurali contemporanei.

4.11 Conclusione

In questa lezione abbiamo spostato l'attenzione da una domanda classica — *quale senso è corretto?* — a una domanda più profonda:

Com'è organizzata la polisemia di una parola, e quanto è complessa?

Abbiamo visto che:

- la polisemia è un fenomeno multidimensionale;
- il numero di sensi non basta a descriverla;
- la distribuzione dei sensi è cruciale e può essere misurata con

l'entropia;

- la separabilità dei sensi nello spazio degli embeddings fornisce un'informazione complementare;
- la polisemia varia tra lingue e non può essere considerata universale;
- la complessità polisemica influenza direttamente le performance dei sistemi NLP;
- le tecniche di visualizzazione e le metriche quantitative permettono di studiare la polisemia in modo sistematico.

Possiamo quindi dire che la polisemia non è soltanto un problema di ambiguità, ma una chiave per comprendere la struttura del significato linguistico.

Studiare la polisemia significa osservare il punto in cui il significato smette di apparire come una lista discreta di etichette e comincia a rivelarsi come uno spazio continuo, dinamico e differenziato tra lingue, usi e contesti.

Domande di Riflessione

1. Una parola con molti sensi è sempre più complessa di una parola con pochi sensi?
2. In che modo l'entropia dei sensi offre una misura più informativa rispetto al semplice conteggio dei sensi?
3. Qual è la differenza tra avere molti sensi e avere sensi molto sovrapposti nello spazio semantico?
4. La polisemia va rappresentata meglio come inventario discreto o come struttura continua?
5. Perché la polisemia varia tra lingue diverse? Dipende solo dal lessico o anche dalla cultura e dall'uso?
6. I Large Language Models rappresentano davvero i sensi oppure apprendono soltanto configurazioni contestuali?

Progetto di laboratorio LAB-3 a scelta tra...

Progetto 1: Misurare la complessità polisemica in un corpus annotato

Obiettivo: calcolare metriche di complessità polisemica su un corpus come SemCor o XL-WSD.

Task:

- estrarre le distribuzioni dei sensi per lemma;
- calcolare numero di sensi, entropia e top-1 share;
- confrontare parole con bassa, media e alta complessità;
- discutere il rapporto tra frequenza e complessità.

Progetto 2: Visualizzare la polisemia con embeddings contestuali

Obiettivo: studiare come i sensi emergano nello spazio degli embeddings.

Task:

- selezionare un insieme di parole target;
- estrarre embeddings contestuali con mBERT o BERT;
- applicare UMAP o t-SNE;
- osservare cluster, sovrapposizioni e dispersione.

Progetto 3: Confronto cross-linguistico

Obiettivo: confrontare la complessità polisemica di lemmi corrispondenti in lingue diverse.

Task:

- selezionare lemmi allineati tramite BabelNet;
- confrontare entropia e numero di sensi in inglese, spagnolo e italiano;
- individuare casi di asimmetria polisemica;
- discutere le implicazioni per machine translation e multilingual NLP.

Progetto 4: Complessità polisemica e performance di disambiguazione

Obiettivo: verificare se la complessità polisemica predice la difficoltà di disambiguazione.

Task:

- applicare un sistema WSD a un insieme di parole con entropia diversa;
- confrontare accuratezza su parole facili e difficili;
- analizzare gli errori per classe di complessità;
- discutere quali metriche sono più predittive.

Progetto 5: Costruire dataset WSD stratificato per quintili di entropia

Benchmark design: Costruire dataset WSD stratificato per quintili di entropia

Obiettivo: usare le metriche di polisemia per selezionare un set di lemmi su cui costruire un nuovo benchmark (generando sinteticamente frasi)

Task:

- calcolare la distribuzione dei lemmi per fasce di entropia;
- selezionare parole ad alta complessità;
- creare prompt adeguati e usare LLM per la creazione di un benchmark sintetico;
- valutare l'accuratezza sul nuovo dataset.

Bibliografia Essenziale

Polisemia e semantica lessicale

- Kilgarriff, A. (1997). *I don't believe in word senses*. Computers and the Humanities, 31(2), 91–113.
- Pustejovsky, J. (1995). *The Generative Lexicon*. MIT Press.
- Apresjan, J. D. (1974). *Regular polysemy*. Linguistics, 142, 5–32.

WSD e complessità dei sensi

- Navigli, R. (2009). *Word sense disambiguation: A survey*. ACM Computing Surveys, 41(2), 1–69.
- McCarthy, D., Koeling, R., Weeds, J., & Carroll, J. (2004). *Finding predominant word senses in untagged text*. ACL.
- Pasini, T., & Navigli, R. (2020). *XL-WSD: An extra-large and cross-lingual evaluation framework for word sense disambiguation*. AAAI.

Risorse lessicali e multilinguismo

- Miller, G. A. (1995). *WordNet: A lexical database for English*. Communications of the ACM, 38(11), 39–41.
- Navigli, R., & Ponzetto, S. P. (2012). *BabelNet: The automatic construction, evaluation and application of a wide-coverage multilingual semantic network*. Artificial Intelligence, 193, 217–250.
- Bond, F., & Foster, R. (2013). *Linking and extending an open multilingual wordnet*. ACL.

Embeddings contestuali e visualizzazione

- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). *BERT: Pre-training of deep bidirectional transformers for language understanding*. NAACL.
- van der Maaten, L., & Hinton, G. (2008). *Visualizing data using t-SNE*. Journal of Machine Learning Research, 9, 2579–2605.
- McInnes, L., Healy, J., & Melville, J. (2018). *UMAP: Uniform manifold approximation and projection for dimension reduction*. arXiv preprint arXiv:1802.03426.

Lavori recenti sulla complessità polisemica

- Xypolopoulos, C., et al. (2021). *Unsupervised measures of polysemy from contextual embeddings*.
- Goel, T., et al. (2022). *Geometric and syntax-aware quantification of polysemy*.
- Di Caro, L., Cataldi, M., & Schifanella, C. (2026). *Beyond Disambiguation: Mapping and Quantifying Multilingual Polysemy Complexity*.

5 Il Problema della Complessità Lessicale

5.1 Introduzione

La ricerca nel campo dell'elaborazione del linguaggio naturale (NLP) si è tradizionalmente concentrata sull'analisi sintattica e grammaticale, relegando lo studio del vocabolario a un ruolo secondario. Tuttavia, questa prospettiva trascura un fatto fondamentale:

Il contenuto semantico è incapsulato nelle parole, che possono avere livelli di complessità radicalmente diversi. Non tutte le parole sono create uguali.

Consideriamo due descrizioni dello stesso oggetto:

Versione 1 (base): “Un *cane* marrone corre nel *parco*.”

Versione 2 (avanzata): “Un *canide domestico* dalla pelliccia *fulva* si muove rapidamente attraverso un'area verde ricreativa urbana.”

Entrambe le frasi descrivono la stessa scena, ma la prima utilizza parole **di base** (*cane, parco, correre*), mentre la seconda impiega termini **avanzati** (*canide domestico, pelliccia fulva, area ricreativa urbana*). Questa differenza non è meramente stilistica: riflette distinzioni cognitive, acquisizionali e comunicative profonde.

5.1.1 Perché il Basic Level è Importante?

Il vocabolario di base è fondamentale per almeno quattro ragioni:

1. **Acquisizione linguistica:** I bambini apprendono prima le parole di base (*cane*) e solo successivamente i loro iperonimi (*animale*) e iponimi (*barboncino*)
2. **Apprendimento L2:** Gli studenti di lingue straniere hanno bisogno di un vocabolario di *sopravvivenza* per comunicare efficacemente
3. **Accessibilità comunicativa:** Testi semplificati per persone con disturbi del linguaggio o bassa alfabetizzazione richiedono un vocabolario controllato
4. **Sistemi NLP:** Machine Translation, Text Simplification, Summarization, Question Answering beneficiano tutti di una comprensione esplicita della complessità lessicale

5.1.2 Il Dinamismo del Vocabolario

A differenza della grammatica (relativamente stabile), il vocabolario è in costante evoluzione:

- **Neologismi:** Nuove parole emergono continuamente (*selfie, click-bait, ghosting*)
- **Shift semantici:** Parole cambiano significato nel tempo (*virale* da biologico a social media)
- **Specializzazione:** Termini tecnici diventano mainstream (*cloud, streaming, podcast*)
- **Variazione sociolinguistica:** Diverse comunità usano vocabolari diversi

Questa dinamicità rende ancora più cruciale identificare un *nucleo stabile* di parole fondamentali.

5.2 L'Origine: Roger Brown e la Domanda Fondamentale

5.2.1 “How Shall a Thing be Called?” (1958)

Nel suo lavoro seminale “*How Shall a Thing be Called?*”, lo psicologo Roger Brown pone una domanda apparentemente semplice ma profondamente complessa:

Data un'entità del mondo reale, quale termine linguistico dovremmo usare per riferirci ad essa?

Consideriamo un esempio concreto. Di fronte a questo oggetto:

[Un'immagine mentale: un cane di razza Golden Retriever]

Potremmo chiamarlo:

- *Animale* (iperonimo, troppo generico)
- *Mammifero* (iperonimo, categoria tassonomica)
- *Cane* (livello base ← **scelta naturale**)
- *Golden Retriever* (iponimo, troppo specifico per uso quotidiano)
- *Rex* (nome proprio, massima specificità)

Osservazione empirica di Brown: In contesti neutri, parlanti nativi convergono sistematicamente verso il *livello base* (*cane*), evitando spontaneamente sia iperonimi troppo vaghi sia iponimi eccessivamente specifici.

5.2.2 Criteri di Brown per il Basic Level

Brown identifica quattro proprietà chiave delle parole di base:

1. **Brevità:** Parole corte, tipicamente 1-2 sillabe
 - *cane* (base) vs *canide domestico* (avanzato)
 - *casa* (base) vs *abitazione residenziale* (avanzato)
2. **Concretezza:** Riferimento a entità percepibili sensorialmente
 - *mela* (base, concreto) vs *malus domestica* (avanzato, botanico)
 - *tavolo* (base, concreto) vs *mobile d'arredo orizzontale* (avanzato, astratto)
3. **Facilità di pronuncia:** Fonologia semplice, assenza di cluster consonantici complessi
 - *gatto* (base) vs *felino domestico* (avanzato)
4. **Alta frequenza:** Uso frequente nella lingua parlata e scritta
 - Analisi di corpora mostrano che le parole di base sono tipicamente nel top 5-10% per frequenza

5.2.3 Il Contributo di Rosch: Basic Level Categories

Eleanor Rosch (1970s) estende il lavoro di Brown dal punto di vista cognitivo, introducendo il concetto di **categorie di livello base**:

Le categorie di livello base sono il livello di astrazione al quale:

- *Si massimizza la distintività percettiva tra categorie*
- *Si massimizza la similarità intra-categoria*
- *Si minimizza lo sforzo cognitivo di categorizzazione*

Esempio empirico di Rosch:

Mostrando immagini di mobili ai partecipanti:

- **Livello superordinato** (*mobile*): riconoscimento lento, pochi attributi distintivi
- **Livello base** (*sedia*): riconoscimento rapido, molti attributi distintivi
- **Livello subordinato** (*sedia da ufficio ergonomica*): riconoscimento lento, attributi ridondanti

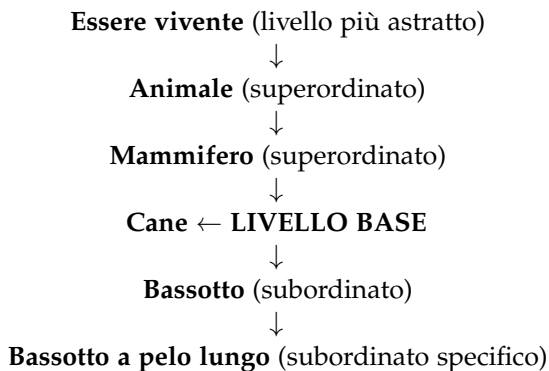
Risultato: I partecipanti categorizzano più velocemente e accuratamente al livello base.

5.3 Basic Level vs. Middle Level: Collocazione Gerarchica

5.3.1 La Gerarchia Tassonomica

Le parole si organizzano naturalmente in gerarchie tassonomiche:

Esempio: Dominio degli animali



Il livello base si colloca tipicamente a metà della gerarchia, da cui il termine alternativo **middle level**.

5.3.2 Sequenza di Acquisizione: Dal Base al Superordinato e Subordinato

Gli studi sullo sviluppo linguistico infantile rivelano un pattern sistematico:

1. **Fase 1 (12-24 mesi):** Acquisizione del livello base
 - “*cane*” per riferirsi a tutti i cani
 - “*palla*” per oggetti sferici
 - “*auto*” per veicoli
2. **Fase 2 (24-36 mesi):** Espansione verso iperonimi
 - Apprendimento di “*animale*” come categoria superiore
 - Comprensione che *cane, gatto, uccello* sono tutti *animali*
3. **Fase 3 (3-5 anni):** Differenziazione subordinata
 - Distinzione tra *bassotto, pastore tedesco, barboncino*
 - Uso di termini più specifici in contesti appropriati

5.3.3 Asimmetria nella Generalizzazione

I bambini mostrano pattern asimmetrici di over-generalizzazione:

Over-extension (verso l’alto): Rara

Un bambino che conosce “*cane*” raramente chiama un gatto “*animale*” invece di cercare un termine più specifico.

Over-extension (verso il basso): Comune

Un bambino che conosce “cane” chiama spesso tutti i cani “cane”, senza distinguere le razze.

Questo suggerisce che il livello base è il **punto di ancoraggio cognitivo** da cui si espandono sia generalizzazioni (verso l’alto) sia specificazioni (verso il basso).

5.4 Definizioni Moderne del Basic Level

5.4.1 Evoluzione del Concetto: Da Ogden a Oggi

Basic English di Ogden (1930)

C.K. Ogden propose un sistema radicale: ridurre l’intero vocabolario inglese a **850 parole fondamentali**.

Principi di Ogden:

- **Minimalismo lessicale:** 850 parole sono sufficienti per esprimere qualsiasi concetto
- **Composizionalità:** Concetti complessi si costruiscono combinando parole base
- **Universalità:** Le parole base sono simili tra lingue diverse

Esempio di parafrasamento in Basic English:

Termine avanzato: “**physician**” (medico)

Basic English: “**person who makes sick people well**” (persona che fa stare bene i malati)

Critiche a Ogden:

- Eccessivamente rigido e prescrittivo
- Non tiene conto della variazione contestuale
- Alcune parafrasi diventano innaturalmente lunghe
- Ignora la dimensione pragmatica (registro, formalità)

Definizioni Contemporanee

La ricerca moderna ha proposto definizioni più sfumate:

Tabella 5.1: Definizioni di Basic Level nella letteratura recente

Approccio	Definizione
Psicolinguistico	Parole apprese precocemente nell'acquisizione L1 e prioritarie nell'apprendimento L2
Cognitivo	Termini che massimizzano la distintività percettiva minimizzando lo sforzo categoriale
Frequenziale	Parole nel top 10% per frequenza in corpora bilanciati
Comunicativo	Lessico necessario per la "sopravvivenza sociale" e comunicazione quotidiana
Computazionale	Parole con alta inter-annotator agreement e bassa ambiguità contestuale

5.4.2 Basic Vocabulary per la Sopravvivenza Sociale

Una definizione particolarmente influente nei contesti di apprendimento L2:

Basic Vocabulary = Insieme minimo di parole necessarie per:

- Soddisfare bisogni primari (cibo, alloggio, salute)
- Interagire in contesti quotidiani (negozi, trasporti, lavoro)
- Comprendere istruzioni e segnaletica essenziali

Esempi di parole per sopravvivenza sociale:

- **Salute:** *medico, ospedale, dolore, medicina, aiuto*
- **Cibo:** *acqua, pane, mangiare, fame, ristorante*
- **Alloggio:** *casa, letto, dormire, bagno, porta*
- **Trasporti:** *autobus, treno, biglietto, fermare, andare*
- **Emergenze:** *polizia, pericolo, perso, chiamare, numero*

Questa prospettiva enfatizza la dimensione **pragmatica** e **funzionale** del basic level.

5.5 Proprietà e Fenomenologia del Basic Level

5.5.1 Il Legame Termine-Concetto: Non Sempre 1:1

Una delle sfide più complesse è che non tutti i concetti hanno un termine di livello base corrispondente in tutte le lingue.

Polisemia e Multi-funzionalità

Parole di base possono riferirsi a concetti avanzati in domini specifici:

Parola: *cane*

Senso base: Mammifero domestico della famiglia dei canidi

Senso avanzato (armeria): “Nelle armi da fuoco, barretta d’acciaio cilindrica che percuote il detonatore provocando l’accensione della carica di lancio e l’espulsione del proiettile”

Questo crea ambiguità:

- *Cane* è basic nel senso zoologico quotidiano
- *Cane* è advanced/tecnico nel dominio armiero

Implicazione: Il basic level è **domain-dependent**.

Lacune Lessicali (Lexical Gaps)

Alcune lingue mancano di termini di livello base per concetti comuni in altre culture:

Esempio: Colori

- Italiano: *blu* (base)
- Russo: *siniy* (blu scuro) vs *goluboy* (blu chiaro) — due termini base distinti
- Alcune lingue amazzoniche: un solo termine per *blu + verde*

Conseguenza: Il basic level NON è universale, ma culturalmente e linguisticamente condizionato.

5.5.2 Soggettività del Basic Level

Studi recenti (Torrielli, Rapp, Di Caro, 2023: “How Shall a Machine Call a Thing?”) hanno dimostrato sperimentalmente che:

La classificazione basic/advanced è soggettiva: annotatori umani mostrano significativa variabilità nel giudicare quali termini siano di base.

Fonti di Variabilità

1. Background educativo:

- Un botanico considera “*quercia*” basic, un non-esperto preferisce “*albero*”
- Un informatico usa “*server*” come basic, altri preferiscono “*computer*”

2. Età:

- Bambini: vocabolario base più ristretto
- Adulti: categoria intermedia più ampia
- Anziani: possibile ritorno a vocabolario più semplice

3. Contesto d’uso:

- Conversazione informale: preferenza per basic
- Scrittura formale: tendenza verso advanced
- Comunicazione specialistica: termini tecnici diventano “basic” nel dominio

4. L1 vs L2:

- Parlanti nativi: intuizione naturale del basic level
- Apprendenti L2: possono sovrastimare la complessità di parole comuni

Implicazioni per l’Annotazione

Quando si costruiscono dataset annotati per basicness:

- **Inter-Annotator Agreement (IAA):** Tipicamente moderato (Cohen’s $\kappa = 0.4-0.6$)

- **Strategie di mitigazione:**

- Usare annotatori multipli (5-10 per item)
- Definire linee guida chiare con esempi
- Calcolare consenso maggioritario
- Identificare “borderline cases” per analisi separate

5.5.3 La Definizione di “Advanced” per Esclusione

A differenza del basic level, i termini **advanced** non hanno una definizione positiva consolidata:

Advanced = $\neg$ **Basic**

Tutto ciò che non è basic è, per definizione, advanced.

Questa definizione negativa è problematica perché:

1. **Assume dicotomia:** Ignora la gradualità (alcune parole sono “moderatamente complesse”)
2. **Manca granularità:** Non distingue tra *tecnico, formale, raro, arcaico*
3. **Non cattura domini:** “*Mitocondrio*” è advanced in generale ma basic in biologia

Proposta alternativa: Modellare basicness come **continuum** piuttosto che dicotomia.

Scala di basicness [0, 1]:

- 1.0: *cane, casa, mangiare, acqua*
- 0.7: *veicolo, abitazione, consumare, liquido*
- 0.4: *quadrupede, domicilio, ingerire, sostanza idrica*

- 0.1: *canide, dimora, deglutire, H₂O*

5.6 Applicazioni del Basic Level in NLP

5.6.1 Text Simplification

La semplificazione testuale è forse l'applicazione più diretta del basic level.

Simplification Pipeline

Input (complesso):

“Il **dermatologo** ha diagnosticato una **dermatite atopica** causata da **allergeni ambientali**.”

Processo di semplificazione:

1. Identificare termini advanced: *dermatologo, dermatite atopica, allergeni ambientali*
2. Sostituire con equivalenti basic:
 - *dermatologo* → *medico della pelle*
 - *dermatite atopica* → *irritazione della pelle*
 - *allergeni ambientali* → *cose nell'aria che causano allergie*

Output (semplificato):

“Il **medico della pelle** ha trovato un'**irritazione della pelle** causata da **cose nell'aria che causano allergie**.”

Sfide nella Simplification

- **Loss of precision:** La semplificazione può perdere informazione
- **Verbosity:** Le parafrasi basic sono spesso più lunghe
- **Naturalezza:** Sostituzioni automatiche possono risultare innaturali
- **Domain adaptation:** In contesti medici, “*dermatologo*” potrebbe essere necessario

5.6.2 Second Language Learning

Il basic vocabulary è fondamentale per l’insegnamento delle lingue straniere.

Curriculum Design

Sequenziamento del vocabolario per livelli:

Tabella 5.2: Vocabolario per livelli CEFR

Livello	Dimensione	Caratteristiche
A1	500-800 parole	Core basic: sopravvivenza quotidiana
A2	1000-1500 parole	Basic esteso: situazioni comuni
B1	2000-3000 parole	Transizione basic-intermediate
B2	4000-5000 parole	Vocabolario semi-advanced
C1	6000-8000 parole	Advanced + idiomatico
C2	10000+ parole	Near-native, inclusi tecnicismi

Adaptive Learning Systems

Sistemi NLP per L2 learning possono:

- **Vocabulary profiling:** Analizzare testi e assegnare livelli di difficoltà
- **Personalized exercises:** Generare esercizi basati sul vocabolario noto
- **Reading assistance:** Glossare automaticamente termini advanced
- **Writing feedback:** Sugerire sostituzioni basic per migliorare chiarezza

5.6.3 Word Sense Disambiguation (WSD)

Il basic level può aiutare la disambiguazione:

Ipotesi: I sensi basic di parole polisemiche sono più frequenti e più facili da disambiguare.

Esempio: “bank”

- *Senso 1 (base):* istituto finanziario (80% occorrenze)
- *Senso 2 (advanced):* riva del fiume (15% occorrenze)
- *Senso 3 (advanced):* inclinazione aereo (5% occorrenze)

Strategia: Dare priorità a sensi basic in assenza di contesto disambiguante.

5.6.4 Machine Translation (MT)

La traduzione può sfruttare la basicness per migliorare la fluenza.

Preservare il Livello di Registro

Inglese → Italiano:

Source (basic): “The **doctor** examined the **patient**.”

Traduzione 1 (preserva basic): “Il **medico** ha visitato il **paziente**.”

Traduzione 2 (eleva registro): “Il **clinico** ha esaminato il **malato**.”

Un sistema MT sensibile al basic level dovrebbe preferire la Traduzione 1 in contesti generali.

Out-of-Vocabulary (OOV) Handling

Quando un sistema MT incontra un termine sconosciuto, può:

1. Identificare se è basic o advanced
2. Se basic: alta priorità per trovare traduzione (probabile gap nel modello)
3. Se advanced: può usare translitterazione o parafrasi esplicativa

5.6.5 Summarization

I riassunti dovrebbero privilegiare vocabolario basic per massimizzare accessibilità.

Documento originale (200 parole):

Testo tecnico con terminologia specialistica

Riassunto standard (50 parole):

Mantiene alcuni termini tecnici, compressione semantica

Riassunto basic-oriented (50 parole):

Sostituisce termini advanced con equivalenti basic, privilegia chiarezza su concisione

5.6.6 Health Communication e Language Disorders

Applicazioni particolarmente critiche dove la basicness è essenziale:

- **Afasia:** Pazienti con disturbi linguistici post-ictus beneficiano di vocabolario semplificato
- **Dislessia:** Parole basic facilitano la decodifica fonologica
- **Comunicazione medico-paziente:** Evitare gergo medico migliora la comprensione
- **Informed consent:** Documenti legali-medici devono essere accessibili

5.7 Metriche e Risorse Computazionali

5.7.1 Come Misurare la Basicness

Diverse metriche sono state proposte per quantificare il basic level:

1. Frequenza

La metrica più semplice e robusta:

$$\text{Basicness}_{\text{freq}}(w) = \frac{\text{count}(w)}{\text{total_tokens}}$$

Normalizzata: $\log(\text{count}(w) + 1)$ per ridurre skewness

Pro:

- Facile da calcolare
- Buona correlazione con giudizi umani

- Risorsa-agnostica (serve solo un corpus)

Contro:

- Dipendenza dal corpus (news vs conversazione)
- Non distingue polisemia (conta tutti i sensi insieme)
- Parole funzionali (*di, a, il*) dominano

2. Age of Acquisition (AoA)

Età media a cui una parola viene appresa:

Dataset AoA (Kuperman et al. 2012):

- *mamma*: AoA = 1.2 anni → molto basic
- *dinosauro*: AoA = 3.5 anni → basic
- *fotosintesi*: AoA = 11 anni → advanced

Formula:

$$\text{Basicness}_{\text{AoA}}(w) = \frac{1}{\text{AoA}(w)}$$

3. Concreteness

Grado di tangibilità/percepibilità:

Scala di Concreteness [1, 5]:

- 5 (massima): *pietra, mela, cane*
- 3 (intermedia): *amore, libertà, scienza*
- 1 (minima): *ontologia, paradigma, epistemologia*

Correlazione: Parole concrete tendono a essere basic.

4. Word Length

Lunghezza in caratteri o sillabe:

$$\text{Basicness}_{\text{len}}(w) = \frac{1}{\text{len}(w)}$$

Osservazione empirica: Parole basic sono tipicamente 4-6 caratteri in italiano/inglese.

5. Semantic Neighborhood Density

Numero di sinonimi/termini correlati:

Parola basic (*cane*):

Molti sinonimi/iponimi: *can, cagnolino, cucciolo, bastardo, randagio, bassotto, ...*

→ Alta densità semantica

Parola advanced (*coleottero*):

Pochi sinonimi generici: *insetto, scarafaggio*

→ Bassa densità semantica

6. Composite Score

Combinare metriche multiple:

$$\text{Basicness}(w) = \alpha \cdot \log(\text{freq}) + \beta \cdot \frac{1}{\text{AoA}} + \gamma \cdot \text{concr} + \delta \cdot \frac{1}{\text{len}}$$

Dove $\alpha, \beta, \gamma, \delta$ sono pesi appresi da annotazioni umane.

5.7.2 Risorse e Dataset

Tabella 5.3: Risorse per Basic Level Research

Risorsa	Descrizione
WordNet	Organizzazione gerarchica permette di identificare livelli tassonomici
BabelNet	WordNet multilingue, utile per studi cross-linguistici
AoA Norms	Dataset con età di acquisizione per migliaia di parole
Concreteness Ratings	Valutazioni umane di concretezza per ~40k parole inglesi
SUBTLEX	Frequenze basate su sottotitoli (più rappresentative del parlato)
CEFR Vocabulary Lists	Liste ufficiali per apprendimento L2
Simple Wikipedia	Versione semplificata di Wikipedia con vocabolario controllato

5.8 Esperimenti e Annotazione

5.8.1 Task di Annotazione: Classificazione Basic/Advanced

Un compito comune è chiedere ad annotatori umani di classificare parole come basic o advanced.

Protocollo di Annotazione

1. Selezione stimoli:

- Campionare parole da fasce di frequenza diverse
- Bilanciare parti del discorso (sostantivi, verbi, aggettivi)
- Includere sia parole concrete che astratte

2. Linee guida per annotatori:

“Una parola è BASIC se:

- *La useresti parlando con un bambino di 6 anni*
- *È una delle prime 1000 parole che impareresti in una lingua straniera*
- *Può essere spiegata senza usare termini tecnici*

Altrimenti, è ADVANCED.”

3. Formato di annotazione:

- Scala binaria: BASIC / ADVANCED
- Scala Likert: 1 (molto advanced) ... 5 (molto basic)
- Scala continua: slider 0-100

4. Calcolo Agreement:

Cohen's Kappa per 2 annotatori:

$$\kappa = \frac{P_o - P_e}{1 - P_e}$$

Dove:

- P_o = proporzione di accordo osservato
- P_e = proporzione di accordo atteso per caso

Interpretazione:

- $\kappa < 0.2$: poor agreement
- $0.2 \leq \kappa < 0.4$: fair
- $0.4 \leq \kappa < 0.6$: moderate
- $0.6 \leq \kappa < 0.8$: substantial
- $\kappa \geq 0.8$: almost perfect

Risultati tipici: $\kappa = 0.45$ - 0.65 per basicness (moderate agreement)

5.8.2 Modelli Supervised per Basicness Prediction

Data un'annotazione gold-standard, si possono addestrare classificatori:

Features

- **Lessicali:**
 - Lunghezza parola (caratteri, sillabe)
 - Presenza di affissi derivazionali (-zione, -ità, -mento)
 - POS tag
- **Corpus-based:**
 - Log-frequenza
 - Document frequency
 - Distribuzione su registri (formale vs informale)
- **Psicolinguistiche:**
 - Age of Acquisition

- Concreteness rating
- Imageability
- **Semantiche (da WordNet):**
 - Profondità nella gerarchia
 - Numero di iperonimi/iponimi
 - Numero di sensi (polisemia)
- **Embeddings:**
 - Word2Vec/FastText vectors
 - BERT contextualized embeddings (media su contesti)

Modelli

1. **Logistic Regression:** Baseline interpretabile
2. **Random Forest:** Cattura interazioni non-lineari
3. **Gradient Boosting (XGBoost):** Tipicamente le migliori performance
4. **Neural Networks:**
 - MLP su features concatenate
 - Fine-tuned BERT classifier

Performance tipiche:

- Accuracy: 75-85%
- F1-score: 0.72-0.82

5.8.3 Possibile progetto di laboratorio: Ranking di Synset in WordNet

Un task più sofisticato: assegnare un basicness score a ogni synset in WordNet.

Approccio Graph-based

WordNet è un grafo diretto dove i nodi sono synset e gli archi sono relazioni semantiche (iponimia, meronimia, ecc.).

Idea: Synset basic sono quelli “centrali” nel grafo.

PageRank-based Basicness:

Applicare PageRank al grafo WordNet:

$$PR(s) = (1 - d) + d \sum_{s' \in \text{in}(s)} \frac{PR(s')}{|\text{out}(s')|}$$

Dove:

- s = synset
- $\text{in}(s)$ = synset che puntano a s (iperonimi)
- $\text{out}(s')$ = synset a cui s' punta (iponimi)
- d = damping factor (0.85)

Ipotesi: Synset con alto PageRank sono più basic.

Approccio Supervised

1. Annotare un subset di synset per basicness

2. Estrarre features per ogni synset:
 - Depth in hierarchy
 - Number of hyponyms
 - Frequency of lemmas
 - Gloss length/complexity
3. Addestrare regressore: synset $\rightarrow$ basicness score $\in [0, 1]$
4. Propagare predizioni a tutto WordNet

5.9 Sfide Aperte e Direzioni Future

5.9.1 Multilingualità

Domanda: Il basic level è universale o language-specific?

- **Ipotesi universalista:** Concetti basic (cibo, famiglia, corpo) sono simili tra lingue
- **Evidenza relativista:** Lessicalizzazioni variano (colori, parentela, spazio)

Ricerca necessaria: Studi comparativi su larga scala con annotazioni parallele.

5.9.2 Dinamicità Temporale

Il basic vocabulary evolve nel tempo:

Esempi di shift:

- “computer”: advanced negli anni '60 $\rightarrow$ basic oggi

- “*smartphone*”: neologismo 2007 → basic entro 2015
- “*telegrafo*”: basic nel 1900 → arcaico oggi

Sfida: Modelli che si adattano dinamicamente, tracciando l’evoluzione diacronica.

5.9.3 Context-Dependent Basicness

La stessa parola può essere basic o advanced a seconda del contesto:

Parola: “*virus*”

- **Contesto medico quotidiano:** BASIC (“Ho preso un virus”)
- **Contesto virologico specialistico:** può essere basic tra esperti ma advanced per pubblico generale
- **Contesto informatico:** BASIC (“virus nel computer”)

Direzione futura: Modelli contestuali che predicono basicness dato (parola, contesto).

5.9.4 Granularità: Oltre la Dicotomia

Come discusso, basic/advanced è una semplificazione. Serve una modellazione più sfumata:

- **Multi-class:** BASIC / INTERMEDIATE / ADVANCED / TECHNICAL
- **Continuous:** Score $\in [0, 1]$
- **Multi-dimensional:**
 - Frequenza d’uso
 - Complessità concettuale

- Registro stilistico
- Dominio di specializzazione

5.9.5 Integrazione in LLM

I Large Language Models potrebbero beneficiare di conoscenza esplicita sul basic level:

- **Controllable generation:** Generare testo con vocabolario di livello specificato
- **Adaptive explanation:** Spiegare concetti complessi usando termini progressivamente più basic
- **Audience adaptation:** Adattare il registro lessicale al pubblico target

Approccio possibile: Fine-tuning con reinforcement learning, premian-do output che rispettano vincoli di basicness.

5.10 Conclusioni

5.10.1 Ricapitolazione

In questa lezione abbiamo esplorato il concetto di **basic level** da molteplici prospettive:

1. **Fondamenti teorici:** Da Roger Brown (1958) a Eleanor Rosch, abbiamo visto come il basic level sia un principio cognitivo fondamentale
2. **Proprietà:** Brevità, concretezza, alta frequenza, facilità di apprendimento

3. **Collocazione gerarchica:** Il basic level come “middle level” in tassonomie concettuali
4. **Variabilità:** Soggettività, dipendenza dal contesto, non-universalità
5. **Applicazioni NLP:** Text simplification, L2 learning, MT, WSD, summarization, health communication
6. **Metriche:** Frequenza, AoA, concreteness, lunghezza, approcci compositi
7. **Modellazione computazionale:** Classificazione supervised, ranking in WordNet, approcci graph-based

5.10.2 Il Paradosso del Basic Level

Un’osservazione finale: il basic level è simultaneamente:

- **Universale:** Pattern simili emergono in tutte le lingue e culture
- **Particolare:** Le specifiche lessicalizzazioni sono language-specific
- **Oggettivo:** Correlazioni robuste con metriche misurabili (frequenza, AoA)
- **Soggettivo:** Variabilità significativa nei giudizi individuali
- **Stabile:** Parole core restano basic attraverso generazioni
- **Dinamico:** Neologismi e arcaismi spostano continuamente i confini

Questa tensione riflette la natura stessa del linguaggio: un sistema contemporaneamente strutturato e fluido, universale e culturalmente situato, biologicamente radicato e socialmente costruito.

5.10.3 Implicazioni per NLP

La ricerca sul basic level ci ricorda che:

Non tutte le parole hanno lo stesso status cognitivo e comunicativo. Sistemi NLP che ignorano questa stratificazione lessicale rischiano di produrre output innaturali, inaccessibili o pragmaticamente inappropriati.

Incorporare consapevolezza del basic level nei modelli NLP può migliorare:

- **Accessibilità:** Generazione di testi comprensibili a pubblici diversi
- **Naturalezza:** Selezione lessicale che rispecchia preferenze dei parlanti nativi
- **Adattività:** Sistemi che si calibrano sul livello linguistico dell'utente

Domande di Riflessione

1. **Il basic level è una proprietà delle parole o dei concetti?** Possiamo avere un concetto basic espresso da una parola advanced in una lingua e viceversa?
2. **Quanto è soggettiva la basicness?** Se annotatori indipendenti non concordano, esiste davvero un basic level “oggettivo”, o è solo un’approssimazione statistica?
3. **Come cambia il basic level con l’expertise?** Per un medico, “ECG” è basic. Significa che il basic level è relativo alla comunità di pratica?
4. **I Large Language Models hanno una nozione implicita di basic level?** GPT-4 può adattare il vocabolario al pubblico — ha appreso il basic level dai dati, o lo simula?
5. **È possibile una lingua con solo parole basic?** Il Basic English di Ogden (850 parole) è realmente sufficiente, o ci sono limiti intrinseci all’espressività?
6. **Come evolverà il basic level con l’AI?** Se le persone interagiscono sempre più con sistemi AI, emergerà un vocabolario “human-AI” condiviso?

Altre possibili idee progettuali (non LAB obbligatorio)

Progetto 1: Basicness Ranking di WordNet Synsets

Obiettivo: Creare un mapping tra synset di WordNet e basicness scores.

Task:

1. Raccogliere features per ogni synset:
 - Depth in hierarchy
 - Hypernym/hyponym count
 - Lemma frequencies
 - Gloss complexity (readability metrics)
2. Annotare un subset (500-1000 synset) per basicness $[0, 1]$
3. Addestrare regressore (Random Forest, XGBoost, Neural Net)
4. Predire basicness per tutti i synset
5. Validare su held-out set e analizzare casi limite

Progetto 2: Basicness-Aware Text Simplification

Obiettivo: Sviluppare un sistema che semplifica testi sostituendo parole advanced con equivalenti basic.

Task:

1. Identificare parole advanced in un testo (soglia di basicness)
2. Per ogni parola advanced, trovare sostituzioni basic:

- WordNet: traversare verso iperonimi
 - Embeddings: vicini semantici con basicness alta
 - Paraphrasing models: generare alternative
3. Ranking delle sostituzioni per:
- Similarità semantica (preservare significato)
 - Fit contestuale (language model scoring)
 - Basicness score
4. Valutazione: confronto con Simple Wikipedia, giudizi umani

Progetto 3: Classificazione Basic/Advanced Automatica

Obiettivo: Classificatore binario per nuovi termini.

Task:

1. Costruire dataset annotato (o usarne uno esistente)
2. Feature engineering:
 - Lexical features (length, morphology)
 - Corpus features (frequency, distribution)
 - Psycholinguistic features (AoA, concreteness)
 - Embeddings
3. Esperimenti con diversi classificatori
4. Analisi di errori: quali parole sono più difficili da classificare?
5. Applicazione: etichettare vocabolari di domini specifici (medicina, legge, tecnologia)

Progetto 4: Basicness in Downstream NLP Tasks

Obiettivo: Dimostrare l'utilità di basicness in un task esistente.

Opzioni:

- **Word Sense Disambiguation:**
 - Ipotesi: sensi basic sono più frequenti
 - Feature: basicness score per ogni senso
 - Valutare se migliora accuracy WSD
- **Machine Translation:**
 - Constraint: mantenere livello di basicness nella traduzione
 - Metrica: differenza di basicness tra source e target
 - Valutare se migliora naturalezza (giudizi umani)
- **Readability Assessment:**
 - Feature: proporzione di parole basic in un testo
 - Task: predire livello di lettura (grade level)
 - Confrontare con metriche tradizionali (Flesch-Kincaid)
- **Question Answering:**
 - Riformulare risposte usando vocabolario basic
 - Valutare comprensibilità per diverse fasce d'età

Progetto 5: Basicness Dynamics - Evoluzione Temporale

Obiettivo: Studiare come il basic vocabulary cambia nel tempo.

Task:

1. Raccogliere corpora diacronici (es. Google Ngrams, archivi giornalistici)
2. Selezionare parole target (neologismi, parole in declino)
3. Calcolare basicness score per decade:
 - Frequenza relativa
 - Age of Acquisition (se disponibile storicamente)
 - Uso in testi semplificati vs formali
4. Identificare pattern:
 - Tecnologia: *computer, internet, smartphone*
 - Arcaismi: *telegrafo, carrozza*
 - Globalizzazione: prestiti linguistici
5. Predire future evoluzioni di basicness

Bibliografia Essenziale

Fondamenti Teorici

- Brown, R. (1958). How shall a thing be called? *Psychological Review*, 65(1), 14-21.
- Rosch, E., Mervis, C. B., Gray, W. D., Johnson, D. M., & Boyes-Braem, P. (1976). Basic objects in natural categories. *Cognitive Psychology*, 8(3), 382-439.
- Lakoff, G. (1987). *Women, Fire, and Dangerous Things: What Categories Reveal about the Mind*. University of Chicago Press.

Basic Vocabulary e Linguistica Applicata

- Ogden, C. K. (1930). *Basic English: A General Introduction with Rules and Grammar*. Kegan Paul, Trench, Trubner & Co.
- Nation, I. S. P. (2001). *Learning Vocabulary in Another Language*. Cambridge University Press.
- Coxhead, A. (2000). A new academic word list. *TESOL Quarterly*, 34(2), 213-238.

Metriche Psicolinguistiche

- Kuperman, V., Stadthagen-Gonzalez, H., & Brysbaert, M. (2012). Age-of-acquisition ratings for 30,000 English words. *Behavior Research Methods*, 44(4), 978-990.
- Brysbaert, M., Warriner, A. B., & Kuperman, V. (2014). Concreteness ratings for 40 thousand generally known English word lemmas. *Behavior Research Methods*, 46(3), 904-911.

- Keuleers, E., Brysbaert, M., & New, B. (2010). SUBTLEX-NL: A new measure for Dutch word frequency based on film subtitles. *Behavior Research Methods*, 42(3), 643-650.

NLP e Complessità Lessicale

- Shardlow, M. (2014). A survey of automated text simplification. *International Journal of Advanced Computer Science and Applications*, 4(1), 58-70.
- Paetzold, G., & Specia, L. (2016). SemEval 2016 Task 11: Complex Word Identification. In *Proceedings of SemEval* (pp. 560-569).
- Vajjala, S., & Meurers, D. (2014). Readability assessment for text simplification: From analysing documents to identifying sentential simplifications. *ITL-International Journal of Applied Linguistics*, 165(2), 194-222.

Basicness Computazionale

- Torrielli, F., Rapp, A., & Di Caro, L. (2023). How shall a machine call a thing? In *International Conference on Natural Language and Information Systems* (pp. 187-200). Springer.

Risorse Lessicali

- Miller, G. A. (1995). WordNet: A lexical database for English. *Communications of the ACM*, 38(11), 39-41.
- Navigli, R., & Ponzetto, S. P. (2012). BabelNet: The automatic construction, evaluation and application of a wide-coverage multilingual semantic network. *Artificial Intelligence*, 193, 217-250.

6 Pre-LLM NLP: Storia, Concetti e Task Relativi a Text Mining, Distributional Semantics, Ontology Learning e Open Information Extraction

6.1 Introduzione

Quando oggi interroghiamo ChatGPT, Claude o GPT-4, quando chiediamo a questi sistemi di generare codice, tradurre lingue o riassumere documenti, tendiamo a dare per scontato ciò che sembrano capacità quasi “magiche”. Ma ogni rivoluzione tecnologica ha radici profonde, e quella dei Large Language Models non fa eccezione.

Prima dell’avvento degli LLM — che possiamo datare simbolicamente al 2017 con il paper “Attention is All You Need” di Vaswani e colleghi, ma che esplode realmente nel 2022-2023 con ChatGPT — esisteva un intero universo di tecniche, filosofie e paradigmi che hanno reso possibile quella rivoluzione. Un universo che oggi chiamiamo, forse con un po’ di nostalgia, **Pre-LLM NLP**.

Questo capitolo non è un museo di tecnologie obsolete. È una mappa genealogica che traccia come siamo arrivati fin qui — e perché molte di quelle “vecchie” tecniche non sono affatto scomparse, ma continuano a vivere, spesso nascoste sotto la superficie dei sistemi moderni che usiamo quotidianamente.

Per capire davvero cosa significhi “fare NLP” oggi, dobbiamo com-

prendere cosa significava farlo ieri. Dobbiamo capire quali problemi si cercava di risolvere quando non avevamo 175 miliardi di parametri a disposizione, quali assunzioni si facevano sul linguaggio quando i dataset erano misurati in megabyte invece che petabyte, quali trade-off si accettavano quando una GPU moderna sarebbe sembrata fantascienza.

E soprattutto, dobbiamo chiederci: cosa abbiamo guadagnato con gli LLM? E cosa, forse, abbiamo perso lungo la strada?

6.1.1 Il Text Mining: Quando i Dati Divennero Protagonisti

Negli anni '80 e '90, mentre la linguistica computazionale classica si concentrava sulla costruzione di grammatiche formali e sistemi basati su regole scritte meticolosamente a mano da esperti umani, una comunità parallela iniziò a guardare al testo in un modo radicalmente diverso. Non più come manifestazione di regole grammaticali astratte da codificare, ma come **fonte di dati** da cui estrarre automaticamente pattern, tendenze, conoscenza.

Nacque così il **Text Mining**: l'applicazione delle tecniche del Data Mining — nate per analizzare database strutturati di vendite, transazioni, comportamenti — al linguaggio naturale non strutturato. Era un approccio profondamente pragmatico, quasi ingegneristico: se abbiamo milioni di documenti, possiamo trovare automaticamente cosa li accomuna? Possiamo raggrupparli per tema senza leggerli? Possiamo predire di cosa parla un testo guardando solo le frequenze delle parole?

Questo cambio di paradigma era filosoficamente profondo. Significava passare da “insegniamo al computer le regole della lingua” a “lasciamo che il computer scopra le regolarità nei dati”. Era il trionfo del **bottom-up** sul **top-down**, dell'empirismo sul razionalismo, dell'induzione sulla deduzione.

E funzionava. Forse non nel modo elegante e teoricamente soddisfacente delle grammatiche formali chomskiane, ma funzionava su scala. Funzio-

nava su milioni di documenti, su lingue diverse, su domini che nessun linguista avrebbe mai potuto codificare manualmente. Le frequenze delle parole, le co-occorrenze, le distribuzioni statistiche catturavano qualcosa di essenziale sul significato — anche se quel “qualcosa” rimaneva difficile da definire con precisione filosofica.

6.2 Una Storia in Quattro Atti: L’Evoluzione del NLP

Se dovessimo raccontare la storia del NLP come un’opera teatrale, vedremmo quattro atti distinti, ciascuno con i suoi protagonisti, le sue ambizioni, le sue illusioni e le sue conquiste durature.

6.2.1 Atto I: L’Era Simbolica (1950s-1980s) — Il sogno della macchina pensante

Tutto inizia con un sogno ambizioso: se il linguaggio è governato da regole — e Chomsky ci aveva appena convinti che lo fosse — e se possiamo codificare quelle regole in forma computazionale, allora possiamo costruire macchine che “capiscono” davvero il linguaggio, non che lo processano superficialmente.

È l’epoca di ELIZA (1966), il primo chatbot che simulava uno psicoterapeuta rogersiano semplicemente usando pattern matching e riformulazioni. “Mi sento triste” diventava “Perché ti senti triste?”. Non c’era comprensione reale, solo manipolazione sintattica intelligente. Eppure, molti utenti si convincevano di parlare con un essere pensante. La lezione era chiara fin da subito: l’illusione di comprensione può emergere da meccanismi sorprendentemente semplici.

È anche l’epoca di SHRDLU (1970) di Terry Winograd, che poteva manipolare blocchi virtuali rispondendo a comandi in inglese naturale. “Prendi il blocco rosso e mettilo sopra il cubo blu” funzionava

perfettamente. Ma solo in quel minuscolo mondo artificiale di blocchi colorati. Appena uscivi da quel dominio ristretto, il sistema crollava. Non generalizzava.

L'approccio simbolico era elegante, teoricamente soddisfacente, basato su fondamenta logiche solide. Ma era anche tremendamente fragile. Ogni eccezione grammaticale richiedeva una nuova regola scritta a mano. Ogni dominio applicativo richiedeva anni di lavoro di codifica. E il linguaggio naturale, con la sua ambiguità intrinseca, variabilità creativa e dipendenza dal contesto culturale, sembrava sempre sfuggire alle reti formali che tentavano di catturarlo.

Eppure, quell'epoca ci ha lasciato intuizioni profonde che vivono ancora oggi: l'idea di parsing sintattico, di rappresentazione della conoscenza in strutture formali, di inferenza logica su proposizioni estratte dal testo. Quando un moderno sistema di question answering costruisce un knowledge graph, sta usando idee che risalgono a quell'era simbolica.

6.2.2 Atto II: L'Era Statistica (1990s-2010s) — Quando i numeri presero il sopravvento

Poi arriva la svolta. Gli anni '90 portano processori più potenti, memorie più capienti, e soprattutto: dati. Montagne di dati. Il web esplode da poche migliaia di pagine a miliardi. I corpora digitali crescono esponenzialmente. E con essi, un nuovo modo di pensare: e se invece di codificare faticosamente le regole del linguaggio, lasciassimo che i modelli le **imparassero** automaticamente dai dati?

Nascono gli Hidden Markov Models per il POS tagging: non servono più regole per decidere se "bank" è sostantivo o verbo, basta osservare abbastanza esempi e il modello apprende i pattern distribuzionali. Nascono le Conditional Random Fields per il Named Entity Recognition: il sistema impara da solo che sequenze come "Barack Obama" tendono a essere nomi di persona, non perché conosce le regole dei nomi propri, ma perché ha visto abbastanza esempi simili.

Nascono i classificatori testuali: Naive Bayes che, pur assumendo ingenuamente l'indipendenza tra parole (un'assunzione palesemente falsa), riesce comunque a distinguere spam da email legittime con accuratezza sorprendente. Support Vector Machines che trovano iperpiani ottimali in spazi a decine di migliaia di dimensioni per separare documenti di categorie diverse.

E nasce, forse più importante di tutto, l'idea che il significato delle parole possa essere catturato dalla loro **distribuzione**. TF-IDF dice: una parola è importante in un documento se appare spesso lì ma raramente altrove. Le matrici di co-occorrenza dicono: due parole sono semanticamente simili se tendono a comparire negli stessi contesti, anche se non compaiono mai nella stessa frase.

È un'epoca pragmatica, quasi anti-teorica. Non si chiede più "come funziona davvero il linguaggio nella mente umana?", ma "cosa posso predire con i dati che ho?". Non è elegante come le grammatiche formali, ma scala. Scala a milioni di documenti, a decine di lingue, a domini che nessun linguista avrebbe mai potuto modellare manualmente.

Questa epoca inventa anche il concetto stesso di **benchmark**: dataset standardizzati su cui confrontare sistemi in modo riproducibile. Inventa le metriche quantitative: precision, recall, F1-score. Inventa, in un certo senso, l'idea che il NLP possa essere una scienza sperimentale basata su evidenze empiriche, non solo speculazione teoretica.

Ma c'era un limite evidente: tutto restava ancora **superficiale**. TF-IDF cattura qualcosa sull'importanza delle parole, ma non capisce che "car" e "automobile" sono sinonimi. Le bag-of-words ignorano completamente l'ordine: "dog bites man" e "man bites dog" sono identiche. I classificatori imparano pattern statistici, ma mancano di quella che potremmo chiamare "comprensione profonda".

6.2.3 Atto III: L'Era Neurale Pre-Transformer (2010-2017) — Il significato come Geometria

Intorno al 2010, le reti neurali — che esistevano da decenni ma erano state abbandonate dopo l'“AI winter” degli anni '80 — tornano prepotentemente sulla scena. Non più con perceptrone a singolo layer, ma con architetture profonde, addestrabili grazie a backpropagation efficiente su GPU sempre più potenti.

E portano con sé un'idea che si rivelerà rivoluzionaria: il significato delle parole può essere catturato come **vettori densi in uno spazio geometrico ad alta dimensionalità**. Non più vettori sparsi one-hot di 100,000 dimensioni dove ogni parola è ortogonale a tutte le altre, ma densi embeddings di 300 dimensioni dove la similarità semantica diventa **prossimità geometrica** (o forse sarebbe meglio dire il contrario).

Word2Vec (2013) di Tomas Mikolov diventa un fenomeno quasi culturale. L'idea è disarmante per semplicità: addestra una rete neurale a predire parole da contesto (o viceversa), e nel processo la rete impara rappresentazioni interne che catturano relazioni semantiche sorprendenti.

Per la prima volta, emergono spontaneamente analogie vettoriali: $\vec{king} - \vec{man} + \vec{woman} \approx \vec{queen}$. Non è un trucco, è una proprietà emergente genuina. Il modello ha imparato che la relazione “king:queen” è analoga alla relazione “man:woman”, e ha codificato questa analogia come trasformazione geometrica nello spazio vettoriale.

Il significato non è più una lista di definizioni in un dizionario, non è più un nodo in un'ontologia costruita a mano. È una **posizione** in uno spazio geometrico. E le relazioni semantiche sono **direzioni** in quello spazio. È un cambio di paradigma profondo: dalla semantica discreta alla semantica continua, dalla logica alla geometria.

Arrivano le LSTM (Long Short-Term Memory), reti ricorrenti che possono “ricordare” dipendenze lunghe nei testi, catturando fenomeni che le n-gram statistiche non potevano vedere. Arrivano i meccanismi di

attention, che permettono ai modelli di “focalizzarsi” dinamicamente su parti rilevanti dell’input — un’idea che si rivelerà centrale nell’atto successivo.

Ma siamo ancora in un mondo dove i task sono separati: un modello per traduzione, uno per sentiment analysis, uno per question answering. Ogni task richiede il suo training specifico, le sue annotazioni costose, la sua architettura dedicata. Non c’è ancora l’idea di un modello davvero universale.

6.2.4 Atto IV: L’Era Transformer e LLM (2017-presente) — L’unificazione

Poi arriva il 2017, e con esso “Attention is All You Need” — il paper di Vaswani e colleghi che introduce l’architettura Transformer. A prima vista, non sembra rivoluzionario. Sembra solo dire: non servono reti ricorrenti complesse, non servono convoluzioni. Serve solo attention: la capacità di mettere in relazione ogni parola con ogni altra parola dell’input, in modo parallelo, efficiente, scalabile a sequenze lunghissime.

Ma si rivela essere molto più di questo. Perché i Transformer scalano in modo che le architetture precedenti non potevano. Più parametri, più dati, più compute producono miglioramenti costanti nelle performance. Non c’è plateau. Almeno, non l’abbiamo ancora trovato.

Nel 2018, BERT introduce il **masked language modeling**: addestra un modello gigantesco (per gli standard dell’epoca: 340 milioni di parametri) a predire parole mancanti in miliardi di frasi prese da Wikipedia e libri. Poi si apre la strada dei fine-tuning su task specifici con relativamente pochi dati. È il paradigma del **pre-training massiccio + fine-tuning leggero**. E funziona straordinariamente bene. Su ogni benchmark NLP esistente, BERT batte i metodi precedenti. E spesso di tanto.

Ma è GPT-3, nel 2020, a cambiare veramente le regole del gioco. Con 175

miliardi di parametri addestrati su 45TB di testo, mostra che un modello abbastanza grande può fare **few-shot learning**: imparare nuovi task da pochi esempi forniti nel prompt, senza fine-tuning. Senza nemmeno bisogno di gradient updates. Solo dalla capacità di riconoscere pattern nei dati di training e adattarli al contesto fornito.

Può scrivere codice, risolvere problemi matematici di base, tradurre tra lingue che non ha visto insieme in training, rispondere a domande su conoscenza fattuale, generare testo creativo — tutto dalla stessa architettura, con gli stessi pesi. È un livello di generalità che nessuna generazione precedente aveva mai ottenuto.

E quando arrivano ChatGPT (fine 2022), GPT-4 (2023), Claude, Gemini, e gli altri, vediamo qualcosa che nessuno aveva davvero anticipato: **conversazione** davvero fluente, contestuale, che sembra “capire” intenzioni, sfumature, implicazioni. Che può seguire istruzioni complesse, ammettere di non sapere, correggere i propri errori quando fatto notare.

Sembra. Quella parola è cruciale. Perché sotto la superficie di questa fluidità impressionante, restano domande profonde senza risposta definitiva: questi modelli “capiscono” davvero il linguaggio? O sono pattern matchers incredibilmente sofisticati che hanno memorizzato così tanti esempi da poter approssimare comprensione senza possederla realmente? È una domanda filosofica che forse trascende la possibilità di risposta empirica.

6.2.5 Il Quinto Atto Ancora da Scrivere

Oggi siamo in una fase di transizione interessante. Gli LLM dominano i titoli di giornale, le conferenze accademiche, gli investimenti venture capital. Sembrano essere il futuro inevitabile del NLP, e forse dell’AI in generale.

Ma sotto la superficie di questo entusiasmo, una realtà più sfumata emerge. I motori di ricerca di Google usano ancora BM25 — una tecnica statistica degli anni ’90 — per il retrieval iniziale, prima di chiamare

reranker neurali. I sistemi di classificazione produttivi in molte aziende usano ancora SVM o Naive Bayes, non perché non conoscano BERT, ma perché servono decisioni interpretabili, auditabili, con tempi di inferenza garantiti sotto i millisecondi.

Le pipeline di sentiment analysis su larga scala usano ancora TF-IDF perché processare 10 milioni di tweet con GPT-4 costerebbe decine di migliaia di dollari e richiederebbe giorni. Con metodi classici: poche ore, qualche dollaro di compute.

Forse la storia del NLP non è una successione di rivoluzioni che cancellano il passato, ma una **stratificazione**: ogni epoca aggiunge layer di capacità, senza necessariamente rimuovere quelli precedenti. Ogni tecnica trova la sua nicchia ecologica dove eccelle. E forse, per costruire il futuro in modo responsabile e sostenibile, dobbiamo capire da dove veniamo, cosa abbiamo imparato lungo il cammino, e quali principi trascendono le mode tecnologiche del momento.

Il quinto atto è ancora da scrivere. Ma se la storia ci insegna qualcosa, è che sarà diverso da quello che immaginiamo.

6.2.6 Dove Si Colloca il Text Mining?

Il Text Mining, protagonista di questo capitolo, si posiziona decisamente nel campo bottom-up. Nasce negli anni '80-'90 esattamente come reazione alla fragilità dei sistemi simbolici. Nasce dalla domanda pragmatica: “cosa possiamo fare con milioni di documenti e tecniche statistiche, senza costruire ontologie a mano o scrivere grammatiche?”

La risposta: sorprendentemente tanto. Possiamo raggruppare documenti per tema senza sapere quali temi esistono (clustering). Possiamo classificare testi in categorie apprendendo pattern dai dati (supervised classification). Possiamo estrarre termini chiave, riassumere documenti, trovare informazioni rilevanti — tutto attraverso metodi statistici che operano su frequenze, co-occorrenze, distribuzioni.

Ma il Text Mining intelligente — e questa è una lezione importante — non ignora completamente la conoscenza linguistica. Usa lemmatization (un processo che richiede conoscenza morfologica) per normalizzare le parole. Usa POS tagging quando serve distinguere “bank” sostantivo da “bank” verbo. Usa dependency parsing quando la semplice bag-of-words non basta a catturare relazioni cruciali.

È un approccio pragmatico: usa la statistica come motore principale, ma incorpora conoscenza linguistica dove aggiunge valore reale misurabile. Non è purismo metodologico, è ingegneria responsabile.

E forse questa è la lezione più importante per il futuro del NLP: non si tratta di scegliere tra top-down e bottom-up, tra simbolico e statistico, tra formale ed empirico. Si tratta di capire **quando** ciascun approccio è appropriato, **dove** i suoi punti di forza risolvono problemi reali, e **come** combinarli intelligentemente invece di vederli come filosofie incompatibili in guerra permanente.

Gli LLM hanno dimostrato la potenza estrema dell’approccio bottom-up quando hai dati a sufficienza. Ma i problemi di interpretabilità, controllabilità, verifica formale, garanzie di correttezza che portano con sé suggeriscono che forse abbiamo buttato via troppo velocemente alcuni degli strumenti dell’approccio top-down. Il futuro probabilmente non sarà vittoria totale di un paradigma, ma integrazione sofisticata dei due.

6.3 Dalle Frequenze al Significato: L’Intuizione Distribuzionale

Abbiamo già visto nella storia come Word2Vec e gli embeddings neurali abbiano trasformato radicalmente il modo in cui rappresentiamo il significato delle parole. Ma prima di arrivare a quel punto — e per capire perché quella trasformazione fu così rivoluzionaria — dobbiamo comprendere l’intuizione fondamentale che sottende tutto: l’**ipotesi distribuzionale**.

6.3.1 “You Shall Know a Word by the Company It Keeps”

Questa frase, attribuita al linguista John Rupert Firth nel 1957, cattura un’idea profonda ma semplice: il significato di una parola non sta in qualche definizione astratta nel cielo platonico delle idee, ma nelle sue **relazioni d’uso** con altre parole nel linguaggio reale.

Se due parole compaiono sistematicamente negli stessi contesti — se possono essere sostituite l’una con l’altra in molte frasi mantenendo una certa coerenza semantica — allora probabilmente hanno significati simili. Non identici, ma simili. “Car” e “automobile”, “dog” e “canine”, “happy” e “joyful” non compaiono MAI nella stessa frase (perché sarebbero ridondanti), ma compaiono in frasi molto simili, con verbi simili, aggettivi simili, contesti simili.

Quindi, paradossalmente, per capire la similarità semantica non devi guardare alla co-occorrenza diretta (“car automobile” è rarissimo), ma alla similarità dei pattern di co-occorrenza con **altre** parole. Se “car” compare spesso con “drive”, “road”, “gasoline”, “park”, e anche “automobile” compare con le stesse parole, allora sono semanticamente correlate.

Questa intuizione — che negli anni ‘50 era più filosofica che operativa — diventa praticabile computazionalmente solo quando hai abbastanza dati per osservare questi pattern statistici in modo affidabile. Servono corpus di milioni di parole per vedere abbastanza occorrenze di parole medio-rare da stimare le loro distribuzioni contestuali.

6.3.2 Dal Vector Space Model agli Embeddings: Un Viaggio Concettuale

Nei capitoli precedenti avete già incontrato il Vector Space Model e TF-IDF. Ricapitoliamo l’idea essenziale, non i dettagli tecnici:

Gerard Salton nel 1974 propone di rappresentare ogni documento come un vettore in uno spazio ad alta dimensionalità, dove ogni dimensione corrisponde a un termine del vocabolario. La similarità tra documenti diventa un problema geometrico: documenti simili puntano in direzioni simili in questo spazio, quindi hanno alto coseno dell'angolo tra i loro vettori.

TF-IDF raffina l'idea pesando i termini: un termine è importante in un documento se appare spesso lì (TF, term frequency) ma raramente in altri documenti (IDF, inverse document frequency). Questo cattura l'intuizione che termini comuni ovunque ("the", "is", "of") non discriminano, mentre termini frequenti localmente ma rari globalmente sono distintivi.

Ma tutto questo opera ancora nel paradigma **sparse**: vettori con 100,000 dimensioni dove il 99% sono zeri. E ogni dimensione ha un'interpretazione esplicita: corrisponde a una specifica parola del vocabolario.

Il salto concettuale — che arriva negli anni 2000 con LSA (Latent Semantic Analysis) e poi esplode con Word2Vec nel 2013 — è passare a rappresentazioni **dense**: vettori di solo 300 dimensioni, tutti non-zero, ma dove le dimensioni non corrispondono più a parole specifiche. Sono "direzioni latenti" nello spazio semantico che il modello scopre automaticamente. Catturano pattern di significato che trascendono singole parole.

E qui la magia: in questo spazio denso, "car" e "automobile" finiscono vicini anche se non co-occorrono mai direttamente, perché condividono pattern di contesto. "King" - "man" + "woman" $\approx$ "queen" perché il modello ha imparato che la relazione di genere è una direzione consistente nello spazio, applicabile a diverse categorie (regalità, professioni, parentele).

6.3.3 Cosa Abbiamo Guadagnato, Cosa Abbiamo Perso

Gli embeddings densi hanno portato capacità straordinarie che le rappresentazioni sparse non potevano dare:

- **Cattura di sinonimi e relazioni semantiche:** “car” e “automobile” sono vicini nello spazio anche senza co-occorrenza diretta
- **Analogie emergenti:** king:queen = man:woman si manifesta come parallelismo geometrico
- **Generalizzazione a parole mai viste:** se una parola nuova compare in contesti simili a “car”, viene automaticamente posizionata vicino in fase di inference
- **Compressione dimensionale:** da 100,000 dimensioni sparse a 300 dense — molto più efficiente per algoritmi downstream
- **Transfer learning:** embeddings pre-addestrati su grandi corpora possono essere usati per task specifici con pochi dati

Ma abbiamo anche perso qualcosa di prezioso:

- **Interpretabilità:** le matrici di co-occorrenza erano trasparenti — potevi vedere esattamente quali parole co-occorrevano. Gli embeddings densi sono opachi — non puoi guardare la dimensione 47 e dire “ah, questa cattura l’animatezza”
- **Determinismo:** TF-IDF è completamente deterministico — stesso input, stesso output, sempre. Word2Vec ha randomness nell’inizializzazione e nel training
- **Verificabilità:** con TF-IDF potevi ispezionare la matrice termine-documento e capire esattamente perché due documenti risultavano simili. Con embeddings, è un mistero dentro le trasformazioni non-lineari della rete
- **Controllo:** potevi modificare manualmente pesi TF-IDF se sapevi che certi termini erano più importanti. Con embeddings neurali,

interventi manuali sono quasi impossibili

Questo pattern — guadagnare potenza ma perdere trasparenza — si ripeterà ancora più drammaticamente con BERT, GPT, e gli LLM moderni. È forse il trade-off centrale dell'era neurale: performance straordinarie al costo di comprensibilità umana di come il sistema raggiunge quei risultati.

E solleva una domanda che attraversa tutto il NLP moderno: fino a che punto siamo disposti ad accettare opacità in cambio di capacità? Dipende dall'applicazione. Per raccomandare film su Netflix, forse va bene un sistema black-box se funziona bene. Per decidere se concedere un prestito bancario, o diagnosticare una malattia, o moderare contenuti online, forse l'interpretabilità conta più della performance marginale.

6.4 I Task del Text Mining: Cosa Cercavamo di Risolvere

Ora che abbiamo il contesto storico e le fondamenta concettuali, guardiamo ai task specifici che il Text Mining cercava di risolvere negli anni '90 e 2000. Non entreremo nei dettagli algoritmici (li avete già studiati), ma cercheremo di capire: **quale problema umano stavamo cercando di risolvere? Come l'abbiamo affrontato allora? E come lo affrontiamo oggi con gli LLM?**

6.4.1 Clustering: Scoprire Strutture Nascoste

Il problema umano è questo: hai 10,000 documenti (email aziendali, articoli di news, paper scientifici). Non sai quali temi contengano. Non hai tempo di leggerli tutti. Puoi far scoprire automaticamente al computer che, diciamo, 2,000 parlano di economia, 3,000 di politica, 1,500 di sport, 3,500 di tecnologia?

È il task del **clustering non supervisionato**. “Non supervisionato” significa: nessuno ha etichettato esempi. Il sistema deve scoprire le categorie da solo guardando solo alle similarità tra documenti.

Negli anni '80, Stuart Lloyd ai Bell Labs propose K-means: un algoritmo iterativo sorprendentemente semplice. Scegli k centroidi casuali. Assegna ogni documento al centroide più vicino. Ricalcola i centroidi come media dei documenti assegnati. Ripeti fino a convergenza. Semplice. Efficiente. E funzionava.

Ma emergeva subito un problema filosofico: **come sapere se un clustering è buono?** Nella classificazione supervisionata, hai una verità oggettiva (le etichette) quindi puoi contare gli errori. Nel clustering, manca. Puoi misurare compattezza dei cluster (quanto sono omogenei internamente) e separazione (quanto sono distinti tra loro), ma queste sono proprietà geometriche. Catturano davvero la “bontà semantica” del clustering?

La risposta onesta è: non sempre. Due sistemi di clustering possono produrre partizioni diverse, entrambe geometricamente valide, ma semanticamente una ha senso per gli umani e l'altra no. Non esiste un modo algoritmico puramente automatico per decidere quale sia “migliore” senza coinvolgere giudizio umano.

Oggi, con gli LLM? Possiamo fare qualcosa di più sofisticato. Invece di clustering puramente su TF-IDF, possiamo generare embeddings contestuali con BERT per ogni documento — catturando significato più profondo — e clusterizzare quelli. Oppure, possiamo chiedere direttamente a GPT-4 di categorizzare documenti, fornendo descrizioni testuali dei cluster emergenti.

Ma il problema fondamentale resta: senza supervisione umana, il clustering cattura regolarità statistiche che possono o meno coincidere con categorie concettuali che gli umani riconoscerebbero come naturali. È un task intrinsecamente più ambiguo della classificazione.

6.4.2 Classificazione: Apprendere da Esempi Etichettati

Qui la situazione è più chiara. Hai esempi etichettati: 5,000 email marcate “spam” o “ham” (non-spam). Vuoi addestrare un sistema che impari dai pattern in quegli esempi e possa classificare nuove email mai viste prima.

Negli anni '90, Andrew McCallum dimostrò che Naive Bayes — un modello incredibilmente semplice che assume ingenuamente indipendenza tra tutte le parole (assunzione palesemente falsa) — funzionava sorprendentemente bene. Perché? Perché anche se l'assunzione è violata, il modello cattura comunque segnali statistici forti: “viagra”, “prince”, “lottery” compaiono molto più in spam che in email legittime. Non serve modellare interazioni complesse tra parole.

Poi arrivarono Support Vector Machines di Joachims (1998), che trovavano iperpiani ottimali separatori in spazi ad altissima dimensionalità. Per testi con 50,000 features diverse, le SVM eccellevano. Divennero lo standard per un decennio.

Cosa hanno in comune questi metodi classici con i classificatori neurali moderni (fine-tuning di BERT, o prompt-based classification con GPT-4)? Più di quanto sembri. Il principio fondamentale resta identico: **apprendere pattern da dati annotati.**

Ciò che è cambiato radicalmente è la capacità di catturare pattern non-lineari complessi, interazioni tra features distanti, rappresentazioni latenti astratte. BERT può “capire” che “not bad” è positivo anche se contiene “bad”, perché ha appreso la negazione come trasformazione semantica. Naive Bayes vedrebbe solo “bad” e classificherebbe probabilmente negativo.

Ma il trade-off interpretabilità vs performance persiste. Con Naive Bayes o SVM lineari, puoi ispezionare i pesi e vedere esattamente perché un documento è stato classificato così. Con BERT fine-tuned, è una scatola nera da 110 milioni di parametri. Funziona meglio, ma non sai davvero perché.

Per molte applicazioni produttive — specialmente in contesti regolamentati come medicina, finanza, legal — questa opacità è inaccettabile. Quindi SVM e Logistic Regression restano in uso, non per ignoranza dei metodi neurali, ma per necessità di accountability.

6.4.3 Summarization: L'Essenza del Testo

Il problema è antico: hai un documento di 50 pagine, devi leggerlo in 5 minuti. Puoi automatizzare la generazione di un riassunto?

I primi sistemi **estrattivi** (anni '90-2000) facevano una cosa molto semplice: selezionavano le frasi più “salient” e le concatenavano. Ma come definire “salience”? Euristiche basiche: la prima frase di ogni paragrafo. Frasi con termini ad alta TF-IDF. Frasi lunghe (contengono più informazione). Non brillante, ma meglio del caso.

Poi nel 2004, Rada Mihalcea ebbe un'intuizione elegante: TextRank. Tratta le frasi come nodi di un grafo, con archi pesati dalla similarità lessicale. Applica PageRank (l'algoritmo di Google per rankare pagine web) per trovare frasi “centrali” — quelle collegate a molte altre. L'idea: frasi centrali nel grafo di coesione lessicale sono probabilmente le più importanti tematicamente.

Ma tutti i metodi estrattivi condividevano un limite fondamentale: potevano solo **selezionare e riordinare**. Non potevano parafrasare, condensare, fondere informazioni da frasi multiple in una frase nuova più concisa. Non potevano davvero “riassumere” come farebbe un umano intelligente.

I metodi **astrattivi** — che generano nuovo testo — rimasero fantascienza fino all'era delle reti neurali seq2seq (2015-2017). Le prime versioni “allucinavano” costantemente: inventavano fatti non presenti nel testo originale. Solo con BART e T5 (2019-2020), pre-addestrati su enormi quantità di testo, i riassunti astrattivi divennero affidabili.

E GPT-4 oggi può riassumere meglio di molti umani, catturando sfuma-

ture, mantenendo fedeltà al testo fonte (quasi sempre), generando prose fluide. È progresso innegabile.

Ma abbiamo anche perso qualcosa: i metodi estrattivi erano **fedeli per costruzione**. Copiavano frasi esatte dal testo. Non potevano inventare. I metodi astrattivi neurali possono generare riassunti più fluidi, più concisi — ma anche dire cose che il documento originale non conteneva. È una feature o un bug? Dipende se ti serve creatività o fedeltà letterale. Per legal document summarization: bug. Per news aggregation: forse feature.

6.4.4 Information Retrieval: Dal Boolean Search al Semantic Search

Cercare informazioni è forse il task più fondamentale del Text Mining. E ha una storia affascinante che riflette tutta l'evoluzione del NLP.

Negli anni '60-'70: **Boolean search**. Query come (machine AND learning) OR (neural AND network). Rigido, binario. Un documento o matcha o no. Nessun ranking, nessuna notion di “rilevanza parziale”. Ma deterministico, comprensibile, controllabile.

Anni '70-'90: Salton introduce il **Vector Space Model** e il ranking graduato basato su cosine similarity. Ora i documenti possono essere “parzialmente rilevanti”. BM25 di Robertson (anni '90) perfeziona l'idea pesando termini per distintività e normalizzando per lunghezza documento. Diventa lo standard de facto per 25+ anni.

E ancora oggi — 2024 — Google Search usa BM25 (o varianti) per il primo stage di retrieval su miliardi di documenti. Perché? Perché è velocissimo, deterministico, comprensibile. Trova i 1000 candidati più promettenti in millisecondi.

Poi, anni 2010 con Word2Vec e GloVe, emerge un'idea: **semantic search**. E se invece di matching lessicale cercassimo nel significato? Query “automobile” dovrebbe matchare documenti con “car” anche senza

sovrapposizione di parole.

Dense retrieval (2019-2020): ogni query diventa embedding denso (via BERT), ogni documento pure. Similarity = dot product nello spazio semantico. Funziona. Cattura sinonimia, parafrasing, relazioni concettuali che BM25 perdeva.

Ma c'è un problema pratico: calcolare similarità tra query e **tutti** i documenti nella collezione non scala bene oltre 10M documenti senza approssimazioni (ANN, approximate nearest neighbors). E richiede GPU per encoding veloce.

Risultato? I sistemi moderni usano **pipeline ibride**:

1. BM25 recupera 1000 candidati (Stage 1: retrieval, millisecondi, CPU)
2. BERT cross-encoder reranka top-100 (Stage 2: reranking, secondi, GPU)
3. GPT-4 genera snippet o risponde direttamente (Stage 3: generation, opzionale)

È una metafora perfetta per tutto il capitolo: tecniche di epoche diverse che coesistono in symbiosi, ciascuna eccellendo dove le altre sono deboli. Non sostituzione, ma stratificazione.

6.4.5 Topic Modeling: Scoprire Temi Nascosti

C'è un ultimo task del Text Mining che merita attenzione particolare, perché rappresenta forse il ponte più evidente tra approccio classico e moderno: il **topic modeling**.

Il problema è intuitivo: hai una collezione di migliaia di documenti — articoli di giornale, paper scientifici, post di blog. Non hai tempo di leggerli tutti. Puoi automaticamente scoprire quali temi dominanti attraversano questa collezione? Quali conversazioni ricorrenti emergono?

Come si raggruppano semanticamente i documenti attorno a concetti latenti?

È diverso dal clustering nel senso che non vuoi solo raggruppare documenti simili — vuoi **caratterizzare** quei gruppi con etichette interpretabili, con liste di parole chiave che catturano l'essenza tematica di ciascun cluster.

L'Era Classica: LDA

Negli anni 2000, il metodo dominante era **Latent Dirichlet Allocation** (LDA) di Blei, Ng e Jordan (2003). L'idea era probabilistica ed elegante: ogni documento è una “miscela” di topic, e ogni topic è una distribuzione di probabilità su parole.

Per esempio, un paper accademico potrebbe essere 40% “machine learning”, 30% “computer vision”, 30% “natural language processing”. E il topic “machine learning” assegna alta probabilità a parole come “training”, “model”, “algorithm”, “optimization”.

Su collezioni di decine di migliaia di documenti, LDA scopriva topic spesso sorprendentemente coerenti. Ma aveva anche limiti evidenti:

- Operava su bag-of-words — ignorava completamente ordine e sintassi
- Non catturava sinonimia (“car” e “automobile” erano parole completamente separate)
- Richiedeva specificare a priori il numero di topic k
- Interpretare i topic richiedeva ancora molto giudizio umano

L'Era Moderna: BERTopic e Neural Topic Models

Con l'avvento degli embeddings neurali, il paradigma cambia radicalmente. Invece di modellare distribuzioni di parole su topic astratti, la nuova generazione di metodi parte da rappresentazioni semantiche dense dei documenti.

BERTopic (Grootendorst, 2022) è un esempio paradigmatico. La pipeline è ingegnosamente ibrida:

1. **Encoding neurale:** ogni documento diventa un embedding denso via modelli transformer (BERT, RoBERTa, o varianti specializzate come Sentence-BERT)
2. **Riduzione dimensionale:** UMAP proietta gli embeddings ad alta dimensione (768 o 1024 dimensioni) in uno spazio più basso (tipicamente 5-10 dimensioni) preservando struttura locale
3. **Clustering denso:** HDBSCAN trova cluster di densità variabile, gestendo naturalmente outliers (documenti che non appartengono chiaramente a nessun topic)
4. **Caratterizzazione topic:** class-based TF-IDF estrae le parole più distintive per ogni cluster — non le parole più frequenti in assoluto, ma quelle che distinguono quel cluster dagli altri

Il risultato? Topic molto più coerenti semanticamente di quelli LDA. Perché gli embeddings BERT catturano sinonimia, polisemia, relazioni concettuali che le bag-of-words perdevano. Documenti su “autonomous vehicles” e “self-driving cars” finiscono naturalmente nello stesso cluster anche senza condividere parole.

E la visualizzazione diventa sorprendentemente intuitiva. Proiettando gli embeddings in 2D (sempre via UMAP), puoi letteralmente **vedere** la mappa concettuale della tua collezione: cluster compatti di documenti simili, con outliers sparsi ai margini, con distanze che riflettono (approssimativamente) similarità semantica.

Il Caso d'Uso: ArXiv Papers

Un esempio concreto aiuta a capire la potenza del metodo. Prendi 45,000 abstract di paper scientifici da ArXiv nel dominio NLP. Applica BERTopic.

Cosa scopri? Topic emergono come:

- *Topic 0*: “translation”, “machine”, “neural”, “attention”, “encoder”
→ Neural Machine Translation
- *Topic 1*: “sentiment”, “opinion”, “polarity”, “aspect”, “review” → Sentiment Analysis
- *Topic 2*: “question”, “answer”, “retrieval”, “passage”, “squad” → Question Answering
- *Topic 3*: “graph”, “knowledge”, “entity”, “relation”, “embedding”
→ Knowledge Graphs

Non hai specificato questi topic a priori. Non hai annotato esempi. Sono emersi automaticamente dalla struttura semantica latente negli embeddings. E sono interpretabili: guardando le parole chiave, capisci immediatamente di cosa parla ciascun cluster.

Cosa Abbiamo Guadagnato, Cosa Perso

I metodi neurali di topic modeling portano capacità straordinarie:

- **Coerenza semantica**: topic molto più interpretabili di LDA
- **Robustezza a sinonimi**: “car” e “automobile” non sono più problemi
- **Gestione outliers**: documenti fuori tema non forzati in cluster arbitrari
- **Visualizzabilità**: puoi letteralmente vedere la mappa concettuale

- **Flessibilità:** cambiare embedding model cambia granularità semantica catturata

Ma abbiamo anche perso qualcosa rispetto a LDA:

- **Interpretabilità probabilistica:** LDA dava distribuzioni esplicite (“questo documento è 40% topic A”). BERTopic assegna cluster hard, senza probabilità
- **Modello generativo:** non puoi “campionare” nuovi documenti da topic come con LDA
- **Trasparenza:** perché BERT ha messo questo documento in quel cluster? Mistero
- **Costi computazionali:** encoding 100k documenti con BERT richiede GPU e ore. LDA scalava meglio su CPU

E ancora una volta, il pattern si ripete: non è sostituzione, ma trade-off. Per collezioni massive dove velocità conta più di qualità (monitoring social media in real-time), LDA potrebbe essere ancora preferibile. Per analisi profonde dove coerenza semantica è cruciale (analisi letteratura scientifica, intelligence su documenti sensibili), i metodi neurali eccellono.

II Laboratorio Pratico

Capire topic modeling richiede **sperimentazione diretta**. La teoria è utile, ma solo mettendo le mani sui dati — provando diversi embedding models, tweaking parametri UMAP, confrontando HDBSCAN con altri algoritmi di clustering — capisci davvero cosa funziona e cosa no.

Per questo, il laboratorio su BERTopic con dataset ArXiv è pedagogicamente prezioso. Proverete:

- Diverse scelte di embedding: General Text Embeddings (GTE) vs altri modelli Sentence-Transformer

- Impatto della riduzione dimensionale: 5 vs 10 componenti UMAP, metric cosine vs euclidean
- Parametri clustering: `min_cluster_size` in HDBSCAN — troppo basso crea micro-cluster rumorosi, troppo alto perde granularità
- Preprocessing: normalizzare testo abstract migliora coerenza? Rimuovere stopwords? Lemmatization?
- Visualizzazione: quali topic sono vicini nello spazio semantico? Quali outliers esistono e perché?

Non c'è una configurazione “corretta” universale. Dipende dal dominio, dalla granularità desiderata, dal trade-off velocità/qualità accettabile. L'obiettivo è sviluppare **intuizione empirica**: quali knobs girare per ottenere risultati più coerenti per il tuo specifico caso d'uso.

E questa è forse la lezione più profonda del topic modeling — e del Text Mining in generale. Non esistono soluzioni one-size-fits-all. Esistono principi solidi, strumenti potenti, e la saggezza di saperli combinare adattivamente. Proprio come un artigiano esperto sceglie il tool giusto per il lavoro specifico, non il tool più avanzato in assoluto.

6.5 Conclusione: Cosa Abbiamo Imparato, Dove Stiamo Andando

Siamo arrivati alla fine di questo viaggio attraverso la storia del Pre-LLM NLP. È il momento di fermarci e riflettere: cosa abbiamo davvero imparato? E cosa ci dice questo sul futuro?

6.5.1 Il Paradosso del Progresso

C'è un paradosso curioso al cuore di questa storia. Mentre celebriamo — giustamente — la rivoluzione degli LLM, rischiamo di dimenticare le

fondamenta su cui quella rivoluzione è stata costruita. E nel dimenticarle, rischiamo di perdere qualcosa di prezioso.

TF-IDF, BM25, Naive Bayes, K-means, TextRank: non sono solo acronimi in paper polverosi. Sono **intuizioni profonde** su come estrarre significato da simboli, su come misurare rilevanza senza “capire”, su come trovare struttura in dati non annotati. E molte di quelle intuizioni vivono ancora, trasformate ma riconoscibili, nei sistemi moderni.

Quando BERT usa masked language modeling, sta facendo distributional semantics — la stessa idea di Firth negli anni '50, solo su scala massiccia. Quando GPT-4 fa in-context learning da pochi esempi, sta facendo una forma di k-NN nei suoi spazi latenti — clustering implicito, come K-means ma infinitamente più sofisticato.

Il paradosso è che mentre i metodi specifici evolvono radicalmente, i **principi sottostanti** mostrano notevole continuità. L'ipotesi distribuzionale — il significato emerge dall'uso — resta valida. Il trade-off interpretabilità vs potenza resta irrisolto. La necessità di valutazione empirica rigorosa resta centrale.

6.5.2 Cosa Gli LLM Hanno Risolto (E Cosa No)

Serve onestà intellettuale qui. Gli LLM hanno portato capacità che solo dieci anni fa sembravano fantascienza:

Hanno risolto (o quasi):

- **Generalizzazione cross-task:** un modello, mille applicazioni senza retraining
- **Comprensione contestuale profonda:** catturano sfumature, sarcasmo, implicazioni che metodi classici perdevano
- **Few-shot learning:** imparano nuovi task da handful di esempi nel prompt
- **Generazione fluente:** testo indistinguibile da umano (spesso)

- **Accessibilità:** chiunque può “programmare” con linguaggio naturale, senza expertise tecnico

Ma non hanno risolto tutto. Hanno anche creato nuovi problemi:

Non hanno risolto (o hanno peggiorato):

- **Interpretabilità:** cosa ha imparato GPT-4 nei suoi 1.7 trillion parametri? Mistero quasi completo
- **Controllabilità:** come garantire che non generi contenuti dannosi, falsi, biased? Problema aperto
- **Efficienza:** TF-IDF classifica 100k documenti in secondi su laptop. GPT-4 richiede cluster GPU e milioni di dollari
- **Riproducibilità:** temperature sampling rende output stocastico. Stesso prompt, risposte diverse
- **Verità e hallucination:** inventano fatti, citazioni, referenze con sicurezza disarmante
- **Costi economici ed ecologici:** training GPT-3 ha emesso CO come cinque auto in tutta la loro vita
- **Concentrazione di potere:** solo poche corporation possono permettersi di addestrare frontier models

Quindi: progresso innegabile, ma non panacea universale. E molti problemi classici — evaluation, bias, fairness, robustness — persistono o si amplificano.

6.5.3 La Stratificazione, Non La Sostituzione

Una delle lezioni più importanti di questo capitolo è questa: **il progresso tecnologico nel NLP raramente è sostituzione completa; più spesso è stratificazione.**

Guardiamo a sistemi reali che girano in produzione oggi, nel 2024:

- **Google Search:** Stage 1 usa BM25 (tecnica anni '90) per retrieval rapido su miliardi di pagine. Stage 2 usa neural rankers (BERT-based, tecnica anni 2019) per riordinare top-100. Stage 3 usa LLMs per generare featured snippets o risposte dirette. Tre epoche del NLP in una pipeline.
- **Classificazione email spam:** Molte aziende usano ancora Naive Bayes o SVM perché processano milioni di email al giorno con latenza garantita <10ms. BERT sarebbe più accurato, ma troppo lento e costoso per quel volume.
- **Sentiment analysis su larga scala:** Analizzare 10 milioni di tweet con GPT-4 costerebbe \$50,000+ e richiederebbe giorni. Con TF-IDF + Logistic Regression: poche ore, \$100 di compute. Per molti use case, l'accuratezza extra non vale il costo 500x.
- **Sistemi di raccomandazione:** Embedding prodotti con Word2Vec (trattando sequenze di acquisti come "frasi") funziona ancora benissimo. È veloce, interpretabile, non richiede GPU. Molte aziende non sono passate a metodi più complessi perché "if it ain't broke, don't fix it".

Non è nostalgia. È **pragmatismo ingegneristico**. È capire che tool diversi sono ottimali per problemi diversi, constraint diversi, requirement diversi. Un martello è meglio di una pistola per piantare chiodi, anche se la pistola è tecnologicamente più avanzata.

6.5.4 Quattro Lezioni dal Passato per il Futuro

Se guardiamo alla storia del NLP con occhi critici ma costruttivi, emergono alcuni pattern che potrebbero guidarci nel futuro:

Lezione 1: L'Importanza delle Assunzioni Esplicite

I metodi pre-LLM avevano un vantaggio spesso sottovalutato: le loro **assunzioni erano esplicite**. TF-IDF assume che termini rari siano di-

scriminativi. Naive Bayes assume indipendenza condizionale tra parole. K-means assume cluster sferici di dimensione simile.

Queste assunzioni potevano essere sbagliate — spesso lo erano. Ma sapevamo quali fossero. Potevamo testarle, discuterle, modificarle se necessario. C'era trasparenza epistemologica.

Con gli LLM, le assunzioni sono **implicite**, sepolte in miliardi di parametri ottimizzati da gradiente descent stocastico. Non sappiamo cosa ha davvero imparato il modello. Quali bias ha assorbito dai dati? Quali scorciatoie spurie ha scoperto? Quali pattern superficiali usa invece di ragionamento profondo?

Per il futuro: dobbiamo trovare modi per rendere interpretabili anche i modelli complessi. Non possiamo accontentarci di “funziona bene sul benchmark”. Servono strumenti per ispezionare, validare, correggere le assunzioni implicite che i modelli hanno appreso.

Lezione 2: Il Valore della Semplicità

TF-IDF fu proposto nel 1972. Cinquant'anni dopo, è ancora usato in produzione in migliaia di sistemi. Perché? Perché è **semplice, veloce, deterministico, comprensibile**.

Non è il metodo più accurato. Non cattura semantica profonda. Ma in molti contesti, queste proprietà — semplicità, velocità, determinismo, comprensibilità — valgono più dell'accuratezza marginale che un modello complesso potrebbe dare.

C'è una tendenza nel ML moderno a sempre cercare state-of-the-art, performance massime, modelli più grandi. Ma complessità ha costi: computazionale, manutentivo, di debugging, di deployment, ecologico.

Per il futuro: prima di chiamare GPT-4 da 1.7T parametri, chiediamoci: serve davvero? O un modello più piccolo, più veloce, più controllabile potrebbe bastare per questo specific use case? Non è luddismo, è ingegneria responsabile.

Lezione 3: L'Illusione della Generalità

Ogni epoca del NLP ha avuto il suo “modello universale” che prometteva di risolvere tutto:

- Anni '80: sistemi esperti basati su logica
- Anni '90: SVM con kernel universali
- Anni 2010: BERT pre-addestrato con fine-tuning
- Anni 2020: GPT-4 e few-shot prompting

Ogni volta, c'era entusiasmo genuino che finalmente avevamo **la soluzione**. E ogni volta, col tempo, scoprivamo limiti, casi edge, failure modes.

GPT-4 è straordinariamente versatile. Ma ci sono task dove metodi specializzati battono ancora i LLM generici. Named Entity Recognition in domini tecnici: CRF addestrate su dati domain-specific. Parsing sintattico formalmente corretto: parser neurali specializzati. Classificazione binaria ad altissimo throughput: SVM ottimizzate.

Per il futuro: gli LLM sono strumenti potentissimi nel toolbox, ma non sono la risposta a tutto. Manteniamo diversità metodologica. Valorizziamo specializzazione quando appropriata.

Lezione 4: Il Costo Nascosto dei Dati

Word2Vec: giorni di training su corpus statico, una volta. BERT: settimane su cluster GPU. GPT-3: mesi e milioni di dollari. GPT-4: costi probabilmente nell'ordine dei 100 milioni di dollari.

Ogni generazione richiede più dati, più compute, più energia. C'è un punto in cui questa crescita diventa **insostenibile** — non solo economicamente (solo poche corporation possono permetterselo), ma anche ecologicamente (emissioni di CO paragonabili a paesi piccoli).

E socialmente: i dati su cui questi modelli sono addestrati sono raschiati

dal web senza consenso degli autori. Solleva questioni di copyright, privacy, equità. Chi possiede il valore generato da modelli addestrati su lavoro collettivo umano?

Per il futuro: dobbiamo investire in metodi **data-efficient** e **compute-efficient**. Non solo modelli sempre più grandi. Anche: distillation, pruning, quantization. Modelli più piccoli che catturano capacità dei grandi. Metodi che imparano da meno dati sfruttando struttura, prior knowledge, ragionamento simbolico integrato.

6.5.5 Guardando Avanti: Il Dialogo Tra Generazioni

Forse il modo migliore per pensare alla relazione tra Pre-LLM NLP e LLM non è come **successione** lineare (vecchio vs nuovo, obsoleto vs moderno), ma come **dialogo** continuo tra approcci complementari.

Gli LLM eccellono in: generazione fluente, adattamento rapido a nuovi task, comprensione contestuale profonda, cattura di pattern complessi. Ma sono opachi, costosi, stocastici, difficili da controllare.

I metodi classici eccellono in: efficienza computazionale, interpretabilità, controllabilità, determinismo, deployment leggero. Ma sono rigidi, richiedono feature engineering manuale, catturano solo pattern superficiali.

Il futuro probabilmente non sarà vittoria totale di un paradigma. Sarà **ibridazione intelligente**:

- LLM per generare candidati creativi, metodi classici per validation e ranking
- Embeddings neurali per rappresentazione ricca, metriche classiche interpretabili per valutazione
- Few-shot prompting per prototipazione rapida, fine-tuning supervisionato per deployment produttivo robusto

- Neural retrieval per recall semantico, BM25 come fallback quando neural fallisce o per constraint di latenza
- GPT-4 per task complessi con alta tolleranza a errori, SVM per task semplici con zero tolleranza

6.6 Laboratorio LAB-4: Topic Modeling con BERTopic

Questo esercizio vi porterà attraverso una pipeline completa di topic modeling moderno, dalla preparazione dati alla visualizzazione dei risultati, facendovi sperimentare le scelte e i trade-off discussi in questo capitolo.

6.6.1 Obiettivo

Costruire una pipeline di clustering e topic modeling su un dataset di articoli scientifici, utilizzando embeddings neurali, riduzione dimensionale e algoritmi di clustering denso. L'obiettivo non è solo ottenere topic coerenti, ma capire **perché** certe scelte funzionano meglio di altre per questo specifico dominio.

6.6.2 Dataset: ArXiv NLP Papers

Utilizzeremo circa 45,000 abstract di paper scientifici dal dominio Natural Language Processing, disponibili su Hugging Face:

- URL: https://huggingface.co/datasets/MaartenGr/arxiv_nlp
- Variabili principali:
 - Abstracts: riassunti dei paper (testo principale per topic modeling)

- Titles: titoli degli articoli (utili per validazione qualitativa)

Perché questo dataset? Perché ha tre proprietà pedagogicamente utili:

1. Abbastanza grande da mostrare pattern statistici robusti (~45k documenti)
2. Domain-specific (NLP) quindi i topic emersi saranno interpretabili per voi
3. Moderatamente challenging (paper scientifici hanno linguaggio tecnico e sovrapposizioni tematiche)

6.6.3 Pipeline Completa

La pipeline si articola in quattro fasi principali, ciascuna con scelte metodologiche da esplorare:

Fase 1: Preprocessing (Opzionale ma Consigliato)

Prima di embedding, potete migliorare la qualità dell'input:

- Normalizzazione lowercase
- Rimozione caratteri speciali, numeri, URLs
- Rimozione stopwords domain-specific ("paper", "method", "propose")
- Lemmatization vs stemming vs nessuno dei due

Domanda da esplorare: il preprocessing migliora davvero la coerenza dei topic? O gli embeddings neurali sono già abbastanza robusti da gestire rumore?

Fase 2: Embedding dei Documenti

Convertite ogni abstract in un vettore denso usando un modello pre-addestrato. Scelta consigliata iniziale:

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("thenlper/gte-small")
embeddings = model.encode(abstracts, show_progress_bar=True)
```

Il modello **General Text Embeddings (GTE)** è ottimizzato per similarity semantica generale. Ma potreste anche provare:

- all-MiniLM-L6-v2: più veloce, dimensioni più basse (384)
- all-mpnet-base-v2: più lento, qualità superiore (768 dim)
- Modelli domain-specific per testi scientifici (es. SciBERT-based)

Domanda da esplorare: come cambia la qualità dei topic cambiando modello di embedding? Vale la pena il costo computazionale extra di modelli più grandi?

Fase 3: Riduzione Dimensionale con UMAP

Gli embeddings hanno tipicamente 384-768 dimensioni. UMAP li proietta in uno spazio più basso preservando struttura locale:

```
from umap import UMAP

umap_model = UMAP(n_components=5,
                  min_dist=0.0,
                  metric="cosine",
                  random_state=42)
reduced_embeddings = umap_model.fit_transform(embeddings)
```

Parametri da sperimentare:

- `n_components`: 5 vs 10 vs 15. Più componenti = più informazione preservata, ma clustering più difficile
- `min_dist`: 0.0 (cluster compatti) vs 0.1 (più spazio tra punti)
- `metric`: “cosine” (angolo tra vettori) vs “euclidean” (distanza)
- `n_neighbors`: 15 (default) vs 30 vs 50 — controlla balance local/-global structure

Domanda da esplorare: quale configurazione massimizza separazione tra cluster preservando coerenza semantica intra-cluster?

Fase 4: Clustering con HDBSCAN

HDBSCAN trova cluster di densità variabile e gestisce outliers automaticamente:

```
from hdbscan import HDBSCAN

hdbscan_model = HDBSCAN(min_cluster_size=50,
                        metric="euclidean",
                        cluster_selection_method="eom",
                        prediction_data=True)
clusters = hdbscan_model.fit_predict(reduced_embeddings)
```

Parametri critici:

- `min_cluster_size`: 50 vs 100 vs 200. Più alto = cluster più grandi e stabili, ma perdi granularità
- `cluster_selection_method`: “eom” (Excess of Mass) vs “leaf” — diversi criteri di validità cluster
- `min_samples`: controllo rumore — se None, default a `min_cluster_size`

Domanda da esplorare: quanti outliers sono accettabili? Se il 30% dei documenti finisce come rumore, i parametri sono troppo stringenti?

Fase 5: Topic Modeling con BERTopic

Ora integriamo tutto in BERTopic per estrarre parole chiave interpretabili:

```
from bertopic import BERTopic

topic_model = BERTopic(embedding_model=model,
                        umap_model=umap_model,
                        hdbscan_model=hdbscan_model,
                        verbose=True)

topics, probs = topic_model.fit_transform(abstracts, embeddings)
```

Esplorazione dei risultati:

```
# Informazioni generali sui topic
topic_info = topic_model.get_topic_info()
print(f"Numero di topic: {len(topic_info) - 1}")
# -1 per escludere outliers

# Parole chiave per topic specifico (es. Topic 4)
topic_4_keywords = topic_model.get_topic(4)
print("Topic 4 keywords:", topic_4_keywords)

# Documenti rappresentativi per un topic
representative_docs = topic_model.get_representative_docs(4)
```


6.6.4 Visualizzazione e Validazione

BERTopic offre potenti strumenti di visualizzazione. Usateli per validazione qualitativa:

Visualizzazione dei Documenti

Proietta embeddings in 2D e colora per cluster:

```
topic_model.visualize_documents(abstracts,  
                                reduced_embeddings=reduced_embeddings,  
                                hide_document_hover=False)
```

Cosa cercare: cluster compatti e ben separati? Outliers distribuiti uniformemente o concentrati in zone?

Barchart dei Topic

Mostra le top parole per ciascun topic:

```
topic_model.visualize_barchart(top_n_topics=10)
```

Validazione qualitativa: le parole hanno senso insieme? “translation”, “machine”, “neural”, “encoder” suggeriscono chiaramente Neural Machine Translation. Ma se vedi “network”, “system”, “data”, “model” — troppo generico, parametri da rivedere.

Hierarchy e Heatmap

Esplora relazioni tra topic:

```
topic_model.visualize_hierarchy()  
topic_model.visualize_heatmap()
```

Utile per capire se hai over-segmented (troppi topic simili tra loro) o under-segmented (topic troppo generici che andrebbero divisi).

6.6.5 Requisiti e Deliverable

Cosa dovete fare:

1. Implementare la pipeline completa su dataset ArXiv
2. Sperimentare **almeno 3 configurazioni diverse** variando:
 - Modello di embedding
 - Parametri UMAP (`n_components`, `n_neighbors`)
 - Parametri HDBSCAN (`min_cluster_size`)
3. Per ogni configurazione, documentare:
 - Numero di topic emersi
 - Percentuale di outliers
 - Top-5 parole per i 10 topic più popolati
 - Tempo computazionale (embedding + riduzione + clustering)
4. Validazione qualitativa: per almeno 5 topic, esaminare documenti rappresentativi e valutare coerenza semantica
5. **Confronto critico:** quale configurazione produce topic più interpretabili? Perché? Quali trade-off avete osservato?

Bonus opzionali:

- Provare su un dataset alternativo di vostro interesse (migliaia di tweet, articoli di news, recensioni prodotti)

- Implementare query-based search: dato un testo, trovare il topic più simile
- Tracking temporale: come evolvono i topic nel tempo? (richiede timestamp nei dati)
- Confronto con LDA classico: stessa collezione, topic LDA vs BERTopic — differenze?

6.6.6 Suggerimenti Pratici

- **Iniziate piccolo:** prima 5k documenti per debug rapido, poi scale a 45k
- **Visualizzate sempre:** i numeri mentono, gli scatter plot rivelano. Se cluster sembrano arbitrari in 2D, probabilmente lo sono anche in dimensioni superiori
- **Leggete documenti:** non fidatevi ciecamente delle parole chiave. Prendete 10 documenti random da un cluster e leggeteli. Hanno davvero senso insieme?
- **Costi computazionali:** embedding 45k documenti con modelli grandi può richiedere 10-30 minuti su GPU, ore su CPU. Pianificate di conseguenza.
- **Seed per riproducibilità:** UMAP e HDBSCAN hanno componenti stocastici. Usate `random_state=42` per risultati riproducibili durante sperimentazione.
- **Iterate intelligentemente:** non cambiate 5 parametri contemporaneamente. Variate uno alla volta per capire impatto individuale.

Domande di Riflessione

1. L'approccio top-down (simbolico) e bottom-up (statistico) riflettono filosofie profondamente diverse sul linguaggio. Quale è più "corretto"? O è la domanda stessa mal posta?
2. TF-IDF cattura importanza senza "capire" significato. GPT-4 genera testo fluente che sembra "capire". Ma cosa significa davvero "capire"? È una proprietà binaria o un continuum?
3. Abbiamo visto che ogni epoca del NLP credeva di essere vicina a "risolvere" il problema. Cosa ci rende certi che gli LLM non siano solo l'ultimo capitolo di questa illusione ricorrente?
4. Il trade-off interpretabilità vs performance è fondamentale. In quali domini è eticamente inaccettabile usare modelli black-box, indipendentemente da quanto performanti? Perché?
5. I metodi pre-LLM (TF-IDF, BM25, Naive Bayes) sono ancora usati in produzione non per ignoranza, ma per scelta. Questo cosa ci dice sui limiti reali — non solo teorici — degli LLM?
6. Se addestrare GPT-5 richiedesse emissioni di CO equivalenti a una nazione di medie dimensioni, sarebbe giustificabile? Come bilanciamo progresso scientifico e responsabilità ecologica?
7. Word2Vec ha mostrato che analogie semantiche emergono automaticamente da semplice predizione contestuale. Questo cosa implica sulla natura del significato linguistico? È davvero "solo" distribuzione?
8. Guardate ai quattro atti della storia NLP. Quale principio, se ce n'è uno, è rimasto costante attraverso tutte le epoche? E quale lezione dal passato rischiamo maggiormente di dimenticare oggi?

Bibliografia Essenziale

Fondamenti Storici e Filosofici

- Firth, J. R. (1957). A synopsis of linguistic theory 1930-1955. In *Studies in Linguistic Analysis* (pp. 1-32). Blackwell.
- Harris, Z. S. (1954). Distributional structure. *Word*, 10(2-3), 146-162.
- Weizenbaum, J. (1966). ELIZA — A computer program for the study of natural language communication between man and machine. *Communications of the ACM*, 9(1), 36-45.
- Winograd, T. (1972). Understanding natural language. *Cognitive Psychology*, 3(1), 1-191.

Vector Space Model e Information Retrieval

- Salton, G., Wong, A., & Yang, C. S. (1975). A vector space model for automatic indexing. *Communications of the ACM*, 18(11), 613-620.
- Robertson, S. E., & Walker, S. (1994). Some simple effective approximations to the 2-Poisson model for probabilistic weighted retrieval. *Proceedings of SIGIR*, 232-241.
- Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.

Embeddings e Distributional Semantics

- Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S., & Dean, J. (2013). Distributed representations of words and phrases and their compositionality. *Advances in Neural Information Processing Systems*, 3111-3119.

- Pennington, J., Socher, R., & Manning, C. D. (2014). GloVe: Global vectors for word representation. *Proceedings of EMNLP*, 1532-1543.
- Peters, M. E., et al. (2018). Deep contextualized word representations. *Proceedings of NAACL*, 2227-2237.

L'Era Transformer e LLM

- Vaswani, A., et al. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 5998-6008.
- Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL*, 4171-4186.
- Brown, T. B., et al. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877-1901.
- OpenAI (2023). GPT-4 Technical Report. arXiv preprint arXiv:2303.08774.

Riflessioni Critiche e Futuro

- Bender, E. M., et al. (2021). On the dangers of stochastic parrots: Can language models be too big? *Proceedings of FAccT*, 610-623.
- Marcus, G., & Davis, E. (2020). GPT-3, Bloviator: OpenAI's language generator has no idea what it's talking about. *MIT Technology Review*.
- Strubell, E., Ganesh, A., & McCallum, A. (2019). Energy and policy considerations for deep learning in NLP. *Proceedings of ACL*, 3645-3650.

7 Retrieval-Augmented Generation: tra memoria esterna, retrieval e LLMs

7.1 Introduzione

I modelli di linguaggio di grandi dimensioni rappresentano uno dei risultati più impressionanti della moderna NLP. Sono in grado di generare testo fluido, coerente e spesso sorprendentemente informato. Tuttavia, questa capacità nasconde una fragilità strutturale: il loro sapere è implicito, distribuito nei parametri, e soprattutto non verificabile.

Un modello come GPT non possiede un archivio consultabile. Non può "andare a controllare" una fonte. Può solo produrre la continuazione più probabile di una sequenza, sulla base di ciò che ha appreso durante il training. Questo significa che, quando il modello non ha conoscenza sufficiente su un argomento, non si ferma: continua comunque a generare.

È qui che emergono fenomeni come le **allucinazioni**, ovvero risposte plausibili ma false, oppure risposte corrette ma obsolete. Questo problema è particolarmente evidente nei domini specialistici, come quello medico o legale, dove la conoscenza evolve rapidamente e richiede aggiornamento continuo.

La Retrieval-Augmented Generation nasce precisamente come risposta a questo limite. Non tenta di rendere il modello "onnisciente", ma introduce un principio diverso: separare la capacità di generare linguaggio dalla capacità di accedere alla conoscenza.

7.1.1 Il problema dell'aggiornamento

Consideriamo un esempio concreto. Supponiamo di chiedere a un LLM addestrato fino al 2023 informazioni su eventi del 2024. Il modello non può sapere cosa è accaduto dopo il suo training cutoff. Può solo:

- Inventare una risposta plausibile (allucinazione)
- Ammettere di non sapere (se opportunamente istruito)
- Generalizzare da pattern simili visti durante il training

Nessuna di queste opzioni è soddisfacente per applicazioni che richiedono precisione fattuale.

La RAG risolve questo problema fondamentale: invece di affidarsi solo alla memoria parametrica del modello, introduce una memoria esterna aggiornabile, interrogabile e verificabile.

7.2 Dal sapere implicito al sapere accessibile

Per comprendere il salto concettuale introdotto dalla RAG, è utile esplicitare una distinzione fondamentale.

Un LLM tradizionale funziona come una memoria compressa. Tutto ciò che "sa" è codificato nei pesi del modello. Non esiste un accesso selettivo a questa conoscenza: non possiamo interrogare direttamente i suoi parametri per estrarre un fatto specifico.

La RAG introduce invece una seconda forma di memoria: una memoria esterna, esplicita, interrogabile.

Possiamo quindi distinguere due livelli:

- **Memoria parametrica (interna):** conoscenza distribuita nei parametri del modello, appresa durante il pretraining. È implicita, compressa, difficile da modificare.

- **Memoria non-parametrica (esterna):** documenti, database, collezioni testuali. È esplicita, strutturata, facilmente aggiornabile.

Il punto cruciale è che la risposta non viene più generata esclusivamente dalla memoria interna, ma è il risultato dell'interazione tra queste due fonti.

Questo porta a un cambiamento epistemologico profondo:

Un modello non deve più sapere tutto. Deve sapere come trovare ciò che non sa.

7.3 Information Retrieval: il problema del trovare

Prima di comprendere a fondo la RAG, è necessario fare un passo indietro e introdurre il campo dell'Information Retrieval (IR).

L'IR si occupa di un problema apparentemente semplice: dato un bisogno informativo, trovare i documenti più rilevanti in una collezione.

Consideriamo una query come:

"Come funziona la fotosintesi?"

Un sistema di Information Retrieval non deve generare una risposta. Deve selezionare, da una collezione di documenti, quelli più pertinenti.

Questo problema è tutt'altro che banale, perché introduce immediatamente una tensione tra due dimensioni:

- **Precision (Precisione):** quanto i documenti recuperati sono rilevanti rispetto alla query
- **Recall (Richiamo):** quanta della conoscenza rilevante disponibile viene effettivamente recuperata

Formalmente:

$$\text{Precision} = \frac{\text{documenti rilevanti recuperati}}{\text{documenti totali recuperati}}$$

$$\text{Recall} = \frac{\text{documenti rilevanti recuperati}}{\text{documenti rilevanti totali}}$$

Esiste un trade-off fondamentale: aumentare il recall spesso significa diminuire la precision, e viceversa.

7.3.1 Approcci classici: TF-IDF e BM25

Storicamente, l'IR è stato affrontato con modelli basati su parole (bag-of-words), come TF-IDF e BM25. Questi modelli rappresentano i documenti come vettori di frequenze lessicali.

TF-IDF (Term Frequency-Inverse Document Frequency) assegna un peso a ogni termine in base a:

- Quanto frequentemente appare nel documento (TF)
- Quanto raramente appare nell'intera collezione (IDF)

L'intuizione è semplice: parole comuni come "il", "di", "e" appaiono ovunque e hanno poco valore discriminante. Parole rare e specifiche sono più informative.

La formula classica è:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log \left(\frac{N}{\text{DF}(t)} \right)$$

dove t è il termine, d il documento, N il numero totale di documenti, e $\text{DF}(t)$ il numero di documenti contenenti t .

BM25 è un'evoluzione di TF-IDF che introduce saturazione: dopo un certo punto, aumentare la frequenza di un termine non aumenta significativamente il suo peso.

Ma questi approcci presentano un limite evidente: **non catturano il significato**.

Ad esempio, una query su "auto" potrebbe non recuperare documenti che parlano di "automobile" o "veicolo", anche se semanticamente equivalenti. Questo è noto come il problema del **vocabulary mismatch**.

7.4 Semantica distribuzionale e retrieval semantico

Come visto nel capitolo precedente, la semantica distribuzionale rappresenta le parole come vettori in uno spazio continuo, dove la vicinanza riflette similarità semantica.

La stessa idea viene estesa ai documenti attraverso gli **embeddings densi**.

In un sistema di **dense retrieval**, sia le query che i documenti vengono proiettati nello stesso spazio vettoriale ad alta dimensionalità (tipicamente 768 o 1024 dimensioni).

Il processo è il seguente:

1. La query viene trasformata in un embedding $\vec{q} \in \mathbb{R}^d$
2. Ogni documento (o porzione di documento) viene rappresentato da un embedding $\vec{d}_i \in \mathbb{R}^d$
3. Si calcola la similarità tra query e documenti
4. Si selezionano i k documenti più vicini (top-k retrieval)

La misura di similarità più comune è la **similarità coseno**:

$$\text{sim}(\vec{q}, \vec{d}) = \cos(\theta) = \frac{\vec{q} \cdot \vec{d}}{\|\vec{q}\| \|\vec{d}\|}$$

Alternativamente, si può usare la distanza euclidea o il prodotto scalare (dot product).

7.4.1 Modelli per dense retrieval

I modelli più utilizzati per generare embeddings sono:

- **Sentence-BERT (SBERT)**: variante di BERT con architetture siamesi o triplette per generare embeddings di frasi semanticamente significativi, confrontabili tramite similarità coseno. Risolve l'inefficienza computazionale di BERT per task come semantic similarity search e clustering, riducendo il tempo da 65 ore a 5 secondi per 10.000 frasi su STS benchmark.
- **DPR (Dense Passage Retrieval)**: framework di retrieval neurale con encoder BERT duali che mappano query e passaggi in spazio vettoriale denso a 768 dimensioni. Supera i metodi sparsi come BM25 del 9-19% in accuracy top-20 per open-domain question answering.
- **ColBERT**: utilizza embeddings contestualizzati a livello di token anziché a livello di documento intero. L'architettura di "late interaction" calcola similarità fine-grained tramite MaxSim, consentendo retrieval efficiente e scalabile su grandi collezioni testuali.
- **Contriever**: modello di retrieval denso self-supervised addestrato tramite contrastive learning senza necessità di dati supervisionati. Competitivo con BM25 su benchmark BEIR e performante in recall@100 dopo fine-tuning.

Questi modelli sono tipicamente addestrati con un obiettivo contrastivo: massimizzare la similarità tra query e documenti rilevanti, minimizzarla

con documenti non rilevanti.

7.5 Architettura RAG: uno schema concettuale

Possiamo ora descrivere la RAG come una composizione di due sistemi:

- Un sistema di retrieval (IR)
- Un sistema generativo (LLM)

Lo schema generale è il seguente:

Utente → **Query** → **Retriever** → **Documenti** → **LLM** → **Risposta**

Ma questo schema lineare nasconde una struttura più ricca. Possiamo identificare cinque componenti fondamentali:

1. **Document Store**: la collezione di documenti
2. **Indexing**: preprocessing e creazione di embeddings
3. **Query Processing**: trasformazione della query in embedding
4. **Retrieval**: selezione dei documenti più rilevanti
5. **Generation**: sintesi della risposta dal contesto recuperato

7.5.1 Esempio completo

Consideriamo la domanda:

"Quali sono i sintomi del diabete di tipo 2?"

Il processo RAG si sviluppa in più fasi:

1. Encoding della query

La domanda viene trasformata in un embedding vettoriale attraverso un encoder (es. SBERT):

$$\vec{q} = \text{Encoder}(\text{"Quali sono i sintomi del diabete di tipo 2?"})$$

2. Retrieval

Il sistema cerca nei documenti medici pre-indicizzati e recupera, ad esempio, i 3 documenti più simili:

- Doc 1 (sim=0.89): "Il diabete di tipo 2 è caratterizzato da iperglicemia persistente dovuta a insulino-resistenza..."
- Doc 2 (sim=0.85): "Sintomi comuni del diabete includono sete eccessiva (polidipsia), minzione frequente (poliuria), affaticamento..."
- Doc 3 (sim=0.82): "La diagnosi di diabete si basa su glicemia a digiuno superiore a 126 mg/dL..."

3. Costruzione del prompt

Il prompt augmented diventa:

Contesto fornito:

- [Doc 1]
- [Doc 2]
- [Doc 3]

Domanda: Quali sono i sintomi del diabete di tipo 2?

Rispondi basandoti esclusivamente sul contesto fornito sopra.

4. Generazione

Il modello produce una risposta sintetizzata:

"I sintomi principali del diabete di tipo 2 includono sete eccessiva (polidipsia), aumento della minzione (poliuria), affaticamento e, in alcuni casi, visione offuscata. Questi sintomi sono causati dall'iperglicemia persistente dovuta all'insulino-resistenza."

La risposta non è più una semplice continuazione linguistica: è una sintesi guidata da evidenze documentali.

7.5.2 Varianti architetturali

Esistono diverse varianti della RAG:

- **RAG-Sequence:** ogni documento recuperato genera una risposta separata, poi si aggregano
- **RAG-Token:** a ogni step di generazione si può re-interrogare il retriever
- **Self-RAG:** il modello decide autonomamente quando invocare il retrieval
- **Iterative RAG:** si alternano cicli di retrieval e generazione per query complesse

7.6 Chunking: il problema della granularità della conoscenza

Un aspetto cruciale della RAG riguarda la rappresentazione dei documenti.

Non si lavora direttamente con documenti interi, ma con porzioni più piccole chiamate *chunk*. Questo è necessario per tre motivi fondamentali:

- Gli LLM hanno limiti di contesto (context window): non possiamo passare documenti di centinaia di pagine
- Troppa informazione irrilevante degrada la qualità della risposta (effetto "dilution")
- Chunk più piccoli consentono retrieval più preciso: invece di recuperare un intero documento, recuperiamo solo il paragrafo rilevante

Ma questo introduce una nuova versione del problema della granularità, già visto nella semantica lessicale.

Qual è la dimensione "giusta" di un'unità di significato?

Se i chunk sono troppo piccoli (es. singole frasi), perdiamo contesto. Una frase isolata può essere ambigua o incomprensibile senza il contesto circostante.

Se sono troppo grandi (es. pagine intere), introduciamo rumore e rischiamo di superare il context window dell'LLM.

7.6.1 Strategie di chunking

1. Chunking fisso

L'approccio più semplice: dividere il testo ogni N token o caratteri.

- **Pro:** semplice da implementare, veloce
- **Contro:** ignora la struttura del testo, può spezzare frasi o concetti

Tipicamente si usa $N \in [256, 512, 1024]$ token, con overlap del 10-20% per non perdere contesto ai confini.

2. Chunking strutturale

Si usano i marcatori naturali del testo: paragrafi, sezioni, capitoli.

- **Pro:** rispetta la struttura logica del documento
- **Contro:** chunk di dimensione molto variabile

3. Chunking semantico

L'approccio più sofisticato: individuare punti in cui il contenuto cambia significativamente.

Il metodo tipico utilizza una finestra scorrevole:

1. Si seleziona una porzione di testo (es. 3 frasi)
2. La si divide in due parti (es. prime 2 frasi vs. ultima frase)
3. Si calcola l'embedding di ciascuna parte
4. Si calcola la distanza coseno tra i due embedding
5. Se la distanza supera una soglia, si inserisce un confine di chunk

Se la distanza è elevata, significa che le due parti trattano argomenti diversi, e quindi rappresentano un punto naturale di separazione.

Questo approccio trasforma il chunking da operazione sintattica a operazione semantica.

Esempio:

Testo:

"La fotosintesi è il processo attraverso cui le piante convertono luce in energia. Avviene nei cloroplasti. [ALTA SIMILARITÀ] I cloroplasti contengono clorofilla. [BASSA SIMILARITÀ] Il cambio climatico sta accelerando. Le temperature globali sono aumentate."

Il chunking semantico riconoscerebbe il salto tematico e creerebbe due chunk separati.

7.6.2 Metadata enrichment

Oltre al chunking, è fondamentale arricchire ogni chunk con metadati:

- Titolo del documento di origine
- Autore, data di pubblicazione
- Sezione/capitolo
- Parole chiave

Questi metadati possono essere utilizzati per:

- Filtrare i risultati (es. "solo articoli dopo il 2020")
- Re-rankare i chunk (dare priorità a fonti autorevoli)
- Fornire citazioni nella risposta generata

7.7 Vector Databases e scalabilità

Per rendere il retrieval efficiente su collezioni di milioni di documenti, non possiamo calcolare la similarità con ogni chunk: sarebbe troppo lento.

Servono strutture dati specializzate: i **vector database**.

Un vector database indicizza gli embedding in modo da consentire ricerche di similarità approssimate ma molto veloci. I metodi più comuni sono:

- **FAISS (Facebook AI Similarity Search)**: libreria open-source di Meta, supporta CPU e GPU
- **Annoy**: sviluppata da Spotify, usa random projection trees

- **HNSW (Hierarchical Navigable Small World):** grafo navigabile gerarchico
- **Pinecone, Weaviate, Qdrant:** soluzioni managed/enterprise
- **ChromaDB:** open-source pensato per semplicità d'uso e integrazione rapida con applicazioni LLM.

7.7.1 Approximate Nearest Neighbors (ANN)

L'idea di base è semplice: invece di confrontare la query con tutti gli embedding, creiamo una struttura che ci permette di "saltare" regioni dello spazio che sicuramente non contengono i vicini più prossimi.

Ad esempio, HNSW costruisce un grafo multi-livello:

- Livelli superiori: navigazione grossolana, salti grandi
- Livelli inferiori: navigazione fine, salti piccoli

Questo riduce la complessità da $O(n)$ (ricerca lineare) a $O(\log n)$, rendendo possibile il retrieval in real-time anche su dataset enormi.

7.8 Hybrid Search: combinazione livelli lessicale e semantico

Un approccio interessante è **hybrid search**, che combina retrieval lessicale (BM25) e semantico (embeddings).

L'intuizione è che i due approcci hanno punti di forza complementari:

- **BM25:** eccellente per keyword esatte, nomi propri, codici, date
- **Semantic search:** eccellente per concetti, sinonimi, parafrasi

La combinazione tipica usa un punteggio pesato:

$$\text{score}(q, d) = \alpha \cdot \text{BM25}(q, d) + (1 - \alpha) \cdot \text{similarity}(\vec{q}, \vec{d})$$

dove $\alpha \in [0, 1]$ è un parametro che bilancia i due contributi.

Alcuni sistemi usano approcci più sofisticati, come il **re-ranking**: prima si recuperano molti candidati con BM25 (veloce), poi si re-rankano con embeddings (più accurati).

7.9 RAG come integrazione di paradigmi

La RAG è interessante non solo come tecnica, ma come punto di convergenza tra tre grandi paradigmi della NLP:

- **Semantica distribuzionale**: rappresentazione vettoriale del significato
- **Information Retrieval**: selezione di informazione rilevante
- **Modelli generativi**: produzione di linguaggio naturale

Possiamo vedere la RAG come una pipeline in cui:

- Il significato viene **rappresentato** (embeddings)
- Il significato viene **cercato** (retrieval)
- Il significato viene **espresso** (generazione)

Questo la rende una delle architetture più complete nel panorama attuale, e un esempio di come diverse tradizioni di ricerca in NLP possano integrarsi sinergicamente.

7.10 Evaluation: misurare la qualità di un sistema RAG

Valutare un sistema RAG è complesso perché coinvolge due componenti:

7.10.1 Metriche per il retrieval

- **Precision@k**: quanti dei top-k documenti recuperati sono rilevanti
- **Recall@k**: quanti documenti rilevanti sono nei top-k
- **MRR (Mean Reciprocal Rank)**: posizione media del primo documento rilevante
- **NDCG (Normalized Discounted Cumulative Gain)**: tiene conto dell'ordine e della rilevanza graduata

7.10.2 Metriche per la generazione

- **Faithfulness**: la risposta è supportata dai documenti recuperati?
- **Answer Relevance**: la risposta è pertinente alla query?
- **Context Relevance**: i documenti recuperati sono pertinenti?

Framework come **RAGAS** (Retrieval-Augmented Generation Assessment) forniscono metriche automatiche per questi aspetti.

7.10.3 End-to-end evaluation

Idealmente, si valuta il sistema nel suo complesso:

- Dataset annotati con (query, risposta corretta, documenti rilevanti)

- Metriche come BLEU, ROUGE (limitazioni: focus su n-gram overlap)
- Valutazione umana: accuracy, completezza, coerenza
- LLM-as-a-judge: usare un LLM forte per valutare le risposte

7.11 Limiti e questioni aperte

Nonostante i vantaggi, la RAG non elimina le difficoltà fondamentali.

Il problema principale resta il seguente:

Il modello non capisce i documenti. Li utilizza.

Il retrieval introduce grounding, ma non garantisce comprensione semantica profonda.

7.11.1 Limiti tecnici

- **Dipendenza critica dalla qualità del retrieval:** se il retriever non trova documenti rilevanti, l'LLM non può generare una buona risposta. "Garbage in, garbage out".
- **Difficoltà nel combinare informazioni da fonti multiple:** se la risposta richiede sintesi di informazioni sparse in molti documenti, la RAG fatica.
- **Limiti della finestra di contesto:** anche con 100k+ token, non possiamo includere intere biblioteche.
- **Latenza:** il retrieval aggiunge overhead temporale.
- **Contraddizioni tra fonti:** cosa fare se documenti recuperati si contraddicono?

7.11.2 Problemi aperti

- **Reasoning multi-hop:** query che richiedono concatenare informazioni da step multipli
- **Temporal reasoning:** gestire informazioni che cambiano nel tempo
- **Bias e fairness:** i documenti possono contenere bias, che vengono amplificati
- **Attribution e citation:** come fornire citazioni precise e verificabili

7.12 Direzioni future

La ricerca sulla RAG sta evolvendo rapidamente. Alcune direzioni promettenti:

- **Adaptive RAG:** il sistema decide autonomamente quando e come usare il retrieval
- **Multimodal RAG:** integrare testo, immagini, tabelle, codice
- **Conversational RAG:** gestire dialoghi multi-turno con memoria
- **Fine-tuning del retriever:** addestrare il retriever end-to-end con il generatore
- **Knowledge editing:** aggiornare selettivamente la conoscenza del modello

7.13 Conclusione

La Retrieval-Augmented Generation rappresenta una delle evoluzioni più significative nell’NLP recente.

Non sostituisce i modelli generativi, ma li completa. Non elimina le allucinazioni, ma le riduce drasticamente. Non risolve il problema del significato, ma lo riformula in termini più pragmatici.

Soprattutto, introduce una nuova idea:

Il significato non è solo ciò che un modello ha appreso, ma anche ciò che è in grado di recuperare e utilizzare nel momento giusto.

Questo sposta l'attenzione dalla rappresentazione statica del significato alla sua **attivazione dinamica e contestuale**.

E apre una nuova direzione per la ricerca:

Non più modelli che sanno tutto, ma sistemi che sanno interagire intelligentemente con la conoscenza.

Nel laboratorio che segue, implementeremo un sistema RAG completo per esplorare concretamente questi concetti.

7.14 Lab-5: Costruire un sistema RAG su articoli scientifici NLP

In questo laboratorio costruiremo un sistema RAG completo utilizzando il dataset MaartenGr/arxiv_nlp di Hugging Face, che contiene abstract di articoli NLP da arXiv.

L'obiettivo è creare un assistente che possa rispondere a domande sulla letteratura scientifica in NLP, citando le fonti.

Il codice qui di seguito riportato è presente anche sulla pagina del corso, in maniera più fruibile (rag_lab_essenziale.ipynb).

7.14.1 Setup e dipendenze

Per prima cosa, installiamo le librerie necessarie:

```
pip install datasets transformers sentence-transformers \
    faiss-cpu torch accelerate
```

Librerie utilizzate:

- datasets: per caricare il dataset da Hugging Face
- sentence-transformers: per generare embeddings semantici
- faiss: vector database per retrieval efficiente
- transformers: per il modello generativo locale
- torch: backend per i modelli

7.14.2 Step 1: Caricamento e preprocessing dei dati

```
from datasets import load_dataset
```

```
import pandas as pd

# Carica il dataset
print("Caricamento dataset arXiv NLP...")
dataset = load_dataset("MaartenGr/arxiv_nlp")
df = pd.DataFrame(dataset['train'])

# Usa subset per testing (rimuovi per dataset completo)
df = df.head(500)

print(f"Numero di articoli: {len(df)}")
print(f"Colonne: {df.columns.tolist()}")

# Prepara i documenti per la RAG
documents = []
metadata = []

# Auto-rileva colonne (gestisce varianti del dataset)
title_col = next((c for c in df.columns
                  if 'title' in c.lower()), df.columns[0])
abstract_col = next((c for c in df.columns
                    if 'abstract' in c.lower()
                    or 'summar' in c.lower()), df.columns[1])

for idx, row in df.iterrows():
    # Estrai campi in modo sicuro
    title = str(row[title_col]) if pd.notna(row[title_col]) \
        else 'No Title'
    abstract = str(row[abstract_col]) if pd.notna(row[abstract_col]) \
        else 'No Abstract'

    # Testo completo del chunk
    text = f"Title: {title}\n\nAbstract: {abstract}"
    documents.append(text)

# Metadata associati
```

```
metadata.append({
    'title': title,
    'year': str(row.get('year', 'Unknown'))
})

print(f"Preparati {len(documents)} documenti")
```

Note:

- L'auto-rilevamento delle colonne gestisce diverse versioni del dataset
- Combiniamo titolo e abstract perché sono semanticamente coesi
- Manteniamo metadata separati per citazioni

7.14.3 Step 2: Generazione degli embeddings

```
from sentence_transformers import SentenceTransformer

# Carica il modello di embedding
print("Caricamento modello di embedding...")
embedding_model = SentenceTransformer('all-MiniLM-L6-v2')

# Genera embeddings per tutti i documenti
print("Generazione embeddings...")
embeddings = embedding_model.encode(
    documents,
    show_progress_bar=True,
    batch_size=32,
    convert_to_numpy=True,
    normalize_embeddings=True
)

print(f"Embeddings generati: shape {embeddings.shape}")
# Output: (N_documenti, 384)
```

Scelta del modello:

- all-MiniLM-L6-v2: veloce, 384 dimensioni
- all-mpnet-base-v2: più accurato, 768 dimensioni
- Per domini specifici: SciBERT per testi scientifici

7.14.4 Step 3: Creazione del vector store con FAISS

```
import faiss

# Crea l'indice FAISS
dimension = embeddings.shape[1]
index = faiss.IndexFlatIP(dimension) # Inner Product

# Aggiungi gli embeddings (già normalizzati)
index.add(embeddings)

print(f"Indice FAISS creato con {index.ntotal} vettori")

# Salva l'indice per riutilizzo
faiss.write_index(index, "arxiv_nlp_faiss.index")
print("Indice salvato su disco")
```

Tipi di indici FAISS:

- IndexFlatIP: ricerca esatta, ottimale per dataset medi
- IndexIVFFlat: clustering + ricerca approssimata
- IndexHNSW: grafo navigabile, ottimo rapporto velocità/accuratezza

7.14.5 Step 4: Implementazione del retriever

```
def retrieve(query, k=5):
```

```
"""Recupera i k documenti più rilevanti"""
# Genera embedding della query
query_emb = embedding_model.encode(
    [query],
    convert_to_numpy=True,
    normalize_embeddings=True
)

# Cerca i k nearest neighbors
similarities, indices = index.search(query_emb, k)

# Prepara i risultati
results = []
for i, (sim, idx) in enumerate(zip(similarities[0], indices[0])):
    results.append({
        'rank': i + 1,
        'similarity': float(sim),
        'document': documents[idx],
        'metadata': metadata[idx]
    })

return results

# Test
test_query = "What are transformer models?"
results = retrieve(test_query, k=3)

print(f"Query: {test_query}\n")
for r in results:
    print(f"[{r['rank']}] Similarity: {r['similarity']:.3f}")
    print(f"    {r['metadata']['title'][:70]}")
```

7.14.6 Step 5: Integrazione con LLM locale

```
from transformers import AutoTokenizer, AutoModelForCausalLM, pipeline
```

```
import torch

# Carica modello locale (Qwen 0.5B)
model_id = "Qwen/Qwen2.5-0.5B-Instruct"
print(f"Caricamento {model_id}...")

device = "cuda" if torch.cuda.is_available() else "cpu"

tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(
    model_id,
    torch_dtype=torch.float16 if device == "cuda" else torch.float32,
    device_map="auto" if device == "cuda" else None,
    low_cpu_mem_usage=True
)

llm = pipeline(
    'text-generation',
    model=model,
    tokenizer=tokenizer,
    device=0 if device == "cuda" else -1,
    max_new_tokens=300
)

print(f"Modello caricato su {device}")
```

Modelli disponibili:

- Qwen2.5-0.5B: 500MB, veloce, qualità buona
- SmolLM-360M: 300MB, velocissimo
- Phi-2: 5GB, qualità eccellente
- OpenAI GPT-3.5/4: richiede API key

7.14.7 Step 6: Sistema RAG completo

```
def rag_answer(query, k=3):
    """Risponde alla query usando RAG"""

    print(f"Query: {query}\n")

    # 1. Retrieval
    results = retrieve(query, k=k)

    print("Documenti recuperati:")
    for r in results:
        print(f"    [{r['rank']}] {r['metadata']['title'][:60]}")
        print(f"        Similarity: {r['similarity']:.3f}")

    # 2. Costruisci contesto
    context_parts = []
    for r in results:
        context_parts.append(
            f"[Document {r['rank']}] \n{r['document']}"
        )

    context = "\n\n".join(context_parts)

    # Limita contesto per modelli piccoli
    if len(context) > 1500:
        context = context[:1500] + "\n[...truncated...]"

    # 3. Costruisci prompt
    messages = [
        {
            "role": "system",
            "content": "You are a helpful assistant specialized "
                        "in NLP research."
        },
        {
```

```
        "role": "user",
        "content": f""""Context from research papers:

{context}

Question: {query}

Answer based on the context above:""""
    }
]

prompt = tokenizer.apply_chat_template(
    messages,
    tokenize=False,
    add_generation_prompt=True
)

# 4. Genera risposta
print("\nGenerazione risposta...\n")

output = llm(
    prompt,
    max_new_tokens=250,
    do_sample=True,
    temperature=0.3,
    pad_token_id=tokenizer.eos_token_id
)

# Estrai risposta
full_text = output[0]['generated_text']
answer = full_text[len(prompt):].strip() \
    if full_text.startswith(prompt) else full_text

print("=" * 60)
print("ANSWER:")
print("=" * 60)
```



```
print(answer)

print("\n" + "=" * 60)
print("SOURCES:")
print("=" * 60)
for r in results:
    print(f"- {r['metadata']['title']} "
          f"({r['metadata']['year']})")

return answer

# Esempio d'uso
result = rag_answer(
    "What is BERT and how does it work?",
    k=3
)
```

7.14.8 Estensioni e miglioramenti

Eventuali estensioni e miglioramenti di questo sistema RAG di base possono essere considerati come progetti di laboratorio finali (oltre i LAB-*).

Esempi possono essere:

- **Hybrid search:** combinare BM25 (lessicale) + embeddings (semantico)
- **Re-ranking:** usare cross-encoder per migliorare precision
- **Evaluation:** misurare Precision@k, Recall@k, faithfulness
- **Self-RAG:** il modello decide quando fare retrieval
- **Iterative RAG:** alternare cicli retrieval-generazione
- **Interfaccia:** creare UI con Gradio o Streamlit

7.14.9 Conclusioni del laboratorio

In questo laboratorio abbiamo:

- Costruito un sistema RAG completo end-to-end
- Utilizzato embeddings semantici per retrieval
- Integrato un LLM locale per generazione
- Testato il sistema su query reali

Takeaway principali:

1. La qualità del retrieval è critica: un cattivo retriever rende inutile anche il miglior LLM
2. Il chunking richiede bilanciamento tra contesto e precisione
3. Hybrid search (lessicale + semantico) spesso supera approcci singoli
4. I modelli locali (Qwen, Phi) sono alternative valide a OpenAI
5. La valutazione è multidimensionale: retrieval, faithfulness, relevance

La RAG trasforma gli LLM da “oracoli statici” a “ricercatori dinamici”, capaci di accedere, valutare e sintetizzare conoscenza esterna. Questo apre possibilità enormi, ma richiede attenzione a dettagli tecnici e design architetturale.

8 Large Language Models, Prompting e Human-AI Interaction

8.1 Introduzione

I Large Language Models (LLMs) rappresentano un punto di svolta nell'elaborazione del linguaggio naturale. A differenza dei modelli tradizionali, addestrati per task specifici, gli LLMs sono sistemi generali capaci di svolgere molteplici compiti attraverso il solo linguaggio naturale.

Questa capacità emerge da due fattori chiave. Innanzitutto, la **scala**: modelli con miliardi di parametri addestrati su enormi quantità di testo catturano pattern linguistici complessi e conoscenza implicita. In secondo luogo, l'**architettura Transformer**: il meccanismo di attenzione consente di catturare dipendenze a lungo raggio nel testo, superando i limiti delle architetture ricorrenti precedenti.

Ma ciò che rende gli LLMs particolarmente significativi non è solo la loro capacità tecnica, bensì il modo in cui modificano l'interazione tra umani e sistemi di intelligenza artificiale. Per la prima volta, possiamo comunicare con un sistema computazionale utilizzando linguaggio naturale non vincolato, senza dover tradurre le nostre intenzioni in comandi predefiniti o interfacce strutturate.

8.1.1 Dai modelli base agli instruction-tuned models

Non tutti gli LLMs sono uguali. Esistono due grandi categorie che differiscono profondamente nel comportamento e nell'utilizzo.

I **base language models** sono addestrati esclusivamente sulla predizione del token successivo. Questi modelli completano testo in modo fluido e coerente, ma non seguono istruzioni esplicite. Se chiediamo a un modello base “La capitale dell'Italia è”, otterremo una continuazione naturale come “Roma, situata nel Lazio centrale, è una città di circa tre milioni di abitanti...”. Il modello completa il testo come se stesse scrivendo un articolo, ma non sta rispondendo a una domanda.

Gli **instruction-tuned models**, invece, sono stati ulteriormente addestrati tramite fine-tuning su coppie domanda-risposta. Questi modelli hanno imparato a interpretare l'input come una richiesta e a generare risposte appropriate. Alla stessa domanda “Qual è la capitale dell'Italia?”, un modello instruction-tuned risponderà semplicemente “La capitale dell'Italia è Roma”, riconoscendo che si tratta di una domanda a cui fornire una risposta diretta.

Il passaggio da modelli base a instruction-tuned avviene attraverso diverse tecniche. La più diretta è il **Supervised Fine-Tuning** (SFT), dove il modello viene addestrato su migliaia di esempi annotati di interazioni umane di alta qualità. Tuttavia, questa tecnica da sola non è sufficiente per catturare le preferenze umane complesse.

Per questo motivo, molti modelli moderni utilizzano il **Reinforcement Learning from Human Feedback** (RLHF). In questo approccio, umani valutano diverse risposte del modello per la stessa domanda, creando un modello di ricompensa che cattura le preferenze umane. Il modello viene poi ottimizzato per massimizzare questa ricompensa. Più recentemente, tecniche come **Direct Preference Optimization** (DPO) hanno semplificato questo processo, ottenendo risultati simili senza la complessità del reinforcement learning esplicito.

8.1.2 Risorse per comprendere i Transformer

Per una comprensione approfondita dell'architettura alla base degli LLMs:

1. [The Illustrated Transformer](#) (visualizzazione interattiva)
2. [Visual Intro to Transformers](#) (spiegazione video)
3. [Attention in Transformers Visually Explained](#)
4. [Attention Is All You Need](#) (paper originale)

8.2 Prompting: L'arte di comunicare con gli LLMs

Il prompting è il processo di formulare input testuali che guidano il comportamento del modello. Non si tratta di semplice “dare comandi”, ma di costruire un'interfaccia linguistica efficace tra intenzione umana e capacità del modello. È un'abilità che richiede comprensione sia delle potenzialità che dei limiti degli LLMs.

8.2.1 Perché il prompting è cruciale

A differenza dei sistemi tradizionali, dove l'interazione è mediata da interfacce grafiche predefinite con pulsanti, menu e moduli, con gli LLMs l'interfaccia è **il linguaggio stesso**. Questo cambiamento apparentemente semplice introduce implicazioni profonde.

Da un lato, otteniamo una **flessibilità** senza precedenti: possiamo esprimere task arbitrariamente complessi senza dover aspettare che qualcuno implementi una funzionalità specifica. Dall'altro, dobbiamo confrontarci con l'**ambiguità** intrinseca del linguaggio naturale. La stessa richiesta può essere interpretata in modi diversi, e piccole variazioni nel prompt

possono portare a output molto diversi. Questa **imprevedibilità** richiede un approccio iterativo e sperimentale.

8.2.2 Principi fondamentali del prompting

Il primo principio fondamentale è la **chiarezza e specificità**. Un prompt vago produce inevitabilmente output generici. Consideriamo la differenza tra chiedere semplicemente “Parlami di Roma” e fornire una richiesta dettagliata: “Descrivi in tre paragrafi la storia di Roma antica, concentrandoti sul periodo repubblicano (509-27 a.C.). Includi l’organizzazione politica, l’espansione territoriale, e i fattori che hanno portato alla transizione verso l’impero.” La seconda versione fornisce vincoli chiari che guidano il modello verso un output più utile e rilevante.

Un secondo aspetto importante riguarda l’**uso di delimitatori** per separare chiaramente le diverse componenti del prompt. Quando forniamo un testo da analizzare insieme alle istruzioni su come analizzarlo, è essenziale che il modello non confonda le due parti. Utilizziamo quindi marcatori espliciti come virgolette triple, tag XML, o separatori visivi per delineare le sezioni.

Altrettanto importante è **specificare il formato di output** desiderato. Se vogliamo che il modello estraiga informazioni strutturate da un testo, possiamo chiedere esplicitamente un formato JSON con campi specifici. Se desideriamo un riassunto, possiamo indicare il numero di frasi o parole. Questa precisione elimina ambiguità e rende l’output direttamente utilizzabile.

Infine, il prompting è intrinsecamente un **processo iterativo**. Raramente il primo tentativo produce il risultato ottimale. L’approccio più efficace consiste nel generare una prima versione del prompt, analizzare attentamente l’output, identificare i problemi (ambiguità, informazioni mancanti, formato errato), raffinare il prompt di conseguenza, e ripetere questo ciclo fino a ottenere risultati soddisfacenti. Ogni iterazione ci insegna qualcosa sul comportamento del modello specifico che stiamo utilizzando.

8.2.3 Strategie avanzate di prompting

Oltre ai principi base, esistono tecniche più sofisticate che migliorano significativamente le prestazioni degli LLMs su task complessi.

Chain-of-Thought (CoT)

Una delle scoperte più significative nell'ambito del prompting è che chiedere al modello di “pensare ad alta voce” migliora drasticamente le performance su task di ragionamento. Invece di richiedere direttamente la risposta finale, chiediamo al modello di esplicitare i passaggi intermedi del suo ragionamento.

Consideriamo un semplice problema aritmetico: “Ho 15 mele. Ne mangio 3, poi ne compro il doppio di quelle mangiate. Quante mele ho ora?” Un prompt standard potrebbe ottenere una risposta errata o non ben giustificata. Se invece aggiungiamo “Ragiona passo-passo prima di rispondere”, il modello tipicamente produce un output strutturato: parte dalle 15 mele iniziali, sottrae le 3 mangiate arrivando a 12, calcola il doppio di 3 che fa 6, e infine somma per ottenere 18 mele totali.

Questo approccio è particolarmente efficace per la matematica, la logica, e qualsiasi task che richiede pianificazione o ragionamento multi-step. Il modello, forzato a esplicitare i passaggi intermedi, commette meno errori di calcolo e mantiene meglio la coerenza logica.

Few-shot Learning

A volte la descrizione verbale di un task non è sufficiente. In questi casi, fornire esempi espliciti di input-output desiderato può fare la differenza. Questa tecnica si chiama few-shot learning perché forniamo “pochi esempi” (tipicamente da 1 a 5) direttamente nel prompt.

La differenza con l'approccio zero-shot (nessun esempio) è sostanziale. Immaginiamo di voler classificare il sentiment di recensioni. Un approc-

cio zero-shot consisterebbe semplicemente nel chiedere “Classifica il sentiment: Il prodotto è arrivato rotto”. Un approccio few-shot, invece, fornirebbe prima alcuni esempi: “Ottimo prodotto, lo consiglio! → Positivo”, “Non funziona, deluso → Negativo”, “Nella media, niente di speciale → Neutro”, e solo dopo presenterebbe il caso da classificare. Gli esempi forniscono al modello un template concreto del formato e del tipo di classificazione desiderato.

Self-Consistency

Un'altra tecnica avanzata sfrutta il fatto che i modelli sono stocastici. Invece di generare una singola risposta, possiamo generare multiple soluzioni indipendenti (tipicamente con temperature leggermente più alta del normale) e poi selezionare la risposta che appare più frequentemente. Questo approccio di “majority voting” tende a migliorare l'accuratezza, specialmente su task dove esistono soluzioni ben definite.

Role Prompting

Assegnare un ruolo o una persona al modello può migliorare la qualità dell'output in modi sottili ma significativi. Prefissare il prompt con “Sei un esperto di storia romana” prima di fare domande storiche, o “Sei un programmatore senior Python” prima di chiedere codice, può influenzare positivamente il tono, il livello di dettaglio, e la competenza apparente della risposta.

8.2.4 Cosa evitare nel prompting

Alcuni pattern portano sistematicamente a risultati peggiori. Uno dei più comuni è l'uso eccessivo di **negazioni**. Invece di dire cosa il modello *non* deve fare (“Non usare termini tecnici”), è molto più efficace specificare positivamente cosa *deve* fare (“Usa linguaggio semplice comprensibile

a un bambino di 10 anni”). I modelli tendono a processare meglio istruzioni costruttive.

Un altro errore frequente è l'**ambiguità**. Pronomi vaghi (“Riassumilo”, “Traducilo”) o riferimenti non chiari creano incertezza. È sempre meglio essere espliciti: “Riassumi il testo sopra in 3 bullet points” indica chiaramente cosa riassumere e in che formato.

8.3 Task comuni con LLMs

Gli LLMs eccellono in una varietà di compiti di elaborazione del linguaggio. In questa sezione esploreremo i task più comuni, mostrando come il prompting appropriato possa guidare il modello verso output di qualità.

8.3.1 Summarization

La sintesi automatica è probabilmente uno dei task più utili e frequentemente utilizzati. Gli LLMs possono condensare documenti lunghi preservando le informazioni essenziali, ma la vera potenza emerge quando riusciamo a controllare finemente cosa e come sintetizzare.

La sintesi può essere orientata in molteplici modi. Possiamo chiedere di focalizzarsi su aspetti specifici: una recensione di ristorante può essere riassunta concentrandosi sulla qualità del cibo, sul servizio, o sul rapporto qualità-prezzo. Possiamo controllare il livello di dettaglio richiesto, specificando se vogliamo un paragrafo, tre frasi, o anche una singola parola chiave.

Un approccio particolarmente efficace è la **sintesi progressiva**, dove chiediamo al modello di produrre versioni sempre più condensate dello stesso contenuto. Partendo da un paragrafo, possiamo ridurlo a tre frasi, poi a una frase, e infine a una singola parola chiave. Questo ci permette di scegliere il livello di granularità più adatto alle nostre esigenze.

La sintesi può anche essere **orientata all'audience**. Lo stesso documento tecnico può essere riassunto in modo molto diverso per un manager (focus su implicazioni business e decisioni), per un tecnico (dettagli implementativi), o per il pubblico generale (linguaggio semplice e concetti ad alto livello).

8.3.2 Inference e Classificazione

Prima degli LLMs, task come sentiment analysis o named entity recognition richiedevano dataset annotati manualmente e training di modelli specifici. Oggi possiamo definire questi task interamente a livello di prompt.

La **sentiment analysis**, ad esempio, può essere ridotta a una semplice domanda: “Questa recensione è positiva, negativa, o neutra?” Ma possiamo andare oltre la semplice polarità. Possiamo chiedere una lista di emozioni specifiche espresse nel testo (gioia, frustrazione, delusione, entusiasmo), o un punteggio numerico su una scala da 1 a 10. La flessibilità del linguaggio naturale ci permette di adattare il task alle nostre esigenze specifiche senza dover riaddestrare modelli.

Il **named entity recognition** tradizionalmente richiedeva modelli addestrati su categorie predefinite (persona, luogo, organizzazione). Con gli LLMs possiamo definire categorie arbitrarie on-the-fly. Possiamo chiedere di estrarre non solo entità standard, ma anche concetti domain-specific come nomi di farmaci, procedure mediche, o tecnologie specifiche.

La **topic detection** è un altro esempio illuminante. Invece di addestrare modelli di topic modeling complessi, possiamo semplicemente chiedere: “Quali sono i 5 argomenti principali discussi in questo articolo?” Possiamo specificare il formato (massimo 3 parole per topic), o fornire una lista predefinita di topic e chiedere quali sono presenti nel testo.

8.3.3 Trasformazione

Gli LLMs non si limitano a analizzare testo, ma possono trasformarlo in modi sofisticati. La **traduzione** è l'esempio più ovvio, ma va ben oltre il semplice trasferimento tra lingue. Possiamo tradurre tra registri linguistici (formale a colloquiale), tra stili (accademico a divulgativo), o persino tra formati (da prosa a bullet points).

La trasformazione può essere puramente strutturale. Un file CSV può essere convertito in JSON, una tabella HTML in formato Markdown, o dati non strutturati in un database relazionale. Quello che un tempo richiedeva script complessi di parsing può ora essere espresso come una semplice richiesta in linguaggio naturale.

La **correzione di errori** è un'altra trasformazione utile. Non solo errori ortografici e grammaticali, ma anche normalizzazione di formati (date, indirizzi, numeri di telefono), pulizia di dati sporchi, o anche riparazione di JSON malformato.

8.3.4 Espansione

Il processo inverso della sintesi è l'espansione: generare testo più lungo e dettagliato a partire da input concisi. Questo è particolarmente utile per task come la generazione di email da bullet points, l'espansione di outline in articoli completi, o la creazione di documentazione a partire da specifiche tecniche schematiche.

L'espansione introduce un parametro cruciale: la **temperatura**. Questo valore controlla la casualità (o "creatività") del modello. Una temperatura bassa (0.0-0.3) produce output deterministico e conservativo, ideale per documentazione tecnica, risposte fattuali, o generazione di codice dove la consistenza è critica. Una temperatura media (0.5-0.7) bilancia prevedibilità e varietà. Una temperatura alta (0.8-1.0 o superiore) produce output più vario e imprevedibile, utile per scrittura creativa, brainstorming, o quando vogliamo esplorare molteplici varianti.

La scelta della temperatura dipende fortemente dal contesto. Per generare un contratto legale o una procedura medica, vogliamo temperatura bassissima. Per scrivere una campagna marketing creativa o generare idee per una storia, una temperatura alta può portare a risultati più interessanti e inaspettati.

8.4 Limiti e Problematiche degli LLMs

Nonostante le capacità impressionanti, gli LLMs presentano limiti strutturali importanti che è fondamentale comprendere per utilizzarli in modo responsabile ed efficace.

8.4.1 Allucinazioni

Il problema più noto e insidioso degli LLMs è la loro tendenza a generare informazioni plausibili ma completamente false, presentandole con alta confidenza. Questo fenomeno, chiamato “allucinazione”, non è un bug occasionale ma una conseguenza diretta di come questi modelli funzionano.

Consideriamo un esempio concreto. Se chiediamo a un LLM “Chi ha vinto il premio Nobel per la letteratura nel 2025?”, il modello potrebbe rispondere con sicurezza: “Lo scrittore giapponese Haruki Murakami ha vinto il Nobel per la letteratura nel 2025 per il suo romanzo ‘I confini dell’alba’”. Questa risposta è completamente inventata. Il modello non sa che siamo nel 2025, non ha accesso a informazioni post-training, e il romanzo menzionato non esiste. Eppure la risposta è formulata in modo così naturale e specifico da sembrare assolutamente credibile.

Le allucinazioni sono particolarmente pericolose perché il modello non mostra alcuna esitazione. Non dice “non sono sicuro” o “potrebbe essere”. Genera fatti inventati con la stessa fluidità di fatti reali. Questo deriva dal fatto che gli LLMs sono fondamentalmente generatori statistici

di testo: producono ciò che è più probabile seguire il prompt, non ciò che è vero.

Esistono strategie per mitigare questo problema. L'approccio più efficace è il Retrieval-Augmented Generation (RAG), visto nel capitolo precedente, che ancora il modello a documenti reali verificabili. Possiamo anche chiedere esplicitamente citazioni o fonti, richiedere verifiche incrociate, o calibrare la confidenza del modello. Ma nessuna di queste tecniche elimina completamente il rischio.

8.4.2 Conoscenza statica

Gli LLMs "congelano" la loro conoscenza al momento del training. Un modello addestrato con dati fino al 2023 non sa nulla di eventi successivi. Non può aggiornarsi autonomamente, non legge le notizie, non apprende continuamente dal mondo.

Questo crea situazioni paradossali. Il modello può spiegare in dettaglio eventi storici di secoli fa, ma non sa chi ha vinto le elezioni dell'anno scorso. Può descrivere teorie scientifiche consolidate, ma ignora scoperte recentissime. La sua conoscenza è vasta ma datata.

La soluzione più robusta è l'integrazione con sistemi di retrieval esterni o strumenti di ricerca. Invece di affidarsi alla memoria statica del modello, questi sistemi recuperano informazioni aggiornate da database, motori di ricerca, o API in tempo reale. Il modello mantiene la capacità di comprendere e sintetizzare, ma attinge a fonti fresche.

8.4.3 Bias e fairness

I modelli riflettono inevitabilmente i bias presenti nei dati di training. Se il corpus di addestramento contiene stereotipi, pregiudizi, o rappresentazioni squilibrate di gruppi sociali, il modello apprenderà e riprodurrà questi pattern.

Gli esempi documentati sono numerosi e preoccupanti. Associazioni stereotipate tra professioni e genere (medici maschi, infermiere femmine), rappresentazione squilibrata di culture e lingue minoritarie, amplificazione di pregiudizi razziali o sociali. Questi bias non sono errori superficiali ma pattern profondamente radicati nei pesi del modello.

La mitigazione richiede interventi a più livelli. Durante la creazione del dataset, attraverso curation attenta e bilanciamento. Durante il training, con tecniche di debiasing. Dopo il deployment, attraverso red teaming (test aggressivi per scoprire comportamenti problematici) e monitoraggio continuo. Possiamo anche usare istruzioni esplicite nel prompt per guidare il modello verso risposte più equilibrate, anche se questo è meno affidabile delle soluzioni tecniche.

8.4.4 Ragionamento superficiale

Gli LLMs eccellono nel pattern matching e nel completamento statistico, ma faticano con forme di ragionamento più profonde. Il loro “pensiero” è fondamentalmente diverso dal ragionamento umano.

Il ragionamento causale è particolarmente problematico. Il modello può sapere che “il fumo causa il cancro” come fatto memorizzato, ma non comprende veramente la catena causale sottostante. Non ha un modello mentale di come funzionano polmoni, cellule, o carcinogenesi. Ripete correlazioni apprese, non ragiona su meccanismi.

La pianificazione multi-step complessa è un'altra area debole. Per task che richiedono tenere traccia di stati intermedi, revisionare piani in base a nuove informazioni, o considerare controfattuali complessi, le performance degradano rapidamente. Il modello tende a perdere coerenza su catene di inferenza lunghe.

La comprensione fisica del mondo reale è quasi assente. Il modello non sa veramente cosa significa che un oggetto è pesante, che l'acqua bagna, che la gravità fa cadere le cose. Ha solo pattern testuali su questi concetti, non esperienza diretta.

8.4.5 Context window limitato

Anche i modelli più recenti con finestre di 100k+ token mostrano limiti. La performance degrada man mano che il contesto si allunga. Fenomeni come “lost in the middle” (peggiore attenzione alle informazioni centrali del contesto) sono ben documentati.

Mantenere coerenza su documenti molto lunghi è difficile. Il modello può contraddirsi tra l’inizio e la fine di una risposta lunga, dimenticare dettagli menzionati prima, o perdere il filo del discorso complessivo.

Inoltre, finestre più grandi hanno costi computazionali quadratici. Processare 100k token richiede risorse enormemente maggiori di processare 4k token, limitando l’applicabilità pratica di questi approcci.

8.4.6 Costi computazionali

Il training di un LLM all’avanguardia costa milioni di euro e produce emissioni di CO2 equiparabili a quelle di centinaia di voli intercontinentali. L’infrastruttura richiesta include migliaia di GPU specializzate in datacenter dedicati.

Anche l’inference ha costi non trascurabili. Ogni query a un modello grande consuma energia, richiede memoria, e ha una latenza misurabile. Per applicazioni ad alto volume, questi costi si accumulano rapidamente.

Il deployment richiede infrastruttura specializzata. Non si può semplicemente eseguire GPT-4 su un laptop. Servono server dedicati, ottimizzazioni software complesse, e spesso hardware specializzato come TPU o GPU high-end.

Questi fattori economici ed ambientali vanno considerati quando si decide se e come usare LLMs. Non ogni task giustifica queste risorse.

8.5 Human-AI Interaction: La nuova frontiera

L'avvento degli LLMs ha trasformato radicalmente il modo in cui gli umani interagiscono con i sistemi di AI. Non si tratta più di interfacce predefinite, ma di **conversazioni aperte**. Questo cambio di paradigma ha implicazioni profonde che vanno ben oltre la semplice comodità d'uso.

8.5.1 Dal task-oriented al goal-oriented

Le interfacce tradizionali richiedono che l'utente conosca in anticipo le funzionalità disponibili. Deve navigare menu predefiniti, compilare form strutturati, e soprattutto deve adattare il proprio obiettivo alle capacità del sistema. Se il sistema non prevede una certa funzionalità, l'utente non può ottenerla. L'interazione è rigidamente task-oriented: il sistema offre task specifici, e l'utente deve scegliere tra quelli disponibili.

Le interfacce basate su LLMs ribaltano questa dinamica. L'utente può esprimere obiettivi in linguaggio naturale, senza dover conoscere in anticipo cosa il sistema sa fare. Può raffinare iterativamente la richiesta attraverso il dialogo, scoprendo le capacità del sistema man mano che interagisce con esso. L'interazione diventa goal-oriented: l'utente descrive cosa vuole ottenere, e il sistema cerca di comprenderlo e soddisfarlo.

Questo passaggio non è solo più comodo: è qualitativamente diverso. Abbassa drasticamente la barriera d'ingresso per utilizzare tecnologie complesse. Un utente non tecnico può chiedere operazioni sofisticate senza dover imparare interfacce complesse o linguaggi di programmazione. Ma introduce anche nuove sfide: come può l'utente sapere cosa chiedere se non conosce i limiti del sistema? Come evita di chiedere l'impossibile?

8.5.2 Nuovi pattern di interazione

Questa trasformazione ha fatto emergere pattern di interazione completamente nuovi, che prima non erano possibili o praticabili.

Co-creazione

L'AI non è più uno strumento passivo che esegue comandi, ma diventa un **partner creativo** nel processo. Nella scrittura collaborativa, ad esempio, il pattern tipico è iterativo: l'umano fornisce l'idea iniziale e la direzione generale, l'AI espande con dettagli e varianti, l'umano seleziona le parti migliori e fornisce feedback, e il ciclo si ripete. Nessuno dei due attori lavora in isolamento: è una vera collaborazione.

Questo pattern si applica a molti domini. Nella programmazione, l'umano descrive l'obiettivo e l'AI genera codice, che l'umano poi rivede, raffina, e usa come punto di partenza per ulteriori richieste. Nel design, l'umano fornisce vincoli e l'AI propone soluzioni che l'umano valuta e modifica. La chiave è che l'AI non sostituisce l'umano, ma amplifica le sue capacità fornendo rapidamente alternative da esplorare.

Debugging conversazionale

Quando qualcosa non funziona, l'approccio tradizionale richiede di interpretare messaggi di errore criptici e modificare parametri o codice. Con gli LLMs, l'utente può semplicemente dire: "L'output non è quello che volevo. Voglio che sia più conciso e tecnico." Il sistema può rispondere chiedendo chiarimenti ("Quanto conciso? Posso vedere un esempio di quello che ti aspetti?"), proporre alternative, o spiegare perché ha prodotto quell'output.

Questo debugging conversazionale è particolarmente potente perché abbassa il carico cognitivo. L'utente non deve tradurre il suo problema

in termini tecnici o navigare documentazione complessa. Può esprimersi nel suo linguaggio naturale, e il sistema fa il lavoro di interpretazione.

Delega progressiva

Gli LLMs permettono livelli crescenti di autonomia nella delega di compiti. Al livello più basso, l'utente fornisce task specifici e ben definiti: "Traduci questo testo in inglese." A un livello intermedio, aggiunge vincoli e preferenze: "Traduci questo testo mantenendo il tono formale e usando terminologia tecnica." Al livello più alto, descrive obiettivi generali: "Adatta questo documento per un pubblico internazionale non tecnico."

Il modello deve inferire sempre più informazioni da solo. Nel primo caso, basta applicare una traduzione letterale. Nel secondo, deve comprendere cosa significa "tono formale" e "terminologia tecnica" nel contesto specifico. Nel terzo, deve decidere autonomamente cosa semplificare, cosa omettere, come ristrutturare.

Questa delega progressiva richiede fiducia crescente nel sistema. L'utente deve sentirsi sicuro che il modello comprenda veramente l'obiettivo alto-livello e non produrrà risultati completamente sbagliati. È un equilibrio delicato tra automazione e controllo.

8.5.3 Sfide nella Human-AI Interaction

Questi nuovi pattern introducono sfide significative che non hanno ancora soluzioni consolidate.

Calibrazione della fiducia

Gli utenti tendono a posizionarsi agli estremi. Alcuni mostrano **over-trust**: accettano output senza verifica, assumono che il modello abbia

sempre ragione, e delegano anche decisioni critiche senza controllo umano. Questo è pericoloso perché gli LLMs fanno errori, anche gravi, ma li presentano con la stessa sicurezza di risposte corrette. Un utente che si fida ciecamente non rileva questi errori.

Altri mostrano **under-trust**: ignorano suggerimenti potenzialmente validi, verificano ossessivamente ogni minimo dettaglio anche ovvio, e sostanzialmente non sfruttano le capacità del sistema. Questo è inefficiente e frustrante, e porta l'utente ad abbandonare lo strumento.

Il problema di fondo è che gli LLMs non comunicano bene la propria incertezza. Quando sono sicuri e quando no, producono output identicamente fluido e sicuro. Non dicono “forse” o “potrei sbagliarmi”. L'utente non ha feedback chiaro su quando fidarsi e quando verificare.

Il design challenge qui è: come rendere l'incertezza visibile e intuitiva? Soluzioni possibili includono confidence scores mostrati accanto all'output, evidenziazione di parti incerte (“questa informazione potrebbe non essere aggiornata”), o presentazione di alternative esplicite (“ho trovato tre interpretazioni diverse della tua domanda”). Ma nessuna di queste è perfetta, e il rischio di sovra-complicare l'interfaccia è reale.

Responsabilità e accountability

Quando un LLM genera contenuto problematico, etico, discriminatorio, o semplicemente sbagliato, chi è responsabile? L'utente che ha fatto la domanda? Il creatore del modello che non ha previsto e mitigato quel caso? L'organizzazione che ha deciso di deployare il sistema?

Questa domanda non ha risposte facili. In sistemi tradizionali, la catena di responsabilità è più chiara: il programmatore scrive codice specifico, che viene testato, revisionato, e deployato. Se quel codice causa danni, si può risalire a chi lo ha scritto e perché. Con gli LLMs, il comportamento emerge da pattern statistici appresi da miliardi di esempi. Non c'è una singola linea di codice che genera una specifica risposta problematica.

Come tracciare la catena decisionale in questi sistemi? Come permettere

agli utenti di fare appello quando ricevono output ingiusti o dannosi? Quali meccanismi di trasparenza e audit sono necessari? Queste sono domande aperte che richiedono sia soluzioni tecniche che policy regolamentari.

Opacità del sistema

Gli LLMs sono fondamentalmente **black box**. L'utente vede input e output, ma non ha visibilità su cosa succede in mezzo. Non sa perché il modello ha dato quella specifica risposta invece di un'altra. Non sa su quali dati è stato addestrato, quindi non può valutare potenziali bias. Non sa come funziona internamente, quindi non può intuire i suoi limiti.

Questa opacità crea problemi multipli. Rende difficile debuggare: se l'output è sbagliato, come capire perché? Rende difficile fidarsi: come sapere che il sistema non ha bias nascosti? Rende difficile imparare: l'utente non può costruire un modello mentale di come funziona il sistema.

Possibili soluzioni includono tecniche di explainability ("ho risposto così perché nel contesto c'era questa informazione"), transparency reports ("questo modello è stato addestrato su questi dataset, ha questi limiti conosciuti"), e audit trails ("ecco la storia di tutte le risposte generate per query simili"). Ma anche qui, non è chiaro quanto queste soluzioni siano praticabili senza sovra-complicare l'interazione.

Deskilling

C'è un rischio reale che gli utenti perdano competenze fondamentali se delegano troppo all'AI. Se uno studente usa sempre l'AI per scrivere saggi, svilupperà mai capacità di scrittura proprie? Se un programmatore genera sempre codice via AI, imparerà mai a programmare veramente? Se un analista usa sempre l'AI per pensiero critico, manterrà quella capacità?

Questo fenomeno, chiamato *deskilling*, non è nuovo. È successo con le calcolatrici (capacità di calcolo mentale diminuite), con i GPS (capacità di orientamento spaziale ridotte), con i correttori ortografici (ortografia peggiorata). Ma gli LLMs operano a un livello più alto: non assistono solo calcoli o memorie, ma processo di pensiero, creatività, ragionamento.

Il bilanciamento ideale è usare l'AI come **amplificatore** di capacità esistenti, non come sostituto. L'AI dovrebbe permettere a chi già sa scrivere di scrivere meglio e più velocemente, non sostituire completamente l'atto di scrivere. Dovrebbe aiutare chi programma a esplorare soluzioni più rapidamente, non eliminare la necessità di capire il codice. Ma ottenere questo bilanciamento richiede sia design attento dell'interfaccia, sia educazione degli utenti su come usare questi strumenti responsabilmente.

8.5.4 Design principles per interfacce LLM

Data la complessità di queste sfide, emergono alcuni principi di design che possono guidare la creazione di interfacce LLM più responsabili ed efficaci.

Il primo principio cruciale è **rendere l'incertezza visibile**. Gli LLMs generano output con diversi livelli di confidenza, ma tipicamente non lo comunicano. Un sistema ben progettato dovrebbe mostrare esplicitamente quando il modello è sicuro e quando no. Questo può avvenire attraverso *confidence scores* numerici, evidenziazione visiva di parti incerte (forse con colori o annotazioni), o presentazione esplicita di alternative ("ecco tre possibili interpretazioni della tua domanda, quale intendevi?"). L'obiettivo non è scoraggiare l'uso, ma calibrare appropriatamente le aspettative dell'utente.

Il secondo principio è **supportare l'iterazione**. Raramente il primo output è perfetto. Il sistema dovrebbe facilitare il processo di raffinamento: bottoni per "prova di nuovo con più dettaglio", "riformula in modo più conciso", "spiega il ragionamento". Dovrebbe mantenere una storia delle varianti generate, permettendo all'utente di confrontarle e combi-

narle. Funzionalità di undo/redo dovrebbero essere ovvie e accessibili. L'interazione non dovrebbe mai sembrare un vicolo cieco: c'è sempre un modo per migliorare.

Il terzo principio è **educare l'utente**. Non si può assumere che gli utenti capiscano automaticamente cosa gli LLMs possono e non possono fare. Il sistema dovrebbe spiegare attivamente le proprie capacità e limiti: "posso aiutarti a scrivere codice, ma non posso eseguirlo o testarlo"; "posso tradurre testi, ma potrebbero esserci errori in lingue che conosco meno". Dovrebbe fornire esempi concreti di prompt efficaci: "invece di chiedere 'parlami di X', prova 'spiega X in 3 paragrafi concentrandoti su Y'". Dovrebbe mostrare best practices attraverso l'uso, non solo in documentazione separata.

Il quarto principio è **dare controllo** all'utente. Anche quando l'AI è molto capace, l'utente dovrebbe sempre sentire di essere al comando. Parametri come temperatura, lunghezza, livello di dettaglio dovrebbero essere regolabili con controlli intuitivi. Dovrebbe sempre essere possibile intervenire manualmente per correggere o guidare l'output. Una modalità "explain why" dovrebbe permettere all'utente di capire il ragionamento del modello quando necessario. Il sistema dovrebbe essere trasparente su cosa sta facendo ("sto cercando informazioni rilevanti...", "sto generando una risposta basata su...").

Questi principi non risolvono tutti i problemi, ma forniscono una direzione per design più responsabile. L'obiettivo ultimo è un sistema che amplifica le capacità umane mantenendo l'umano informato, in controllo, e consapevole dei limiti.

8.5.5 Casi d'uso emergenti

Man mano che la tecnologia matura e il design migliora, emergono nuovi casi d'uso interessanti che vanno oltre la semplice generazione di testo.

Gli **assistenti personali contestuali** non sono più generici, ma altamente

specializzati. Un assistente per il dominio medico comprende terminologia clinica, linee guida, e protocolli. Un assistente legale conosce precedenti, statuti, e linguaggio contrattuale. Un assistente per ricercatori sa come strutturare paper, citare correttamente, e mantenere rigore metodologico. La specializzazione non riguarda solo la conoscenza, ma anche lo stile di interazione: un assistente per studenti adotta un tono educativo e socratico, uno per manager è conciso e orientato alle decisioni.

Gli LLMs come **cognitive scaffolding** rappresentano un uso ancora più interessante. Invece di produrre output finali, aiutano il processo di pensiero. In sessioni di brainstorming, il modello genera idee iniziali che l'umano può espandere e combinare. Come "devil's advocate" automatico, il modello sfida attivamente le assunzioni dell'utente ("hai considerato che X potrebbe avere l'effetto opposto?"). Nel Socratic questioning, il modello non dà risposte ma pone domande che guidano l'utente a scoprire la soluzione da solo. Questi usi trasformano l'AI da "rispositore" a "facilitatore di pensiero".

La **mediazione culturale e linguistica** va oltre la semplice traduzione letterale. Un buon sistema non traduce parola-per-parola, ma adatta il contenuto per essere culturalmente appropriato. Un riferimento culturale incomprensibile in un'altra cultura viene sostituito con un equivalente locale. L'umorismo basato su giochi di parole viene ri-creato piuttosto che tradotto. Espressioni idiomatiche vengono localizzate. Il modello spiega anche contesti culturali che potrebbero essere opachi ("questa frase fa riferimento a un evento storico specifico in quella cultura"), e adatta il livello di formalità alle norme locali.

Questi casi d'uso mostrano che gli LLMs stanno evolvendo da semplici "generatori di testo" a strumenti sofisticati per amplificare diverse forme di intelligenza umana. La chiave è mantenere l'umano al centro: la tecnologia facilita, media, espande, ma non sostituisce il giudizio, la creatività, e la responsabilità umana.

8.6 Laboratorio (non obbligatorio): Esperimenti pratici con strategie di prompting

Questo laboratorio rappresenta solamente un approfondimento per mostrare empiricamente limiti e funzionalità degli LLMs attraverso esperimenti controllati.

8.6.1 Obiettivi

1. Osservare l'impatto di diverse strategie di prompting
2. Identificare pattern di fallimento comuni
3. Comprendere quando usare tecniche avanzate (CoT, few-shot)
4. Valutare trade-off tra accuratezza e computazione

8.6.2 Setup: Modelli open-source

Utilizzeremo modelli piccoli, facilmente eseguibili su hardware consumer:

- **Qwen2.5-0.5B-Instruct**: 500MB, veloce
- **Phi-3.5-mini-instruct**: 3.8B, qualità migliore

Perché modelli piccoli?

- Accessibili (CPU-only)
- Gratuiti
- Riproducibili

- Mostrano limiti in modo più evidente

8.6.3 Setup tecnico

```
import torch
from transformers import AutoModelForCausalLM,
    AutoTokenizer, pipeline

# Carica modello
model_id = "Qwen/Qwen2.5-0.5B-Instruct"

tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(
    model_id,
    torch_dtype=torch.float16 if torch.cuda.is_available()
        else torch.float32,
    device_map="auto",
    low_cpu_mem_usage=True
)

# Crea pipeline
llm = pipeline(
    "text-generation",
    model=model,
    tokenizer=tokenizer,
    max_new_tokens=300,
    temperature=0.3
)
```

8.6.4 Esperimento 1: Matematica semplice

Task: Problemi aritmetici multi-step

Test 1.1: Zero-shot senza CoT

```
prompt = ""
```

```
Ho 23 mele. Ne mangio 7, poi compro il triplo di quelle mangiate.  
Quante mele ho ora?
```

```
""
```

```
# Output tipico (modello piccolo):
```

```
# "Hai 37 mele." [SBAGLIATO - risposta corretta: 37]
```

Test 1.2: Zero-shot con CoT

```
prompt = ""
```

```
Ho 23 mele. Ne mangio 7, poi compro il triplo di quelle mangiate.  
Quante mele ho ora?
```

```
Ragiona passo-passo prima di rispondere.
```

```
""
```

```
# Output tipico (migliorato):
```

```
# Passo 1: Inizio con 23 mele
```

```
# Passo 2: Mangio 7 mele:  $23 - 7 = 16$ 
```

```
# Passo 3: Compro il triplo di 7:  $7 \times 3 = 21$ 
```

```
# Passo 4: Totale:  $16 + 21 = 37$ 
```

```
# Risposta: 37 mele
```

Osservazione: CoT migliora significativamente l'accuratezza.

Test 1.3: Few-shot con esempi

```
prompt = ""
```

```
Risolvi questi problemi:
```

Esempio 1:

Ho 10 caramelle, ne mangio 3, poi ne compro 5.

Quante ne ho ora?

Soluzione: $10 - 3 + 5 = 12$

Esempio 2:

Ho 20 euro, spendo 8 euro, poi guadagno il doppio di quanto speso.

Quanti euro ho ora?

Soluzione: $20 - 8 + (8 \times 2) = 20 - 8 + 16 = 28$

Ora risolvi:

Ho 23 mele. Ne mangio 7, poi compro il triplo di quelle mangiate.

Quante mele ho ora?

"""

Output: più affidabile

8.6.5 Esperimento 2: Classificazione e Sentiment

Test 2.1: Zero-shot classificazione

```
prompt = """
```

```
Classifica il sentiment: Positivo, Negativo, o Neutro.
```

```
Testo: "Il prodotto funziona ma la confezione era danneggiata."
```

```
Sentiment:
```

```
"""
```

Potrebbe dare risposte inconsistenti

Test 2.2: Few-shot con esempi calibrati

```
prompt = ""
Classifica il sentiment come Positivo, Negativo, o Neutro.

"Fantastico, lo ricomprirei!" → Positivo
"Non funziona, pessimo acquisto." → Negativo
"Va bene, niente di speciale." → Neutro
"Ottimo prezzo ma qualità scarsa." → Neutro
"Arrivato rotto, rimborso immediato." → Negativo

"Il prodotto funziona ma la confezione era danneggiata."
→
""

# Output più consistente
```

8.6.6 Esperimento 3: Allucinazioni

Test 3.1: Domande fattuali oltre il training cutoff

```
# Modello con cutoff 2023
prompt = "Chi ha vinto il campionato di Serie A 2024-2025?"

# Output tipico: inventa una risposta plausibile
# "L'Inter ha vinto il campionato..." [potenzialmente falso]
```

Test 3.2: Domande su entità inesistenti

```
prompt = ""
Parlami del romanzo "I confini dell'alba"
di Haruki Murakami.
""

# Il libro non esiste, ma il modello potrebbe:
# - Inventare una trama
```

- # - Descrivere temi "tipici" di Murakami
- # - Generare citazioni false

Mitigazione: Richiesta esplicita di incertezza

```
prompt = ""  
Parlami del romanzo "I confini dell'alba"  
di Haruki Murakami.
```

```
Se non sei sicuro o il libro non esiste, dillo chiaramente.  
""
```

```
# Output migliorato:  
# "Non ho informazioni su un libro con questo titolo  
# di Murakami..."
```

8.6.7 Esperimento 4: Effetto della temperatura

Test 4.1: Temperatura bassa (0.1)

```
temperature = 0.1  
prompt = "Scrivi una breve storia su un gatto."  
  
# Output: molto consistente tra run multiple  
# Tende a generare storie simili/ripetitive
```

Test 4.2: Temperatura alta (1.0)

```
temperature = 1.0  
prompt = "Scrivi una breve storia su un gatto."  
  
# Output: varia significativamente tra run  
# Più creativo ma potenzialmente meno coerente
```

8.6.8 Esperimento 5: Robustezza a variazioni prompt

Test 5.1: Sensibilità a frasi

```
# Variante 1
"Classifica il sentiment di questo testo:"

# Variante 2
"Qual è il sentiment espresso qui?"

# Variante 3
"Questo testo è positivo, negativo o neutro?"

# Possono dare risultati diversi!
```

Test 5.2: Ordine degli esempi (few-shot)

```
# Ordine A: esempi positivi prima
# Ordine B: esempi negativi prima
# Ordine C: alternati

# L'ordine può influenzare il bias del modello
```

8.6.9 Esperimento 6: Limiti di ragionamento

Test 6.1: Ragionamento controfattuale

```
prompt = """
Se l'acqua bollisse a 50°C invece che 100°C,
cosa cambierebbe nella cucina quotidiana?
"""

# Modelli piccoli faticano con scenari controfattuali
# Potrebbero dare risposte superficiali o incoerenti
```

Test 6.2: Logica multi-step

```
prompt = ""  
Tutti i gatti sono animali.  
Alcuni animali volano.  
Quindi: alcuni gatti volano?
```

```
Ragiona logicamente.  
""
```

Senza CoT, può sbagliare sillogismi semplici

8.6.10 Metriche di valutazione

Per ciascun esperimento, traccia:

- **Accuratezza:** % risposte corrette
- **Consistenza:** similarità tra run multiple
- **Latenza:** tempo di generazione
- **Failure modes:** tipi di errori osservati

8.6.11 Deliverable del laboratorio

1. **Report:** Documenta risultati di tutti gli esperimenti
2. **Notebook:** Codice riproducibile
3. **Analisi:** Pattern di successo/fallimento
4. **Best practices:** Linee guida per prompting efficace

8.6.12 Domande di riflessione

- Quando vale la pena usare CoT vs. zero-shot?
- Come bilanciare accuratezza e costo computazionale?
- Quali task sono inadatti per LLMs (anche grandi)?
- Come progettare prompt robusti a variazioni?
- Quali strategie mitigano le allucinazioni?

8.7 Conclusioni

Gli LLMs rappresentano un cambio di paradigma nell'interazione human-computer. Non sono semplicemente "strumenti più potenti", ma introducono una modalità fondamentalmente nuova di comunicazione con i sistemi.

8.7.1 Takeaway chiave

1. Il **prompting** è un'abilità che richiede pratica e iterazione
2. Gli LLMs hanno **limiti strutturali** (allucinazioni, bias, ragionamento)
3. Le **strategie avanzate** (CoT, few-shot) migliorano significativamente le performance
4. La **human-AI interaction** richiede nuovi principi di design
5. La **responsabilità** nell'uso di LLMs è cruciale

8.7.2 Direzioni future

- **Multimodalità:** integrazione testo-immagine-audio
- **Agenticità:** LLMs come agenti autonomi con strumenti
- **Personalizzazione:** modelli adattati a utenti/domini specifici
- **Efficienza:** modelli più piccoli con capacità maggiori
- **Alignment:** modelli sempre più allineati con valori umani

8.7.3 Messaggio finale

Gli LLMs non sono “intelligenza artificiale generale”. Sono sistemi potenti ma limitati, che richiedono:

- Comprensione dei loro meccanismi
- Uso consapevole e critico
- Verifica degli output
- Considerazione etica

La vera potenza degli LLMs emerge quando vengono usati come **amplificatori dell'intelligenza umana**, non come sostituti.

9 Valutazione per la terza parte del corso di TLN (2025/2026)

9.1 La mia idea di valutazione

Valutare la conoscenza nel Natural Language Processing è un esercizio complicato. Questo capitolo cerca di spiegare bene la mia visione, senza usare un freddo elenco di criteri. Volendo riassumere la mia *idea* di valutazione, sarebbe la seguente:

L'obiettivo non è superare l'esame. L'obiettivo è diventare qualcuno che sa fare NLP — che sa quando usare BERT e quando bastano SVM, che capisce perché un topic model ha fallito, che può spiegare trade-off complessi a non-tecnici, che costruisce sistemi robusti invece di demo fragili.

L'esame è strutturato per testare questo tipo di competenza profonda, non la memorizzazione superficiale. Vediamo come.

9.2 Struttura dell'esame

L'esame è composto da due parti complementari, ciascuna progettata per valutare aspetti diversi ma ugualmente cruciali:

1. **Parte Teorico-Critica:** Domande sulla parte descrittiva e concettuale del corso
2. **Parte Laboratoriale:** Discussione degli esercizi pratici implementati (Lab- n più il progetto esteso)

9.3 Parte I: Teoria e pensiero critico

9.3.1 Cosa viene valutato (e perché)

La prima parte dell'esame non testa se avete memorizzato le slide. Testa se avete **internalizzato** i concetti al punto da poterli usare come strumenti di pensiero.

Tre criteri guidano la valutazione:

1. Linguaggio tecnico preciso

Cosa cerco: la capacità di usare terminologia tecnica con precisione, senza vaghe approssimazioni.

Perché conta: Nell'NLP, le parole hanno significati precisi. "Embedding" non è un sinonimo generico di "rappresentazione". "Attention" non significa "il modello guarda qualcosa". TF-IDF non è "una specie di frequenza delle parole". La precisione lessicale riflette precisione concettuale.

2. Chiarezza concettuale e contestuale

Cosa cerco: idee precise e chiare sulle definizioni, e comprensione del **contesto storico, metodologico e applicativo** in cui quei concetti vivono.

Perché conta: Un concetto isolato è sterile. Capire "cosa è Word2Vec" senza capire *perché* fu rivoluzionario nel 2013, quali problemi risolveva rispetto a bag-of-words, quali limiti aveva che BERT poi ha superato è memorizzazione, non comprensione.

3. Pensiero comparativo e integrativo

Cosa cerco: la capacità di **mettere in dialogo** concetti diversi, confrontare approcci, valutare trade-off, costruire ponti tra aree apparentemente separate del corso.

Perché conta: Questa è la competenza che distingue un esperto da un novizio. Il novizio sa definire cos'è un embedding. L'esperto sa quando Word2Vec è sufficiente e quando serve BERT contestuale, perché TF-IDF potrebbe ancora battere embeddings neurali su certi task di retrieval, come gli embeddings si inseriscono in pipeline downstream (classificazione, clustering, semantic search), quali assunzioni porta ciascun metodo (Word2Vec assume stazionarietà del significato, BERT cattura polisemia contestuale ma è computazionalmente costoso) e come queste limitano l'applicabilità in scenari reali.

9.4 Parte II: Laboratorio e Implementazione

9.4.1 Cosa viene valutato (e perché)

La seconda parte dell'esame parte dagli esercizi di laboratorio che avete implementato. Ma non vi chiedo di recitare il codice a memoria. Vi chiedo di dimostrare **ownership** - che quel codice capite cosa fa, perché lo fa così, e cosa potreste fare diversamente.

Tre criteri:

1. Conoscenza personale del codice

Cosa cerco: Capacità di navigare il codice, spiegare sezioni specifiche, modificare al volo, rispondere a "cosa succederebbe se...".

Perché conta: Chiunque può copiare un notebook da GitHub e farlo girare. Ma se non capisci *perché* quella riga di codice è lì, cosa romperebbe se la togliessi, quale assunzione sottende, è un problema.

Durante l'esame, potrei chiedere:

- “Mostrami dove fai embedding dei documenti. Quale modello hai usato? Perché quello?”
- “Qui usi `n_components=5` in UMAP. Cosa succederebbe con 15? Quando vorresti più componenti?”
- “Vedo che hai `min_cluster_size=50`. Come hai scelto questo valore? Hai sperimentato con altri numeri?”

2. Correttezza, semplicità, leggibilità

Cosa cerco: Codice che funziona, ma anche codice **leggibile**, con nomi variabili sensati, struttura chiara, commenti dove servono (e silenzio dove sono ovvi).

Perché conta: Nel mondo reale, il codice è letto 10 volte più spesso di quanto è scritto. Codice funzionante ma incomprensibile crea problemi.

3. Profondità di analisi dei risultati

Cosa cerco: Questa è la parte che separa davvero competenza superficiale da profonda. Non basta dire “ho ottenuto 15 topic”. Devi saper dire: “Ho ottenuto 15 topic, ma 3 sembrano overlap semantico, 2 sono rumore probabilmente, il topic 7 è sorprendentemente coerente e cattura X che non mi aspettavo. Ho provato `min_cluster_size=100` e sono scesi a 10 topic più stabili ma ho perso granularità su Y. Il trade-off dipende da...”

Perché conta: Questa è scienza sperimentale. Gli esperimenti falliscono, danno risultati inaspettati, rivelano assunzioni sbagliate. La capacità di **riflettere criticamente** su risultati è il cuore del metodo scientifico.

9.5 Criteri di valutazione: griglia sintetica

Per trasparenza, ecco come i criteri si traducono in valutazione:

9.5.1 Parte I: Teoria (circa 50% del voto)

Criterio	Insufficiente	Eccellente
Linguaggio tecnico	Termini vaghi, imprecisi, o usati scorrettamente	Terminologia precisa, usata con consapevolezza di sfumature
Chiarezza concettuale	Definizioni isolate, senza contesto storico/metodologico	Concetti contestualizzati, con motivazioni, assunzioni, limiti
Pensiero comparativo	Risposte isolate per ogni domanda, nessuna connessione	Capacità di confrontare, integrare, valutare trade-off tra approcci

9.5.2 Parte II: Laboratorio (circa 50% del voto)

Criterio	Insufficiente	Eccellente
Ownership codice	Non sa spiegare sezioni, "ho copiato da tutorial"	Naviga fluentemente, spiega scelte, modifica al volo
Qualità codice	Funziona ma illeggibile, nomi variabili opachi	Leggibile, strutturato, commenti appropriati
Analisi risultati	"Ho ottenuto X topic" senza riflessione	Validazione qualitativa, comparazione configurazioni, consapevolezza limiti

10 Conclusioni: Semantica, Conoscenza e Futuro del NLP

10.1 Il percorso compiuto

Questo corso ha tracciato un percorso attraverso diverse prospettive sulla comprensione computazionale del linguaggio. Non si è trattato di una semplice sequenza di tecniche, ma di un viaggio concettuale che ha esplorato come possiamo catturare, rappresentare e generare significato attraverso sistemi computazionali.

Abbiamo iniziato dalla semantica computazionale nella sua triplice articolazione: il significato come inventario strutturato (semantica lessicale), come condizioni di verità (semantica formale), e come distribuzione nell'uso (semantica distribuzionale). Questi tre approcci non sono alternativi ma complementari, e la loro integrazione rimane una sfida aperta. La semantica lessicale ci fornisce struttura e organizzazione esplicita; la semantica formale ci dà rigore e composizionalità; la semantica distribuzionale ci offre flessibilità e capacità di apprendimento dai dati. I sistemi moderni, inclusi i Large Language Models, in realtà combinano implicitamente tutti e tre questi paradigmi, anche se in modi che non sempre comprendiamo completamente.

Le definizioni, oggetto del secondo capitolo, rappresentano un ponte cruciale tra conoscenza esplicita e computazione. Abbiamo visto come le definizioni non siano semplici stringhe di testo, ma strutture complesse che codificano relazioni semantiche, vincoli d'uso, e knowledge organization. La distinzione tra approccio semasiologico (dal termine al significato) e onomasiologico (dal concetto al termine) ci ha mostrato

che la direzione del processo semantico non è neutrale: cercare il significato di una parola è qualitativamente diverso dal cercare la parola per esprimere un concetto. Questo diventa particolarmente rilevante quando integriamo sistemi di retrieval con modelli generativi, come nel caso del RAG.

Il terzo capitolo ha affrontato temi che vanno oltre la pura tecnica: privacy, rilascio dati, e disinformazione sono problemi semantici tanto quanto computazionali. La capacità di identificare e mitigare misinformation richiede comprensione profonda del linguaggio, dei bias, e dei meccanismi di fiducia. L'explainability non è un lusso ma una necessità etica: gli utenti hanno diritto di capire perché un sistema produce certe risposte, specialmente quando quelle risposte influenzano decisioni importanti. Il progetto PRIN KURAMi (<https://kurami.disco.unimib.it>) menzionato a lezione e discusso in quel capitolo, mostra come l'annotazione assistita possa aumentare la consapevolezza degli utenti.

La polisemia, analizzata nel quarto capitolo, ci ha rivelato che l'ambiguità non è un bug ma una feature fondamentale del linguaggio umano. Non tutte le parole polisemiche sono ugualmente complesse: la struttura della polisemia, la distribuzione dei sensi, la separabilità semantica variano enormemente. Abbiamo visto che questa complessità non è universale ma dipende dalla lingua, dalla cultura, dal dominio. Gli embeddings contestuali catturano questa variazione in modi sofisticati, ma la sfida di disambiguare rimane aperta, specialmente quando i sensi non sono nettamente separati ma formano un continuum.

Il concetto di basic level, esplorato nel quinto capitolo, ci ha mostrato che esiste una granularità privilegiata nella concettualizzazione: né troppo generica né troppo specifica, ma cognitivamente naturale. Questa nozione, derivata dalla psicologia cognitiva, ha implicazioni pratiche profonde per text simplification, language learning, e human-computer interaction. Non esiste però un basic level universale: ciò che è basilico dipende dal contesto, dalla competenza dell'utente, dal dominio di applicazione. Riconoscere e modellare questa variabilità rimane una sfida aperta.

Il sesto capitolo ha tracciato la storia del NLP pre-LLM, mostrando come siamo passati da approcci simbolici a statistici, poi neurali, e infine ai Transformer. Questa non è stata una semplice successione di paradigmi: ogni era ha lasciato contributi concettuali e metodologici che informano ancora il presente. Il text mining, con task come clustering, classificazione, topic modeling, e information retrieval, rappresenta un patrimonio di tecniche che non sono scomparse con l'avvento degli LLM, ma si sono integrate in architetture più complesse. La lezione chiave è che il progresso non cancella il passato, ma lo stratifica.

Il RAG, discusso nel settimo capitolo, rappresenta un punto di convergenza tra retrieval classico e generazione neurale. Invece di affidarsi solo alla memoria parametrica (limitata e statica) degli LLM, ancoriamo la generazione a documenti esterni recuperati dinamicamente. Questa architettura non solo mitiga le allucinazioni e aggiorna la conoscenza, ma introduce anche verificabilità: possiamo tracciare da dove provengono le informazioni. Il RAG non è una soluzione perfetta, ma un paradigma che riconosce i limiti degli LLM puri e li complementa con accesso esplicito a knowledge bases.

Infine, l'ottavo capitolo ha affrontato gli LLM stessi: le loro capacità impressionanti, i loro limiti strutturali, e soprattutto il modo in cui trasformano l'interazione human-AI. Il prompting non è una tecnica superficiale ma un'abilità complessa che richiede comprensione dei modelli, iterazione, e creatività. Gli LLM introducono nuovi pattern di interazione (co-creazione, debugging conversazionale, delega progressiva) ma anche nuove sfide (calibrazione della fiducia, responsabilità, opacità, deskilling). La human-AI interaction non è più mediata da interfacce rigide ma da conversazioni aperte, con tutti i benefici e i rischi che questo comporta.

10.2 Temi trasversali

Attraversando questi capitoli, emergono temi ricorrenti che vale la pena esplicitare.

10.2.1 La tensione tra simbolico e subsimbolico

Il NLP contemporaneo vive una tensione produttiva tra rappresentazioni simboliche esplicite (ontologie, grafi di conoscenza, definizioni formali) e rappresentazioni distribuite subsimboliche (embeddings, neuroni, layer). I sistemi puramente simbolici offrono interpretabilità e controllo ma faticano a scalare e gestire l'ambiguità. I sistemi puramente neurali apprendono pattern complessi dai dati ma sono opachi e difficili da debuggare. La direzione più promettente sembra essere l'integrazione: neuro-symbolic AI che combina apprendimento statistico con reasoning strutturato. Il RAG è un esempio di questa integrazione: retrieval esplicito su documenti strutturati combinato con generazione neurale.

10.2.2 Conoscenza implicita vs esplicita

Gli LLM codificano enormi quantità di conoscenza nei loro parametri, ma questa conoscenza è implicita, non accessibile direttamente, non verificabile. Le knowledge bases tradizionali (WordNet, Wikidata, ontologie di dominio) codificano conoscenza esplicita, strutturata, verificabile, ma statica e costosa da mantenere. Anche qui, l'integrazione sembra la via: sistemi ibridi che usano LLM per popolare e aggiornare knowledge bases, e knowledge bases per guidare e verificare la generazione degli LLM.

10.2.3 Task-specific vs general-purpose

Il NLP pre-LLM era prevalentemente task-specific: modelli diversi per sentiment analysis, NER, POS tagging, parsing, translation, etc. Gli LLM sono sistemi general-purpose che risolvono task multipli attraverso prompting. Questo ha vantaggi enormi (flessibilità, zero-shot, riduzione costi di sviluppo) ma anche svantaggi (performance non ottimale su task specifici, difficoltà di controllo, consumo risorse). La tendenza

attuale è verso sistemi ibridi: LLM come backbone generale, fine-tuned o augmented per task specifici.

10.2.4 Dati vs modelli

Una domanda ricorrente: cosa conta di più, i dati o i modelli? La risposta evolve. Nell'era statistica, i dati erano tutto: "there's no data like more data". Nell'era deep learning, l'architettura divenne cruciale: Transformer ha battuto LSTM non per più dati ma per design migliore. Nell'era LLM, la scala di entrambi è essenziale: modelli enormi su dataset enormi. Ma emergono tecniche che sfidano questa narrativa: few-shot learning, parameter-efficient fine-tuning, retrieval-augmented generation. Forse la domanda stessa è mal posta: dati e modelli co-evolvono, e l'ottimizzazione di uno senza l'altro è limitante.

10.2.5 Evaluation: il problema della misura

Come valutiamo un sistema NLP? La risposta dipende dal task, dal contesto, dagli stakeholder. Metriche automatiche (BLEU, ROUGE, F1, perplexity) sono riproducibili ma spesso correlano male con qualità percepita. Valutazioni umane catturano sfumature ma sono costose, soggettive, non scalabili. Valutazioni estrinseche (task downstream) misurano utilità reale ma confondono molteplici fattori. Non esiste una metrica perfetta, e la triangolazione (combinare metodi diversi) è essenziale. Con gli LLM, la sfida si amplifica: come valutiamo creatività, faithfulness, harmlessness? Nuove metriche emergono (instruction following, toxicity detection, factual accuracy) ma rimangono imperfette.

10.2.6 Multilingualità e universalità

L'NLP ha storicamente favorito l'inglese: più dati, più ricerca, più risorse. Gli LLM multilingue (mBERT, XLM-R, mT5) hanno migliorato la

situazione, ma asimmetrie persistono. Concetti che sembrano universali (basic level, polisemia, definizioni) variano cross-linguisticamente in modi sottili ma significativi. Un sistema addestrato principalmente su inglese potrebbe non generalizzare bene ad altre lingue con strutture semantiche diverse. La vera multilingualità richiede non solo traduzione ma comprensione di variazione culturale e concettuale.

10.3 Sfide aperte

Nonostante i progressi impressionanti, l’NLP affronta sfide fondamentali che rimangono irrisolte.

10.3.1 Comprensione profonda vs pattern matching

Gli LLM eccellono nel riconoscere pattern statistici su scale enormi, ma quanto comprendono veramente? Possono completare frasi fluentemente, ma capiscono la causalità? Possono generare codice corretto, ma hanno un modello mentale di come funziona? Possono tradurre testi, ma colgono sfumature pragmatiche e culturali? Il dibattito su cosa significhi “comprensione” per un sistema computazionale è filosofico tanto quanto tecnico, ma ha implicazioni pratiche: se un modello non comprende veramente, i suoi errori saranno sistematici e imprevedibili.

10.3.2 Groundedness e riferimento al mondo

Il linguaggio umano è *grounded*: le parole si riferiscono a oggetti, eventi, relazioni nel mondo fisico e sociale. Gli LLM sono addestrati solo su testo, senza esperienza diretta del mondo. Questo crea problemi: possono descrivere come “preparare il caffè” senza aver mai visto una macchinetta; possono parlare di “gravità” senza aver mai visto oggetti cadere. I sistemi multimodali (che integrano testo, immagini, audio,

video) stanno affrontando questa limitazione, ma siamo ancora lontani da una vera *grounded understanding*.

10.3.3 Ragionamento e composizionalità

Il linguaggio umano è *compositional*: il significato di frasi complesse emerge dalla combinazione sistematica di parti più semplici. Gli LLM mostrano capacità composizionali emergenti, ma limitate. Faticano con ragionamento multi-step, specialmente quando si richiede backtracking o revisione di assunzioni. Chain-of-Thought prompting aiuta, ma non risolve il problema fondamentale: i Transformer processano sequenze, non alberi sintattici o grafi semantici. Architetture che integrano struttura esplicita potrebbero essere necessarie ed emergenti nel futuro della ricerca.

10.3.4 Long-term memory e learning continuo

Gli LLM hanno memoria contestuale (il prompt, eventualmente arricchito con RAG) e memoria parametrica (i pesi addestrati), ma nessuna memoria episodica a lungo termine. Non ricordano conversazioni precedenti, non apprendono incrementalmente dall'uso, non costruiscono knowledge bases personali. Ogni interazione è isolata. Sistemi con memoria esterna (database, knowledge graphs, retrieval systems) mitigano questo limite, ma l'integrazione profonda di learning continuo e memoria episodica rimane aperta.

10.3.5 Efficienza e sostenibilità

Il training di LLM all'avanguardia richiede risorse computazionali immense, con costi economici ed ambientali significativi. L'inference non è trascurabile, specialmente su larga scala. La ricerca su modelli più efficienti (distillation, pruning, quantization, sparse models) è cruciale.

Ma oltre l'efficienza computazionale, c'è la questione di sostenibilità sociale: chi può permettersi di addestrare questi modelli? Chi controlla i dati? Come garantire accesso equo? Questi non sono solo problemi tecnici ma politici ed etici.

10.3.6 Robustezza e adversarial attacks

Gli LLM sono vulnerabili a input adversariali: prompt appositamente costruiti per elicitare comportamenti indesiderati, estrarre informazioni private, o bypassare safety filters. La robustness a questi attacchi è difficile perché lo spazio degli input è infinito e creativo. Tecniche come red teaming, adversarial training, e constitutional AI aiutano, ma è una corsa continua tra attaccanti e difensori.

10.3.7 Alignment e valori

Come assicuriamo che gli LLM siano allineati con valori umani? RLHF è un passo, ma quali valori? Chi li definisce? Valori universali non esistono: culture, società, individui hanno priorità diverse. Un modello addestrato prevalentemente su testi occidentali riflette quei valori. Come costruiamo sistemi che rispettano diversità culturale pur evitando relativismo morale su questioni fondamentali (diritti umani, dignità, giustizia)? Questa è una sfida profondamente interdisciplinare che richiede filosofia, etica, sociologia, oltre all'Informatica.

10.4 Direzioni future

Guardando avanti, diverse direzioni sembrano particolarmente promettenti.

10.4.1 Agenticità e tool use

Gli LLM stanno evolvendo da generatori passivi ad agenti attivi che possono usare strumenti esterni: eseguire codice, interrogare database, navigare il web, invocare API. Sistemi come AutoGPT, LangChain, ecc. mostrano questa direzione. L'agenticità introduce nuove capacità ma anche nuovi rischi: un agente autonomo può fare errori con conseguenze reali. Il design di safety constraints e human-in-the-loop mechanisms è essenziale.

10.4.2 Multimodalità profonda

I sistemi multimodali attuali (GPT-4V, Gemini, CLIP) integrano visione e linguaggio, ma spesso in modo superficiale. La vera multimodalità richiederebbe rappresentazioni unificate dove concetti visivi, linguistici, uditivi, e tattili coesistono e si informano reciprocamente. Questo avvicinerebbe i sistemi alla cognizione umana *embodied*, dove il significato emerge dall'interazione sensorimotoria con il mondo.

10.4.3 Personalizzazione e contextualizzazione

Gli LLM attuali sono general-purpose, ma il futuro potrebbe vedere modelli personali che si adattano a utenti specifici, ricordando preferenze, apprendendo stili comunicativi. Questo solleva questioni di privacy (quanti e quali dati personali immagazzinare?) e lock-in (quanto dipendente si diventa da un assistente personalizzato?), ma offre anche UX superiore e assistenza più utile.

10.4.4 Neuro-symbolic integration

Combinare forza degli approcci neurali (apprendimento dai dati, robustezza, generalizzazione) con vantaggi degli approcci simbolici (inter-

pretabilità, reasoning, verificabilità) è una frontiera attiva. Knowledge graphs aumentati con embeddings, logical reasoning guidato da LLM, program synthesis con neural networks sono tutte direzioni promettenti.

10.4.5 Modelli small e più efficienti

Mentre i modelli *leader* diventano sempre più grandi, c'è una tendenza parallela verso modelli più piccoli ma altamente efficienti. Distillation, quantization, mixture-of-experts, sparse activation permettono di comprimere capacità in footprint ridotti. Modelli come Phi-2 o Mistral mostrano che architetture intelligenti e dati di qualità possono competere con modelli 10x più grandi.

10.4.6 Evaluation e benchmarking next-generation

I benchmark attuali (GLUE, SuperGLUE, MMLU) saturano rapidamente. Servono valutazioni che testino capacità più profonde: reasoning causale, planning a lungo termine, robustezza a distributional shift, comprensione pragmatica, creatività genuina. Benchmark dinamici che evolvono e si adattano (per evitare overfitting) sono una direzione interessante.

10.5 Riflessioni metodologiche

Oltre ai contenuti tecnici, questa terza parte del corso TLN ha cercato di trasmettere un approccio metodologico e variegato all'NLP.

10.5.1 Pensiero critico sui sistemi

Non basta sapere come usare un tool o una libreria. Serve capire le assunzioni sottostanti, i limiti, i trade-off. Quando usiamo un LLM,

chiediamoci: su quali dati è stato addestrato? Quali bias potrebbe avere? Quando tende a fallire? Quando usiamo un dataset, chiediamoci: come è stato raccolto? Quanto è rappresentativo? Quali artefatti potrebbe contenere? Il pensiero critico porta ad analisi costruttive.

10.5.2 Iterazione e sperimentazione

L’NLP è empirico. Le ipotesi vanno testate, i modelli vanno validati, i risultati vanno replicati. Ma sperimentazione non significa trial-and-error cieco: serve design sperimentale attento, controllo delle variabili, interpretazione rigorosa. La cultura della riproducibilità (condividere codice, dati, procedure) è essenziale per un progresso cumulativo.

10.5.3 Interdisciplinarietà

L’NLP non è solo computer science. Incorpora linguistica (fonologia, morfologia, sintassi, semantica, pragmatica), psicologia cognitiva (memoria, attenzione, categorizzazione), filosofia del linguaggio (riferimento, significato, verità), sociologia (variazione sociale, potere, identità), etica (bias, fairness, privacy). Un ricercatore NLP completo deve navigare questi domini o almeno riconoscere quando servono expertise esterne.

10.5.4 Consapevolezza storica

Molte “nuove” idee hanno radici profonde. Gli embeddings distribuiti echeggiano i vector space models degli anni ’70, come abbiamo visto in qualche capitolo precedente. Il prompting ricorda i template-based systems. L’attention mechanism ha precedenti come le memory networks. Conoscere la storia aiuta a capire cosa è genuinamente nuovo e cosa è re-packaging. Aiuta anche a evitare di ripetere errori del passato.

10.5.5 Responsabilità etica

La tecnologia NLP ha impatto reale su persone reali. Sistemi di sentiment analysis influenzano assunzioni, sistemi di moderazione decidono cosa è visibile online, sistemi di traduzione mediano comunicazione cross-culturale. Le scelte che facciamo (quali dati usare, come valutare, quali applicazioni deployare) hanno conseguenze etiche. Serve quindi una **consapevolezza attiva** di bias, fairness, privacy, accountability.

10.6 Il futuro è conversazione

Se dovessi riassumere in una frase la trasformazione portata dagli LLM, sarebbe questa: abbiamo spostato l'interfaccia human-computer dal codice alla conversazione.

Prima degli LLM, per interagire con un computer serviva tradurre intenzioni in linguaggio formale: command-line, SQL, API calls, GUI clicks. L'utente doveva adattarsi alla macchina. Con gli LLM, la macchina si adatta all'utente: possiamo esprimere obiettivi in linguaggio naturale, raffinare attraverso dialogo, scoprire capacità conversando. Con limiti e rischi, ovviamente.

Questo abbassa drasticamente le barriere d'accesso. Ma introduce anche **responsabilità**: se tutti possono usare l'AI, come preveniamo il *misuse*? Se l'AI diventa così fluente, come distinguiamo verità da finzione? Se deleghiamo tanto, cosa perdiamo in competenza?

La conversazione come interfaccia non è solo più comoda: è qualitativamente diversa. Cambia cosa è possibile, chi può partecipare, come pensiamo al rapporto human-machine. Siamo solo all'inizio dell'esplorazione di queste implicazioni.